

Capgemini Past-Year Coding Questions — Complete Compilation

#	Problem	Topic
1	Majority Element – I	Arrays (POTD)
2	Two Sum	Hashing
3	Longest Substring Without Repeating Characters	Strings
4	Kadane's Algorithm (Maximum Subarray Sum)	Arrays
5	Merge Two Sorted Lists	Linked List
6	Remove N-th Node from the End of a Linked List	Linked List
7	Minimum Window Substring	Strings
8	LRU Cache	Linked List (Design)
9	Trapping Rain Water	Stack & Queues
10	Next Permutation	Arrays
11	Longest Consecutive Sequence in an Array	Hashing
12	Longest Increasing Subsequence	Binary Search
13	Maximum Product Subarray	Arrays
14	Sort an Array of 0's, 1's and 2's	Sorting
15	Check if a Binary Tree is a BST	Binary Search Trees
16	Jump Game – I	Greedy Algorithms
17	Clone a Linked List with Random and Next Pointers	Linked List
18	Add Two Numbers in Linked List	Linked List
19	Largest Rectangle in a Histogram	Stack & Queues
20	Sliding Window Maximum	Stack & Queues

#	Problem	Topic
21	Construct a Binary Tree from Preorder and Inorder Traversals	Hashing (Trees)
22	Implement Min Stack	Stack & Queues
23	Merge Two Sorted Arrays Without Extra Space	Arrays
24	Search in a 2D Matrix – II	Binary Search
25	Find Peak Element	Binary Search
26	Median of Two Sorted Arrays	Binary Search
27	Kth Smallest and Kth Largest Element in a BST	Binary Search Trees
28	Candy	Greedy Algorithms
29	Valid Parenthesis Checker (with Wildcard)	Greedy Algorithms
30	Count Subarrays with Given Sum	Arrays
31	Find the Intersection Point of a Y-Shaped Linked List	Linked List
32	Next Greater Element	Stack & Queues
33	Meeting Rooms	Greedy Algorithms
34	Asteroid Collision	Arrays
35	Single Element in a Sorted Array	Binary Search
36	Finding the Equilibrium Index in an Array	Arrays
37	Lowest Common Ancestor in a BST	Binary Search Trees
38	Single Number – II	Arrays
39	Best Time to Buy and Sell Stock – IV	Dynamic Programming
40	Number of Substrings Containing All Three Characters	Strings
41	Remove K Digits	Stack & Queues
42	Sum of Subarray Minimums	Stack & Queues
43	Two Sum in a BST	Binary Search Trees

#	Problem	Topic
44	Find Out How Many Times the Array Is Rotated	Binary Search
45	Maximum Points You Can Obtain from Cards	Arrays
46	Binary Subarrays with Sum	Hashing
47	Matrix Median	Binary Search
48	Find the Repeating and Missing Number	Sorting
49	Correct a BST with Two Nodes Swapped	Binary Search Trees
50	Job Sequencing Problem	Greedy Algorithms
51	LFU Cache	Linked List (Design)
52	Trie Implementation and Advanced Operations	Tries
53	Largest BST in a Binary Tree	Binary Search Trees
54	KMP Algorithm / LPS Array	Strings
55	Celebrity Problem	Arrays
56	Rabin–Karp Algorithm	Strings
57	Z Function	Strings
58	Build Heap from a Given Array	Arrays
59	Sort a Linked List of 0's, 1's and 2's	Linked List
60	3 Sum	Arrays
61	Longest Repeating Character Replacement	Strings
62	4 Sum	Arrays
63	Longest Subarray with Sum K	Arrays
64	Longest Substring with At Most K Distinct Characters	Strings
65	Next Greater Element – II	Stack & Queues
66	Max Consecutive Ones – III	Arrays
67	Majority Element – II	Hashing

#	Problem	Topic
68	Kth Element of Two Sorted Arrays	Binary Search
69	Best Time to Buy and Sell Stock with Transaction Fee	Dynamic Programming
70	Find Peak Element – II	Binary Search
71	Longest Word with All Prefixes	Tries
72	Shortest Palindrome	Strings
73	3Sum Smaller	Arrays
74	Alt and Tab Simulation	Hashing
75	Armstrong Number	Mathematics
76	Array Upper Bound	Binary Search
77	Number of Jumps	Mathematics
78	Set Difference of Two Arrays	Hashing
79	Rearrange Array Elements by Sign	Arrays
80	Intersection of Two Sorted Arrays	Arrays
81	Move Zeros to End	Arrays
82	Remove Duplicates from Sorted Array	Arrays
83	Union of Two Sorted Arrays	Arrays
84	Automorphic Number	Mathematics
85	Find Row with Maximum 1's	Binary Search
86	Search in a 2D Matrix	Binary Search
87	Lower Bound	Binary Search
88	Search X in a Sorted Array	Binary Search
89	Upper Bound	Binary Search
90	Calculate Frequency of Characters in a String	Hashing
91	Candy Crush	Arrays
92	Check if a Number is a Harshad (Niven) Number	Mathematics
93	Check if a Number is a Strong Number	Mathematics

#	Problem	Topic
94	Check if a Number is an Abundant Number	Mathematics
95	Check if an Array Is a Subset of Another	Hashing
96	Check Whether a Floating-Point Number Is Even or Odd	Mathematics
97	Check Whether a Given Number Is Even or Odd	Mathematics
98	Check Whether a Number Is Positive or Negative	Mathematics
99	Confusing Number	Mathematics
100	Convert a Number to Hexadecimal	Mathematics
101	Convert Decimal to Binary Number	Mathematics
102	Convert Decimal to Octal Number	Mathematics
103	Convert Digits / Numbers to Words	Mathematics
104	Convex Polygon	Mathematics
105	Count Prefix Appearances	Hashing
106	Count the Number of Good Subsequences	Hashing
107	Counting Elements	Hashing
108	Counting Frequencies of Array Elements	Hashing
109	Divide Array Into Increasing Sequences	Hashing
110	Dot Product of Two Sparse Vectors	Arrays
111	MEXX (Minimum Excluded Value)	Hashing
112	Highest Occurring Element in an Array	Hashing
113	Second Highest Occurring Element	Hashing
114	Sum of Highest and Lowest Frequency	Hashing
115	Reverse an Array – 2 (Reverse in Groups)	Arrays
116	Isomorphic Strings	Hashing

#	Problem	Topic
117	Sort Characters by Frequency	Hashing
118	Valid Anagram	Hashing
119	Furthest Empty Plot from Buildings	Arrays
120	Faulty Sensor	Arrays
121	Find All Non-Repeating Elements in an Array	Hashing
122	Find All Repeating Elements in an Array	Hashing
123	Find All Symmetric Pairs	Hashing
124	Find Distinct Expressions	Hashing
125	Flatten 2D Vector	Arrays
126	Intersection of Three Sorted Arrays	Arrays
127	Line Reflection	Hashing
128	Match Alphanumerical Pattern in Matrix – I	Hashing
129	Minimum Unlocked Indices to Sort Nums	Hashing
130	Remove Duplicates from an Unsorted Linked List	Hashing
131	Remove Duplicates from an Unsorted Array	Hashing
132	Analyze User Website Visit Pattern	Sorting
133	Before and After Puzzle	Sorting
134	Best Meeting Point	Sorting
135	Best Time to Buy and Sell Stock with Cooldown	Dynamic Programming
136	Better Compression of String	Sorting
137	Bold Words in String	Tries
138	Campus Bikes	Sorting
139	Capitalize First and Last Character of Each Word	Strings
140	Change Case of Each Character in a String	Strings

#	Problem	Topic
141	Change Every Letter with the Next Lexicographic	Strings
142	Check if String Is Decomposable Into Value-Equal Substrings	Strings
143	Concatenate One String to Another	Strings
144	Count Substrings with Only One Distinct Letter	Strings
145	Count the Number of Words in a Given String	Strings
146	Count Vowels, Consonants, and Spaces in a String	Strings
147	Count Pairs of Equal Substrings with Minimum Difference	Greedy Algorithms
148	Find Pattern in Infinite Stream II	Strings
149	Find the ASCII Value of a Character	Strings
150	Find the Largest Word in a String	Strings
151	Flip Game	Strings
152	Hexspeak	Strings
153	Number of Distinct Substrings in a String	Tries
154	Range Sum in BST	Binary Search Trees
155	Rotate a Linked List	Linked List
156	Delete a Node in BST	Binary Search Trees
157	Inorder Successor and Predecessor in BST	Binary Search Trees
158	Floor and Ceil in a BST	Binary Search Trees
159	Count Number of Nice Subarrays	Mathematics
160	Stock Span Problem	Stack & Queues
161	Sum of Subarray Ranges	Stack & Queues
162	Maximum Rectangles	Stack & Queues

#	Problem	Topic
163	Distinct Substrings	Tries
164	Search in BST	Binary Search Trees
165	Insert a Given Node in BST	Binary Search Trees
166	Construct a BST from a Preorder Traversal	Binary Search Trees
167	Best Time to Buy and Sell Stock	Dynamic Programming
168	Best Time to Buy and Sell Stock II	Dynamic Programming
169	Best Time to Buy and Sell Stock III	Arrays
170	Non-overlapping Intervals	Arrays
171	Longest Common Prefix	Tries
172	One Away	Strings
173	Count Even Odd	Hashing
174	Group Anagrams	Hashing
175	Find the Elements Removed from Nested Arrays ($A \supset B \supset C$)	Arrays
176	Longest Consecutive Path in a Grid	Arrays
177	Move All '#' to the Front of a String	Strings
178	Run-Length Encoding of a String	Strings
179	Spiral Matrix Traversal	Arrays
180	Evaluate the Polynomial $a^3 + a^2b + 2a^2b + 2ab^2 + ab^2 + b^3$	Mathematics
181	Dealership Tyres	Mathematics
182	Print Palindrome Numbers in a Range	Mathematics
183	Reverse a String	Strings
184	Fibonacci Series	Mathematics
185	Swap Two Numbers Without a Temporary Variable	Mathematics

1. Majority Element – I

Topic: Arrays (POTD)

Problem Statement

Given an array of n integers, find the majority element: the value that appears strictly more than $n/2$ times. You may assume that the array always contains such an element. Your solution should ideally run in linear time and constant extra space, for example using the Boyer–Moore voting technique, rather than sorting the array or storing counts in a hash map. Print the majority element. This problem checks whether you can spot that a true majority element survives repeated cancellation against different elements, which makes a single pass with one candidate and one counter sufficient. Read the input from standard input and print only the required value.

Input / Output format: Line 1: n . Line 2: n integers. Output: the majority element.

Example

Input

```
7
2 2 1 1 1 2 2
```

Output

```
2
```

Explanation: 2 occurs 4 times out of 7, which is more than 3.5.

Constraints

- $1 \leq n \leq 10^5$
- $-10^9 \leq a[i] \leq 10^9$
- A majority element is guaranteed to exist

Test Cases

Test 1 — Basic

```
Input:
7
3 2 3 3 1 3 2

Output:
3
```

Test 2 — Single element

```
Input:
1
5

Output:
5
```

Test 3 — Two equal elements

```
Input:
2
3 3

Output:
3
```

Test 4 — Negative values

```
Input:
5
-1 -1 -1 2 3

Output:
-1
```

Test 5 — Majority only at the end

```
Input:
5
1 2 3 3 3

Output:
3
```

Test 6 — Large values

```
Input:
6
1000000000 1000000000 1000000000 7 8 1000000000

Output:
1000000000
```

2. Two Sum

Topic: Hashing

Problem Statement

You are given an array of integers and a target value. Find two different positions i and j such that the elements stored at those positions add up to the target, and print the two indices in increasing order using zero-based indexing. It is guaranteed that exactly one valid pair exists, and you may not use the same element twice. A brute force check of every pair takes quadratic time, so aim for a single pass that stores each value seen so far in a hash map together with its index, and looks up the needed complement before inserting the current element.

Input / Output format: Line 1: n and target. Line 2: n integers. Output: the two indices separated by a space.

Example

Input

```
4 9
2 7 11 15
```

Output

```
0 1
```

Explanation: $a[0] + a[1] = 2 + 7 = 9$, so the answer is 0 1.

Constraints

- $2 \leq n \leq 10^5$
- $-10^9 \leq a[i], \text{target} \leq 10^9$
- Exactly one valid pair exists

Test Cases

Test 1 — Basic

```
Input:
3 6
3 2 4

Output:
1 2
```

Test 2 — Minimum size

```
Input:
2 7
3 4

Output:
0 1
```

Test 3 — Duplicate values form the pair

```
Input:
4 6
3 3 1 5

Output:
0 1
```

Test 4 — Negative numbers

```
Input:
5 -8
-1 -2 -3 -5 4
```

```
Output:  
2 3
```

Test 5 — Pair with zero

```
Input:  
4 0  
0 4 3 0  
  
Output:  
0 3
```

Test 6 — Large magnitude

```
Input:  
3 2000000000  
1000000000 5 1000000000  
  
Output:  
0 2
```

3. Longest Substring Without Repeating Characters

Topic: Strings

Problem Statement

Given a string s , determine the length of the longest contiguous substring that contains no repeated character. A substring must be made of consecutive characters of the original string, so a subsequence such as removing letters from the middle is not allowed. The classic approach uses a sliding window with two pointers and a map from character to its most recent

index, so that whenever a duplicate enters the window, the left boundary jumps just past the previous occurrence. This gives a linear time solution. Print a single integer, the maximum window length that was ever seen while scanning the whole string.

Input / Output format: Line 1: the string s. Output: one integer.

Example

Input

```
abcabcbb
```

Output

```
3
```

Explanation: The longest such substring is "abc", whose length is 3.

Constraints

- $1 \leq |s| \leq 5 \cdot 10^4$
- s consists of English letters, digits, symbols and spaces (no whitespace in tests)

Test Cases

Test 1 — Basic

```
Input :  
pwwkew
```

```
Output:  
3
```

Test 2 — Single character

```
Input:  
a  
  
Output:  
1
```

Test 3 — All characters identical

```
Input:  
bbbbbb  
  
Output:  
1
```

Test 4 — All characters distinct

```
Input:  
abcdefg  
  
Output:  
7
```

Test 5 — Repeat far apart

```
Input:  
abba  
  
Output:
```


Test 6 — Digits and symbols

```
Input:  
a1b2c3a1#
```

```
Output:  
7
```

4. Kadane's Algorithm (Maximum Subarray Sum)

Topic: Arrays

Problem Statement

Given an integer array that may contain negative numbers, find the contiguous non-empty subarray that has the largest possible sum and print that sum. A brute force method that checks all subarrays needs cubic or quadratic time, but Kadane's algorithm solves the task in one pass by keeping the best sum of a subarray ending at the current position, which is either the current element alone or the current element added to the previous best. Remember that the chosen subarray must contain at least one element, so when every value is negative the answer is the largest single element, not zero.

Input / Output format: Line 1: n. Line 2: n integers. Output: the maximum subarray sum.

Example

Input

```
9
-2 1 -3 4 -1 2 1 -5 4
```

Output

```
6
```

Explanation: The subarray [4, -1, 2, 1] has the largest sum, 6.

Constraints

- $1 \leq n \leq 10^5$
- $-10^4 \leq a[i] \leq 10^4$

Test Cases

Test 1 — Basic

```
Input:
5
1 2 3 -2 5

Output:
9
```

Test 2 — Single element

```
Input:
1
-7
```

```
Output:  
-7
```

Test 3 — All negative

```
Input:  
4  
-3 -1 -8 -2  
  
Output:  
-1
```

Test 4 — All positive

```
Input:  
4  
2 3 4 5  
  
Output:  
14
```

Test 5 — Contains zeros

```
Input:  
5  
0 0 -1 0 0  
  
Output:  
0
```

Test 6 — Best subarray is the whole array

```
Input:  
3  
5 -1 5
```

```
Output:
9
```

5. Merge Two Sorted Lists

Topic: Linked List

Problem Statement

You are given the heads of two singly linked lists whose values are already sorted in non-decreasing order. Merge them into one sorted linked list by splicing together the existing nodes and print the values of the merged list from head to tail. Either list may be empty, and duplicate values are allowed. The efficient method walks along both lists with two pointers, always attaching the smaller front node to the result, and finally attaches whatever remains of the non-empty list. This takes time proportional to the total number of nodes. If the merged list is empty, print [].

Input / Output format: Line 1: n1. Line 2: n1 sorted integers. Line 3: n2. Line 4: n2 sorted integers. Output: the merged list, space separated ([] if empty).

Example

Input

```
3
1 2 4
3
1 3 4
```

Output

```
1 1 2 3 4 4
```

Explanation: Merging gives 1 1 2 3 4 4.

Constraints

- $0 \leq n1, n2 \leq 5 \cdot 10^4$
- $-10^5 \leq \text{value} \leq 10^5$
- Both lists are sorted in non-decreasing order

Test Cases

Test 1 — Basic

```
Input:
3
1 3 5
3
2 4 6

Output:
1 2 3 4 5 6
```

Test 2 — Both lists empty

```
Input:
0

0

Output:
[]
```

Test 3 — First list empty

```
Input:
0

2
1 2

Output:
1 2
```

Test 4 — Second list empty

```
Input:
3
4 5 6
0

Output:
4 5 6
```

Test 5 — Duplicates across lists

```
Input:
3
1 1 2
3
1 2 2

Output:
1 1 1 2 2 2
```

Test 6 — Lists do not overlap

```
Input:
```

```
3
1 2 3
3
7 8 9

Output:
1 2 3 7 8 9
```

6. Remove N-th Node from the End of a Linked List

Topic: Linked List

Problem Statement

Given the head of a singly linked list and an integer k , delete the k -th node counted from the end of the list, then print the remaining list from head to tail. The value k is always valid, meaning it does not exceed the length of the list. A neat one-pass solution uses two pointers: move the fast pointer k steps ahead, then advance both pointers together until the fast one reaches the last node, at which point the slow pointer sits just before the node that must be removed. A dummy head node makes the case of removing the first node easy. Print [] if the list becomes empty.

Input / Output format: Line 1: n . Line 2: n integers (list values). Line 3: k . Output: the list after deletion, space separated ([] if empty).

Example

Input

```
5
1 2 3 4 5
2
```

Output

```
1 2 3 5
```

Explanation: The 2nd node from the end is 4, so the result is 1 2 3 5.

Constraints

- $1 \leq n \leq 10^4$
- $1 \leq k \leq n$
- $-10^5 \leq \text{value} \leq 10^5$

Test Cases

Test 1 — Basic

```
Input:
6
10 20 30 40 50 60
3

Output:
10 20 30 50 60
```

Test 2 — Single node removed

```
Input:
1
7
1

Output:
[]
```


Test 3 — Remove the head

```
Input:
4
1 2 3 4
4

Output:
2 3 4
```

Test 4 — Remove the tail

```
Input:
4
1 2 3 4
1

Output:
1 2 3
```

Test 5 — Two nodes

```
Input:
2
1 2
1

Output:
1
```

Test 6 — Duplicate values

```
Input:
5
2 2 2 2 2
3
```

```
Output:  
2 2 2 2
```

7. Minimum Window Substring

Topic: Strings

Problem Statement

Given two strings s and t , find the smallest substring of s that contains every character of t , including duplicates. If t contains two letters 'a', the window must also contain at least two letters 'a'. If several windows share the minimum length, print the one that starts earliest. If no such window exists, print -1. The standard method is a sliding window: expand the right end until all required characters are covered, then shrink the left end as far as possible while keeping the window valid, recording the best window seen. Frequency counts for the characters of t make each step constant time.

Input / Output format: Line 1: string s . Line 2: string t . Output: the minimum window, or -1.

Example

Input

```
ADOBECODEBANC  
ABC
```

Output

```
BANC
```

Explanation: The shortest window that holds A, B and C is "BANC".

Constraints

- $1 \leq |s|, |t| \leq 10^5$
- s and t contain only English letters (case sensitive)

Test Cases

Test 1 — Basic

```
Input:
aa
aa

Output:
aa
```

Test 2 — No valid window

```
Input:
a
b

Output:
-1
```

Test 3 — t longer than s

```
Input:
ab
abc

Output:
-1
```

Test 4 — Whole string is the answer

```
Input:
abc
cba

Output:
abc
```

Test 5 — Duplicates needed in t

```
Input:
AAABBC
AABC

Output:
AABBC
```

Test 6 — Case sensitivity

```
Input:
aAbB
AB

Output:
AbB
```

8. LRU Cache

Topic: Linked List (Design)

Problem Statement

Design a data structure that behaves as a Least Recently Used cache with a fixed capacity. It must support `put(key, value)`, which inserts or updates a pair, and `get(key)`, which returns the stored value or -1 when the key is absent. Both operations must run in average constant time, which is normally achieved with a hash map combined with a doubly linked list ordered by recency. Reading or writing a key makes it the most recently used one. When an insertion would exceed the capacity, evict the least recently used key. Process the given operations in order and print the result of every get operation on its own line.

Input / Output format: Line 1: capacity. Line 2: q (number of operations). Next q lines: 'put key value' or 'get key'. Output: result of each get on its own line ('(no output)' if there are no gets).

Example

Input

```
2
9
put 1 1
put 2 2
get 1
put 3 3
get 2
put 4 4
get 1
get 3
get 4
```

Output

```
1
-1
-1
3
4
```

Explanation: Key 2 is evicted when 3 is added, and key 1 is evicted when 4 is added, giving 1, -1, -1, 3, 4.

Constraints

- $1 \leq \text{capacity} \leq 3000$
- $1 \leq q \leq 2 \cdot 10^5$
- $0 \leq \text{key}, \text{value} \leq 10^5$

Test Cases

Test 1 — Basic

```
Input:
2
5
put 1 10
put 2 20
get 1
put 3 30
get 2

Output:
10
-1
```

Test 2 — Capacity one

```
Input:
1
5
```

```
put 1 1
put 2 2
get 1
get 2
get 2
```

```
Output:
-1
2
2
```

Test 3 — Update existing key

```
Input:
2
6
put 1 1
put 2 2
put 1 100
put 3 3
get 1
get 2
```

```
Output:
100
-1
```

Test 4 — Get on empty cache

```
Input:
2
2
get 5
get 6
```

```
Output:
-1
-1
```

Test 5 — Only puts, no gets

```
Input:
```

```
2
3
put 1 1
put 2 2
put 3 3

Output:
(no output)
```

Test 6 — Get refreshes recency

```
Input:
3
8
put 1 1
put 2 2
put 3 3
get 1
put 4 4
get 2
get 1
get 3

Output:
1
-1
1
3
```

9. Trapping Rain Water

Topic: Stack & Queues

Problem Statement

You are given n non-negative integers that represent an elevation map in which every bar has width one. After rain, water gets trapped in the gaps between bars. Compute the total volume of water that can be stored. The water above a position is limited by the smaller of the tallest bar to its left and the tallest bar to its right, minus the height of the bar itself. You can

compute this using prefix and suffix maxima, a monotonic stack, or a two pointer technique in constant extra space. Print a single integer, the total trapped water.

Input / Output format: Line 1: n . Line 2: n non-negative integers. Output: total trapped water.

Example

Input

```
12
0 1 0 2 1 0 1 3 2 1 2 1
```

Output

```
6
```

Explanation: Six units of water are trapped between the bars.

Constraints

- $1 \leq n \leq 2 \cdot 10^4$
- $0 \leq \text{height}[i] \leq 10^5$

Test Cases

Test 1 — Basic

```
Input :
```

```
6
4 2 0 3 2 5
```

```
Output:
9
```

Test 2 — Single bar

```
Input:
1
5
```

```
Output:
0
```

Test 3 — Two bars cannot trap water

```
Input:
2
3 4
```

```
Output:
0
```

Test 4 — Strictly increasing

```
Input:
5
1 2 3 4 5
```

```
Output:
0
```

Test 5 — Flat surface

```
Input:
4
3 3 3 3

Output:
0
```

Test 6 — Deep valley

```
Input:
5
5 0 0 0 5

Output:
15
```

10. Next Permutation

Topic: Arrays

Problem Statement

A permutation of an array is one of its possible orderings. The next permutation is the next lexicographically greater ordering of the same numbers. If the given arrangement is already the largest possible, wrap around and return the smallest arrangement, which is the array sorted in ascending order. Rearrange the elements to obtain the next permutation and print the result. The array may contain duplicates. An efficient algorithm finds the rightmost position whose value is smaller than its successor, swaps it with the smallest larger value to its right, and then reverses the suffix after that position. It works in linear time and constant space.

Input / Output format: Line 1: n . Line 2: n integers. Output: the next permutation, space separated.

Example

Input

```
3
1 2 3
```

Output

```
1 3 2
```

Explanation: The next arrangement after 1 2 3 in dictionary order is 1 3 2.

Constraints

- $1 \leq n \leq 10^5$
- $0 \leq a[i] \leq 10^9$

Test Cases

Test 1 — Basic

```
Input:
4
1 3 5 4

Output:
1 4 3 5
```

Test 2 — Single element

```
Input:
1
9

Output:
9
```

Test 3 — Already largest wraps to smallest

```
Input:
3
3 2 1

Output:
1 2 3
```

Test 4 — Duplicates

```
Input:
4
1 5 1 1

Output:
5 1 1 1
```

Test 5 — Suffix reversal needed

```
Input:
6
1 2 7 4 3 1

Output:
1 3 1 2 4 7
```

Test 6 — All equal

```
Input:
3
2 2 2

Output:
2 2 2
```

11. Longest Consecutive Sequence in an Array

Topic: Hashing

Problem Statement

Given an unsorted array of integers, find the length of the longest sequence of consecutive values that can be formed using elements of the array. The elements do not need to be adjacent in the array, and their order in the input does not matter, so for instance 4, 2, 3, 1 forms the sequence 1, 2, 3, 4. Duplicates are counted only once. Sorting would take $O(n \log n)$ time. To achieve $O(n)$, place all numbers in a hash set and start counting only from those numbers whose predecessor is not present. Print the length of the longest run.

Input / Output format: Line 1: n . Line 2: n integers. Output: one integer.

Example

Input

```
6
100 4 200 1 3 2
```

Output

```
4
```

Explanation: The longest run is 1, 2, 3, 4, so the answer is 4.

Constraints

- $1 \leq n \leq 10^5$
- $-10^9 \leq a[i] \leq 10^9$

Test Cases

Test 1 — Basic

```
Input:
10
0 3 7 2 5 8 4 6 0 1

Output:
9
```

Test 2 — Single element

```
Input:
1
42

Output:
1
```

Test 3 — All duplicates

```
Input:
5
7 7 7 7 7

Output:
1
```

Test 4 — No two consecutive

```
Input:
4
10 20 30 40

Output:
1
```

Test 5 — Negative and positive range

```
Input:
6
-2 -1 0 1 2 10

Output:
5
```

Test 6 — Two separate runs

```
Input:
8
1 2 3 10 11 12 13 5

Output:
4
```

12. Longest Increasing Subsequence

Topic: Binary Search

Problem Statement

Given an integer array, find the length of its longest strictly increasing subsequence. A subsequence is obtained by deleting zero or more elements without changing the order of the remaining ones, so the chosen elements need not be contiguous. Equal values cannot both belong to a strictly increasing subsequence. A dynamic programming solution takes quadratic time, but you can reach $O(n \log n)$ by maintaining a list where entry i holds the smallest possible tail value of an increasing subsequence of length $i + 1$, and using binary search to place each new element. Print the length only, not the subsequence itself.

Input / Output format: Line 1: n . Line 2: n integers. Output: one integer.

Example

Input

```
8
10 9 2 5 3 7 101 18
```

Output

```
4
```

Explanation: One longest choice is 2, 3, 7, 18 (length 4).

Constraints

- $1 \leq n \leq 10^5$
- $-10^4 \leq a[i] \leq 10^4$

Test Cases

Test 1 — Basic

```
Input:
6
0 8 4 12 2 10

Output:
3
```

Test 2 — Single element

```
Input:
1
3

Output:
1
```

Test 3 — All equal

```
Input:
5
7 7 7 7 7

Output:
1
```

Test 4 — Strictly decreasing

```
Input:
5
5 4 3 2 1

Output:
1
```

Test 5 — Strictly increasing

```
Input:
5
1 2 3 4 5

Output:
5
```

Test 6 — Negative values mixed

```
Input:
7
-5 -3 -4 0 -1 2 1

Output:
4
```

13. Maximum Product Subarray

Topic: Arrays

Problem Statement

Given an integer array, find the contiguous non-empty subarray whose elements have the largest product and print that product. The array can contain positive numbers, negative numbers and zeros, which makes the problem trickier than the maximum sum version: multiplying by a negative number can turn the smallest product into the largest one, and a zero resets everything. A linear time solution tracks both the maximum and the minimum product of a subarray ending at each position and updates them together. It is guaranteed that the answer fits into a 64 bit signed integer.

Input / Output format: Line 1: n. Line 2: n integers. Output: the maximum product.

Example

Input

```
4
2 3 -2 4
```

Output

```
6
```

Explanation: The subarray [2, 3] gives 6, which is the maximum.

Constraints

- $1 \leq n \leq 2 \cdot 10^4$
- $-10 \leq a[i] \leq 10$
- The result fits in a 64-bit signed integer

Test Cases

Test 1 — Basic

```
Input:
5
1 -2 -3 4 -1

Output:
24
```

Test 2 — Single negative element

```
Input:
1
-5

Output:
-5
```

Test 3 — Contains zero

```
Input:
3
-2 0 -1

Output:
0
```

Test 4 — Two negatives make a positive

```
Input:
4
-2 3 -4 1

Output:
24
```

Test 5 — All zeros

```
Input:
3
0 0 0

Output:
0
```

Test 6 — Negative count odd

```
Input:
5
-1 -1 -1 2 3

Output:
6
```

14. Sort an Array of 0's, 1's and 2's

Topic: Sorting

Problem Statement

You are given an array that contains only the values 0, 1 and 2. Sort the array in ascending order without using a general sorting function. This is the well known Dutch National Flag problem. It can be solved in a single pass with three pointers: low marks the boundary of zeros, high marks the boundary of twos, and mid scans the unknown region, swapping elements to the correct side. The algorithm runs in linear time and uses constant extra space. Print the sorted array as space separated integers on one line. The array may already be sorted or may contain only one distinct value.

Input / Output format: Line 1: n. Line 2: n integers, each 0, 1 or 2. Output: the sorted array.

Example

Input

```
6
2 0 2 1 1 0
```

Output

```
0 0 1 1 2 2
```

Explanation: Sorting gives 0 0 1 1 2 2.

Constraints

- $1 \leq n \leq 10^5$
- $a[i] \in \{0, 1, 2\}$

Test Cases

Test 1 — Basic

```
Input:
8
1 2 0 1 0 2 1 0

Output:
0 0 0 1 1 1 2 2
```

Test 2 — Single element

```
Input:
1
2

Output:
2
```

Test 3 — Only zeros

```
Input:
4
0 0 0 0

Output:
0 0 0 0
```

Test 4 — Already sorted

```
Input:
6
0 0 1 1 2 2

Output:
0 0 1 1 2 2
```

Test 5 — Reverse sorted

```
Input:
6
2 2 1 1 0 0

Output:
0 0 1 1 2 2
```


Test 6 — Missing one value

```
Input:
5
2 0 2 0 2

Output:
0 0 2 2 2
```

15. Check if a Binary Tree is a BST

Topic: Binary Search Trees

Problem Statement

Given the root of a binary tree, determine whether it is a valid binary search tree. In a valid BST, every node's left subtree contains only values strictly smaller than the node, every node's right subtree contains only values strictly greater than the node, and both subtrees are themselves valid BSTs. Comparing a node only with its direct children is not enough, because a deeper descendant can still violate an ancestor's bound. Pass a permitted range down the recursion, or verify that an inorder traversal is strictly increasing. Print true if the tree is a valid BST and false otherwise.

Input / Output format: One line: the tree in level-order, with 'null' for missing children.
Output: true or false.

Example

Input

```
2 1 3
```

Output

```
true
```

Explanation: Left child $1 < 2 <$ right child 3 , so the tree is a valid BST: true.

Constraints

- $1 \leq \text{number of nodes} \leq 10^4$
- $-2^{31} \leq \text{value} \leq 2^{31} - 1$

Test Cases

Test 1 — Valid BST

```
Input:
5 3 8 1 4 7 9

Output:
true
```

Test 2 — Single node

```
Input:
10

Output:
true
```

Test 3 — Right child smaller than root

```
Input:
5 1 4 null null 3 6

Output:
false
```

Test 4 — Deep violation of an ancestor bound

```
Input:
10 5 15 null null 6 20

Output:
false
```

Test 5 — Duplicate value on the left

```
Input:
2 2 null

Output:
false
```

Test 6 — Left-skewed valid tree

```
Input:
5 4 null 3 null 2

Output:
true
```

16. Jump Game – I

Topic: Greedy Algorithms

Problem Statement

You are standing at the first index of an array of non-negative integers. Each element tells you the maximum number of positions you can jump forward from that index; you may jump any smaller distance too. Decide whether you can reach the last index. A greedy strategy works in linear time: keep track of the farthest index reachable so far while scanning from left to right, and if the current index is ever beyond that farthest reach, you are stuck. Print true if the last index can be reached and false otherwise. A single element array is trivially reachable.

Input / Output format: Line 1: n. Line 2: n non-negative integers. Output: true or false.

Example

Input

```
5
2 3 1 1 4
```

Output

```
true
```

Explanation: Jump 1 step to index 1, then 3 steps to the last index: true.

Constraints

- $1 \leq n \leq 10^4$
- $0 \leq a[i] \leq 10^5$

Test Cases

Test 1 — Reachable

```
Input:
6
2 0 2 0 1 0

Output:
true
```

Test 2 — Single element

```
Input:
1
0

Output:
true
```

Test 3 — Blocked by a zero

```
Input:
5
3 2 1 0 4

Output:
false
```

Test 4 — Big first jump

```
Input:
5
10 0 0 0 0

Output:
true
```

Test 5 — Starts with zero, $n > 1$

```
Input:
2
0 5

Output:
false
```

Test 6 — Exactly reaches the end

```
Input:
4
1 1 1 1

Output:
true
```

17. Clone a Linked List with Random and Next Pointers

Topic: Linked List

Problem Statement

Each node of a linked list carries a value, a next pointer and a random pointer that may refer to any node of the list or to null. Create a deep copy of the list: every node of the copy must be a brand new node, and the next and random pointers of the copied nodes must point only to nodes within the copy. An $O(1)$ space technique interleaves copied nodes between original nodes, assigns random pointers using the interleaving, and then separates the two lists. For verification, print one line per copied node in list order containing its value and the index of its random target, or -1 for null.

Input / Output format: Line 1: n . Next n lines: 'value randomIndex' (randomIndex is 0-based, or -1 for null). Output: the same n lines, describing the deep copy.

Example

Input

```
3
7 -1
13 0
11 -1
```

Output

```
7 -1 13 0 11 -1
```

Explanation: Node 2 (value 13) has its random pointer to node 0, and the copy preserves that: 7 -1 / 13 0 / 11 -1.

Constraints

- $0 \leq n \leq 1000$ (tests use $n \geq 1$)
- $-10^4 \leq \text{value} \leq 10^4$
- $-1 \leq \text{randomIndex} < n$

Test Cases

Test 1 — Basic

```
Input:
5
7 -1
13 0
11 4
10 2
1 0

Output:
7 -1 13 0 11 4 10 2 1 0
```

Test 2 — Single node, null random

```
Input:
1
5 -1

Output:
5 -1
```

Test 3 — Single node, random to itself

```
Input:
1
5 0

Output:
5 0
```

Test 4 — All random pointers null


```
Input:
3
1 -1
2 -1
3 -1

Output:
1 -1 2 -1 3 -1
```

Test 5 — Random points forward and backward

```
Input:
4
1 3
2 0
3 1
4 2

Output:
1 3 2 0 3 1 4 2
```

Test 6 — Duplicate values

```
Input:
3
9 2
9 0
9 1

Output:
9 2 9 0 9 1
```

18. Add Two Numbers in Linked List

Topic: Linked List

Problem Statement

Two non-negative integers are stored in two linked lists, where each node holds a single digit and the digits are stored in reverse order, so the head holds the ones digit. Add the two numbers and print the sum in the same representation, again with the least significant digit first. The lists may have different lengths, and a final carry can create one extra digit, such as $5 + 5$ giving 0 followed by 1. Walk both lists together, keep a running carry, and append one digit for each step. Neither number has leading zeros except the number zero itself.

Input / Output format: Line 1: n_1 . Line 2: n_1 digits. Line 3: n_2 . Line 4: n_2 digits. Output: the sum's digits, least significant first.

Example

Input

```
3
2 4 3
3
5 6 4
```

Output

```
7 0 8
```

Explanation: $342 + 465 = 807$, stored in reverse as 7 0 8.

Constraints

- $1 \leq n_1, n_2 \leq 100$
- $0 \leq \text{digit} \leq 9$
- No leading zeros except for the number 0

Test Cases

Test 1 — Basic

```
Input:
3
9 9 9
1
1

Output:
0 0 0 1
```

Test 2 — Both numbers are zero

```
Input:
1
0
1
0

Output:
0
```

Test 3 — Different lengths

```
Input:
2
1 8
4
5 5 9 1

Output:
6 3 0 2
```

Test 4 — Final carry adds a digit

```
Input:
```

```
1
```

```
5
```

```
1
```

```
5
```

```
Output:
```

```
0 1
```

Test 5 — Long carry chain

```
Input:
```

```
4
```

```
9 9 9 9
```

```
4
```

```
1 0 0 0
```

```
Output:
```

```
0 0 0 0 1
```

Test 6 — Single digits, no carry

```
Input:
```

```
1
```

```
3
```

```
1
```

```
4
```

```
Output:
```

```
7
```

19. Largest Rectangle in a Histogram

Topic: Stack & Queues

Problem Statement

You are given an array of non-negative integers that represent the heights of bars in a histogram, where each bar has width one. Find the area of the largest rectangle that can be formed inside the histogram using contiguous bars. The rectangle's height is limited by the shortest bar it spans. A brute force approach that checks every pair of boundaries takes quadratic time. Use a monotonic stack that keeps bar indices with increasing heights, and when a shorter bar arrives, pop taller bars and compute the area they could span. The overall running time is linear. Print the maximum area.

Input / Output format: Line 1: n . Line 2: n heights. Output: the largest rectangle area.

Example

Input

```
6
2 1 5 6 2 3
```

Output

```
10
```

Explanation: Bars of height 5 and 6 form a rectangle of height 5 and width 2, area 10.

Constraints

- $1 \leq n \leq 10^5$
- $0 \leq \text{heights}[i] \leq 10^4$

Test Cases

Test 1 — Basic

```
Input:
7
6 2 5 4 5 1 6

Output:
12
```

Test 2 — Single bar

```
Input:
1
7

Output:
7
```

Test 3 — All zero heights

```
Input:
3
0 0 0

Output:
0
```

Test 4 — All bars equal

```
Input:
5
3 3 3 3 3

Output:
```

15

Test 5 — Strictly increasing

```
Input:
5
1 2 3 4 5

Output:
9
```

Test 6 — Strictly decreasing

```
Input:
5
5 4 3 2 1

Output:
9
```

20. Sliding Window Maximum

Topic: Stack & Queues

Problem Statement

You are given an integer array and a window size k . The window starts at the leftmost k elements and slides one position to the right at each step until it reaches the end of the array. For every window position, determine the maximum element inside the window, and print all these maximums in order. A naive scan of each window costs $O(n \cdot k)$, which is too slow for large inputs. Use a monotonic deque that stores indices of candidates in decreasing order of

their values, removing indices that fall out of the window from the front and smaller values from the back. The total time is linear.

Input / Output format: Line 1: n and k . Line 2: n integers. Output: the $n - k + 1$ window maximums, space separated.

Example

Input

```
8 3
1 3 -1 -3 5 3 6 7
```

Output

```
3 3 5 5 6 7
```

Explanation: Windows are $[1\ 3\ -1]$, $[3\ -1\ -3]$, $[-1\ -3\ 5]$, $[-3\ 5\ 3]$, $[5\ 3\ 6]$, $[3\ 6\ 7]$, giving $3\ 3\ 5\ 5\ 6\ 7$.

Constraints

- $1 \leq k \leq n \leq 10^5$
- $-10^4 \leq a[i] \leq 10^4$

Test Cases

Test 1 — Basic


```
Input:
6 2
4 2 12 3 8 1

Output:
4 12 12 8 8
```

Test 2 — Window of size one

```
Input:
4 1
9 8 7 6

Output:
9 8 7 6
```

Test 3 — Window equals array

```
Input:
4 4
1 5 2 4

Output:
5
```

Test 4 — Single element

```
Input:
1 1
-3

Output:
-3
```

Test 5 — Decreasing array

```
Input:
5 3
9 8 7 6 5

Output:
9 8 7
```

Test 6 — Duplicates and negatives

```
Input:
7 3
-1 -1 -1 -2 -2 -2 -1

Output:
-1 -1 -1 -2 -1
```

21. Construct a Binary Tree from Preorder and Inorder Traversals

Topic: Hashing (Trees)

Problem Statement

You are given the preorder and inorder traversals of a binary tree that contains distinct values. Rebuild the tree and print it in level-order, using the word null for missing children and dropping trailing nulls. The first element of the preorder list is always the root, and its position in the inorder list splits the remaining values into the left and right subtrees. To avoid searching for the root repeatedly, store the inorder index of every value in a hash map, which reduces the running time to linear. Recursion on the left part first and then on the right part consumes the preorder list correctly.

Input / Output format: Line 1: n. Line 2: preorder. Line 3: inorder. Output: level-order of the rebuilt tree.

Example

Input

```
5
3 9 20 15 7
9 3 15 20 7
```

Output

```
3 9 20 null null 15 7
```

Explanation: Root 3 has left child 9 and right child 20, which has children 15 and 7: 3 9 20 null null 15 7.

Constraints

- $1 \leq n \leq 3000$
- All node values are distinct
- $-3000 \leq \text{value} \leq 3000$

Test Cases

Test 1 — Basic

```
Input:
4
1 2 4 3
4 2 1 3

Output:
```

```
1 2 3 4
```

Test 2 — Single node

```
Input:
1
8
8

Output:
8
```

Test 3 — Left skewed

```
Input:
3
3 2 1
1 2 3

Output:
3 2 null 1
```

Test 4 — Right skewed

```
Input:
3
1 2 3
1 2 3

Output:
1 null 2 null 3
```

Test 5 — Negative values

```
Input:
```

```
3
0 -5 5
-5 0 5

Output:
0 -5 5
```

Test 6 — Complete tree

```
Input:
7
1 2 4 5 3 6 7
4 2 5 1 6 3 7

Output:
1 2 3 4 5 6 7
```

22. Implement Min Stack

Topic: Stack & Queues

Problem Statement

Design a stack that supports the usual push, pop and top operations and can also report the minimum element currently stored in the stack, all in constant time. The command `getMin` returns the smallest value among the elements that are currently in the stack, and it must remain correct after any number of pops. A common trick is to store with every element the minimum of the stack up to that element, or to keep a second stack of minimums. Process the given commands in order and print the answer of each top and `getMin` command on its own line. Pop and top are never called on an empty stack.

Input / Output format: Line 1: `q`. Next `q` lines: 'push `x`', 'pop', 'top' or 'getMin'. Output: result of every top / `getMin`, one per line.

Example

Input

```
7
push -2
push 0
push -3
getMin
pop
top
getMin
```

Output

```
-3
0
-2
```

Explanation: Minimum is -3; after pop, top is 0 and minimum is -2.

Constraints

- $1 \leq q \leq 3 \cdot 10^4$
- $-2^{31} \leq x \leq 2^{31} - 1$
- pop, top and getMin are only called on a non-empty stack

Test Cases

Test 1 — Basic

```
Input :
6
```

```
push 5
push 3
getMin
push 7
getMin
top
```

```
Output:
3
3
7
```

Test 2 — Single push

```
Input:
2
push 4
getMin
```

```
Output:
4
```

Test 3 — Duplicate minimum values

```
Input:
7
push 2
push 2
push 5
getMin
pop
pop
getMin
```

```
Output:
2
2
```

Test 4 — Minimum restored after pop

```
Input:
6
```

```
push 3
push 1
pop
getMin
top
getMin
```

```
Output:
3
3
3
```

Test 5 — Negative values

```
Input:
5
push -10
push -20
getMin
pop
getMin
```

```
Output:
-20
-10
```

Test 6 — No queries, only updates

```
Input:
3
push 1
push 2
pop
```

```
Output:
(no output)
```

23. Merge Two Sorted Arrays Without Extra Space

Topic: Arrays

Problem Statement

You are given two sorted arrays *a* and *b* of sizes *n* and *m*. Merge them so that the smallest *n* elements of the combined data end up in array *a* and the remaining *m* elements end up in array *b*, with both arrays sorted in non-decreasing order. Do not create an auxiliary array of size $n + m$. Efficient techniques include the insertion based approach that swaps the largest element of *a* with the first element of *b* and re-sorts *b*, or the gap method inspired by shell sort, which uses $O((n + m) \log(n + m))$ time and $O(1)$ extra space. Print the final array *a* on the first line and array *b* on the second line.

Input / Output format: Line 1: *n* and *m*. Line 2: array *a* (sorted). Line 3: array *b* (sorted).
Output: final *a* on line 1, final *b* on line 2.

Example

Input

```
4 3
1 3 5 7
0 2 6
```

Output

```
0 1 2 3
5 6 7
```

Explanation: Combined sorted order is 0 1 2 3 5 6 7; *a* takes the first 4, *b* takes the last 3.

Constraints

- $1 \leq n, m \leq 10^5$
- $-10^9 \leq \text{elements} \leq 10^9$
- Both inputs are sorted in non-decreasing order

Test Cases

Test 1 — Basic

```
Input:
3 3
1 4 7
2 3 9

Output:
1 2 3
4 7 9
```

Test 2 — Single elements, needs swap

```
Input:
1 1
5
2

Output:
2
5
```

Test 3 — All of b smaller

```
Input:
3 2
10 11 12
1 2

Output:
1 2 10
11 12
```

Test 4 — All of a smaller

```
Input:
2 3
1 2
5 6 7

Output:
1 2
5 6 7
```

Test 5 — Duplicates

```
Input:
3 3
1 1 3
1 2 3

Output:
1 1 1
2 3 3
```

Test 6 — Negative numbers

```
Input:
3 2
-5 -1 4
-3 0

Output:
-5 -3 -1
0 4
```

24. Search in a 2D Matrix – II

Topic: Binary Search

Problem Statement

You are given an m by n matrix in which every row is sorted in ascending order from left to right and every column is sorted in ascending order from top to bottom. Note that, unlike the simpler version, the first element of a row is not necessarily larger than the last element of the previous row. Decide whether a target value appears in the matrix and print true or false. A clever staircase search starts at the top right corner and moves left when the current value is larger than the target and down when it is smaller, achieving $O(m + n)$ time with constant extra space.

Input / Output format: Line 1: m and n . Next m lines: the rows. Last line: target. Output: true or false.

Example

Input

```
3 3
1 4 7
2 5 8
3 6 9
5
```

Output

```
true
```

Explanation: 5 sits at row 1, column 1, so the answer is true.

Constraints

- $1 \leq m, n \leq 300$
- $-10^9 \leq \text{matrix}[i][j], \text{target} \leq 10^9$
- Rows and columns are sorted ascending

Test Cases

Test 1 — Target present

```
Input:
3 4
1 4 7 11
2 5 8 12
3 6 9 16
9
```

```
Output:
true
```

Test 2 — Target absent

```
Input:
3 4
1 4 7 11
2 5 8 12
3 6 9 16
10
```

```
Output:
false
```

Test 3 — Single cell, match

```
Input:
1 1
5
5
```

```
Output:  
true
```

Test 4 — Single cell, no match

```
Input:  
1 1  
5  
6  
  
Output:  
false
```

Test 5 — Target smaller than all

```
Input:  
2 2  
2 3  
4 5  
1  
  
Output:  
false
```

Test 6 — Target larger than all

```
Input:  
2 2  
2 3  
4 5  
6  
  
Output:  
false
```

25. Find Peak Element

Topic: Binary Search

Problem Statement

A peak element is an element that is strictly greater than its neighbours. Given an array in which no two adjacent elements are equal, find the index of any peak element. You may imagine that both ends of the array are surrounded by negative infinity, so the first or last element can be a peak if it beats its only neighbour. Your algorithm should run in $O(\log n)$ time: compare the middle element with its right neighbour and move toward the side that is rising, because a peak is guaranteed there. Any valid peak index is accepted by the checker; the expected outputs below show the smallest such index.

Input / Output format: Line 1: n . Line 2: n integers. Output: an index of a peak (the smallest one is shown).

Example

Input

```
4
1 2 3 1
```

Output

```
2
```

Explanation: Element 3 at index 2 is greater than both neighbours, so print 2.

Constraints

- $1 \leq n \leq 1000$
- $-2^{31} \leq a[i] \leq 2^{31} - 1$
- $a[i] \neq a[i+1]$ for all valid i

Test Cases

Test 1 — Basic

```
Input:
7
1 2 1 3 5 6 4

Output:
1
```

Test 2 — Single element

```
Input:
1
10

Output:
0
```

Test 3 — Peak at first index

```
Input:
3
5 3 1

Output:
0
```

Test 4 — Peak at last index


```
Input:
4
1 2 3 4

Output:
3
```

Test 5 — Two elements

```
Input:
2
2 1

Output:
0
```

Test 6 — Negative values

```
Input:
5
-5 -3 -4 -2 -6

Output:
1
```

26. Median of Two Sorted Arrays

Topic: Binary Search

Problem Statement

Given two sorted arrays a and b of sizes n and m, find the median of the combined sorted data. If the total count is odd, the median is the middle element; if it is even, the median is the average of the two middle elements. Merging the arrays takes $O(n + m)$ time, but the intended solution is $O(\log(\min(n, m)))$ and applies binary search on the shorter array to find a partition where every element on the left side is not larger than every element on the right side. Print the median as a decimal number with exactly one digit after the decimal point.

Input / Output format: Line 1: n. Line 2: array a. Line 3: m. Line 4: array b. Output: the median with one decimal place.

Example

Input

```
2
1 3
1
2
```

Output

```
2.0
```

Explanation: Merged data is 1 2 3, whose median is 2.0.

Constraints

- $0 \leq n, m \leq 10^5$ and $n + m \geq 1$
- $-10^6 \leq \text{elements} \leq 10^6$
- Both arrays are sorted

Test Cases

Test 1 — Even total count

```
Input:
2
1 2
2
3 4

Output:
2.5
```

Test 2 — One array empty

```
Input:
0

3
4 5 6

Output:
5.0
```

Test 3 — Single element overall

```
Input:
1
7
0

Output:
7.0
```

Test 4 — Odd total count

```
Input:
```

```
3
1 3 8
2
7 9
```

```
Output:
7.0
```

Test 5 — Identical arrays

```
Input:
3
2 2 2
3
2 2 2
```

```
Output:
2.0
```

Test 6 — Negative values

```
Input:
2
-5 -3
2
-1 0
```

```
Output:
-2.0
```

27. Kth Smallest and Kth Largest Element in a BST

Topic: Binary Search Trees

Problem Statement

Given the root of a binary search tree and an integer k , find both the k -th smallest and the k -th largest values stored in the tree. Because an inorder traversal of a BST visits values in ascending order, the k -th smallest value is the k -th node visited in that traversal, while the k -th largest is found by traversing in reverse inorder (right, node, left) or as the $(n - k + 1)$ -th smallest. Try to stop the traversal as soon as the answer is found. Print the two numbers on one line, first the k -th smallest and then the k -th largest, separated by a single space.

Input / Output format: Line 1: the BST in level-order ('null' for missing). Line 2: k . Output: 'kthSmallest kthLargest'.

Example

Input

```
5 3 6 2 4 null null 1
3
```

Output

```
3 4
```

Explanation: Inorder is 1 2 3 4 5 6; the 3rd smallest is 3 and the 3rd largest is 4.

Constraints

- $1 \leq k \leq \text{number of nodes} \leq 10^4$
- $0 \leq \text{value} \leq 10^4$
- Values are distinct

Test Cases

Test 1 — Basic

```
Input:
8 4 12 2 6 10 14
2

Output:
4 12
```

Test 2 — Single node

```
Input:
5
1

Output:
5 5
```

Test 3 — k equals 1

```
Input:
8 4 12 2 6 10 14
1

Output:
2 14
```

Test 4 — k equals number of nodes

```
Input:
8 4 12 2 6 10 14
7

Output:
14 2
```

Test 5 — Right skewed tree

```
Input:
1 null 2 null 3 null 4
2

Output:
2 3
```

Test 6 — Left skewed tree

```
Input:
4 3 null 2 null 1
3

Output:
3 2
```

28. Candy

Topic: Greedy Algorithms

Problem Statement

There are n children standing in a line and each child has a rating value. You must give candies to the children so that every child receives at least one candy, and a child with a higher rating than an immediate neighbour must receive more candies than that neighbour. Children with equal ratings may receive any number of candies relative to each other. Find the minimum total number of candies needed. A greedy solution makes two passes: left to right to satisfy the left neighbour rule and right to left to satisfy the right neighbour rule, taking the maximum at each child. Print the total.

Input / Output format: Line 1: n. Line 2: n ratings. Output: the minimum number of candies.

Example

Input

```
3
1 0 2
```

Output

```
5
```

Explanation: Give 2, 1 and 2 candies, so the total is 5.

Constraints

- $1 \leq n \leq 2 \cdot 10^4$
- $0 \leq \text{ratings}[i] \leq 2 \cdot 10^4$

Test Cases

Test 1 — Basic

```
Input:
3
1 2 2

Output:
```



```
4
```

Test 2 — Single child

```
Input:
1
9

Output:
1
```

Test 3 — All equal ratings

```
Input:
4
5 5 5 5

Output:
4
```

Test 4 — Strictly increasing

```
Input:
5
1 2 3 4 5

Output:
15
```

Test 5 — Strictly decreasing

```
Input:
5
5 4 3 2 1
```

```
Output:
15
```

Test 6 — Valley then peak

```
Input:
7
1 3 4 5 2 1 2

Output:
15
```

29. Valid Parenthesis Checker (with Wildcard)

Topic: Greedy Algorithms

Problem Statement

Given a string containing only the characters '(', ')', and '*', *check whether it can be considered valid. Every '(' must have a matching ')' that comes after it, every ')' must have a matching '(' before it, and the character '*' can be treated as '(', as ')' or as an empty string.* The greedy technique tracks the smallest and largest possible number of unmatched opening brackets while scanning: a star widens the range, an opening bracket increases both bounds, and a closing bracket decreases both. If the maximum ever becomes negative the string is invalid. Print true or false.

Input / Output format: One line: the string. Output: true or false.

Example

Input

```
(*)
```

Output

```
true
```

Explanation: The star can be an empty string, so the string is valid: true.

Constraints

- $1 \leq \text{length} \leq 100$
- Characters are only '(', ')', and '*'

Test Cases

Test 1 — Balanced without stars

```
Input:
(())()

Output:
true
```

Test 2 — Single star

```
Input:
*

Output:
true
```

Test 3 — Star acts as opening bracket

```
Input:
(*)

Output:
true
```

Test 4 — Star acts as closing bracket

```
Input:
(*) )

Output:
true
```

Test 5 — Unmatched closing bracket

```
Input:
() ) *

Output:
false
```

Test 6 — Stars cannot fix wrong order

```
Input:
) * (

Output:
false
```

30. Count Subarrays with Given Sum

Topic: Arrays

Year / Source: May 2024 (Code360 on-campus experience); also on the past-year list

Problem Statement

Given an unsorted array of integers, which may contain negative numbers and zeros, count the number of contiguous subarrays whose elements add up exactly to a given value k . Because negative numbers are allowed, a sliding window will not work. Instead, compute prefix sums while scanning and store in a hash map how many times each prefix sum has appeared. For each new prefix sum, the number of subarrays ending here with sum k equals the number of earlier prefixes equal to the current prefix minus k . This solves the task in linear time. Print the total count as a single integer.

Input / Output format: Line 1: n and k . Line 2: n integers. Output: the number of subarrays with sum k .

Example

Input

```
3 5
2 3 5
```

Output

```
2
```

Explanation: Subarrays $[2, 3]$ and $[5]$ both sum to 5, so the answer is 2.

Constraints

- $1 \leq n \leq 2 \cdot 10^4$
- $-1000 \leq a[i] \leq 1000$
- $-10^7 \leq k \leq 10^7$

Test Cases

Test 1 — Basic

```
Input:
5 3
1 2 3 -1 1

Output:
3
```

Test 2 — Single element equals k

```
Input:
1 4
4

Output:
1
```

Test 3 — No valid subarray

```
Input:
3 100
1 2 3

Output:
```

```
0
```

Test 4 — Zeros create many subarrays

```
Input:
4 0
0 0 0 0

Output:
10
```

Test 5 — Negative numbers

```
Input:
6 0
1 -1 1 -1 1 -1

Output:
9
```

Test 6 — Whole array sum

```
Input:
4 10
1 2 3 4

Output:
1
```

31. Find the Intersection Point of a Y-Shaped Linked List

Topic: Linked List

Problem Statement

Two singly linked lists start separately but may merge into a common tail, forming the shape of the letter Y. Find the node where the two lists meet and print its value, or print -1 if they never meet. Intersection is decided by node identity, not by value, so two different nodes holding equal values do not count. A space efficient method uses two pointers: when a pointer reaches the end of its list it jumps to the head of the other list, so both pointers travel the same total distance and meet at the intersection node, or both become null together when no intersection exists.

Input / Output format: Line 1: a (nodes only in list A). Line 2: their values. Line 3: b (nodes only in list B). Line 4: their values. Line 5: c (shared nodes). Line 6: shared values. Output: value of the first shared node, or -1 if c = 0.

Example

Input

```
2
4 1
3
5 6 1
3
8 4 5
```

Output

```
8
```

Explanation: List A is 4 1 8 4 5 and list B is 5 6 1 8 4 5; they first share the node with value 8.

Constraints

- $0 \leq a, b, c \leq 10^4$
- $1 \leq a + c$ and $1 \leq b + c$
- $-10^5 \leq \text{value} \leq 10^5$

Test Cases

Test 1 — Basic

```
Input:
2
4 1
3
5 6 1
3
8 4 5

Output:
8
```

Test 2 — No intersection

```
Input:
2
1 2
2
3 4
0

Output:
-1
```

Test 3 — Both lists are the same list

```
Input:
```

```
0
0
3
1 2 3

Output:
1
```

Test 4 — One list is a tail of the other

```
Input:
0

2
7 8
2
1 2

Output:
1
```

Test 5 — Single shared node

```
Input:
1
5
1
6
1
9

Output:
9
```

Test 6 — Equal values before the meeting point

```
Input:
2
3 4
```

```
2
3 4
2
5 6
```

```
Output:
5
```

32. Next Greater Element

Topic: Stack & Queues

Problem Statement

For every element of an array, find the first element to its right that is strictly greater than it. This element is called its next greater element. If no such element exists, the answer for that position is -1. Print the answers for all positions in the original order. A brute force search costs quadratic time. A monotonic stack makes it linear: scan from left to right, keep indices whose next greater element is still unknown in a stack with non-increasing values, and whenever the current element is larger than the top of the stack, pop the stack and record the current element as its answer.

Input / Output format: Line 1: n. Line 2: n integers. Output: n integers, the next greater element of each position.

Example

Input

```
4
4 5 2 25
```

Output

```
5 25 25 -1
```

Explanation: $4 \rightarrow 5$, $5 \rightarrow 25$, $2 \rightarrow 25$, and 25 has none: 5 25 25 -1.

Constraints

- $1 \leq n \leq 10^5$
- $-10^9 \leq a[i] \leq 10^9$

Test Cases

Test 1 — Basic

```
Input:
5
13 7 6 12 5

Output:
-1 12 12 -1 -1
```

Test 2 — Single element

```
Input:
1
3

Output:
-1
```

Test 3 — Strictly decreasing

```
Input:
4
9 7 5 3

Output:
-1 -1 -1 -1
```

Test 4 — Strictly increasing

```
Input:
4
1 2 3 4

Output:
2 3 4 -1
```

Test 5 — All equal

```
Input:
4
5 5 5 5

Output:
-1 -1 -1 -1
```

Test 6 — Negative values

```
Input:
5
-3 -5 -1 -2 0

Output:
-1 -1 0 0 -1
```

33. Meeting Rooms

Topic: Greedy Algorithms

Problem Statement

You are given the start and end times of several meetings, each represented as an interval [start, end). Determine whether one person can attend all of the meetings. A person can move directly from one meeting to another if the earlier meeting ends at exactly the same time the next one starts, so touching intervals do not conflict. Sort the intervals by start time and check that every meeting starts no earlier than the previous meeting ends. Any overlap means the person cannot attend all meetings. Print true if all meetings can be attended without overlaps, and false otherwise.

Input / Output format: Line 1: n. Next n lines: 'start end'. Output: true or false.

Example

Input

```
3
0 30
5 10
15 20
```

Output

```
false
```

Explanation: Meeting [5, 10) lies inside [0, 30), so they overlap: false.

Constraints

- $1 \leq n \leq 10^4$
- $0 \leq \text{start} < \text{end} \leq 10^6$

Test Cases

Test 1 — No overlap

```
Input:
3
7 10
2 4
12 15

Output:
true
```

Test 2 — Single meeting

```
Input:
1
1 5

Output:
true
```

Test 3 — Touching intervals are allowed

```
Input:
2
1 5
5 9

Output:
true
```

Test 4 — Overlap by one unit

```
Input:
2
1 6
5 9

Output:
false
```

Test 5 — Identical meetings

```
Input:
2
3 8
3 8

Output:
false
```

Test 6 — Unsorted input with overlap

```
Input:
4
20 25
1 5
10 22
6 9

Output:
false
```

34. Asteroid Collision

Topic: Arrays

Problem Statement

An array describes asteroids moving along a line. The absolute value of each number is the size of the asteroid, and the sign is its direction: positive moves to the right and negative moves to the left. All asteroids move at the same speed. When two asteroids meet, the smaller one explodes; if they have equal size, both explode; asteroids moving in the same direction never meet. Simulate all collisions and print the sizes with directions of the asteroids that survive, in their original order. A stack handles this neatly, because a new left-moving asteroid can only collide with right-moving asteroids at the top. Print [] if none survive.

Input / Output format: Line 1: n. Line 2: n non-zero integers. Output: the surviving asteroids, space separated ([] if none).

Example

Input

```
3
5 10 -5
```

Output

```
5 10
```

Explanation: 10 and -5 collide and -5 explodes; 5 and 10 never meet. Result: 5 10.

Constraints

- $2 \leq n \leq 10^4$

- $-1000 \leq a[i] \leq 1000$
- $a[i] \neq 0$

Test Cases

Test 1 — Both explode on equal size

```
Input:
2
8 -8

Output:
[]
```

Test 2 — Big left mover survives

```
Input:
4
1 2 3 -10

Output:
-10
```

Test 3 — Chain of collisions

```
Input:
5
10 2 -5 -7 4

Output:
10 4
```

Test 4 — No collisions, all move left

```
Input:
3
-2 -1 -3

Output:
-2 -1 -3
```

Test 5 — No collisions, left then right

```
Input:
4
-2 -1 1 2

Output:
-2 -1 1 2
```

Test 6 — Everything explodes

```
Input:
4
2 1 -1 -2

Output:
[]
```

35. Single Element in a Sorted Array

Topic: Binary Search

Problem Statement

You are given a sorted array in which every element appears exactly twice, except for one element that appears only once. Find and print the single element. The array length is always odd. Although XOR-ing all elements gives the answer in linear time, the sorted order allows a solution in $O(\log n)$ time and $O(1)$ space. Before the single element, the first copy of each pair is at an even index; after it, the pairs are shifted and the first copy lies at an odd index. Use this parity pattern to decide whether to keep searching in the left half or in the right half.

Input / Output format: Line 1: n (odd). Line 2: n sorted integers. Output: the element that appears once.

Example

Input

```
9
1 1 2 3 3 4 4 8 8
```

Output

```
2
```

Explanation: Every value except 2 appears twice, so the answer is 2.

Constraints

- $1 \leq n \leq 10^5$, n is odd
- $0 \leq a[i] \leq 10^5$

Test Cases

Test 1 — Basic

```
Input:
7
1 1 2 2 3 4 4

Output:
3
```

Test 2 — Single element array

```
Input:
1
7

Output:
7
```

Test 3 — Unique value at the start

```
Input:
5
1 2 2 3 3

Output:
1
```

Test 4 — Unique value at the end

```
Input:
5
1 1 2 2 3

Output:
3
```

Test 5 — Unique value in the middle

```
Input:
5
1 1 2 3 3

Output:
2
```

Test 6 — Zero as the single element

```
Input:
3
0 1 1

Output:
0
```

36. Finding the Equilibrium Index in an Array

Topic: Arrays

Problem Statement

An equilibrium index of an array is an index where the sum of all elements strictly to its left equals the sum of all elements strictly to its right. The element at the index itself is not included in either sum, and an empty side has a sum of zero. Given an array of integers, print the smallest equilibrium index, or -1 if there is none. Recomputing both sums for every position takes quadratic time, but you can compute the total sum once and keep a running left sum while scanning, deriving the right sum as total minus left minus the current element. This takes linear time.

Input / Output format: Line 1: n . Line 2: n integers. Output: the leftmost equilibrium index or -1.

Example

Input

```
7
-7 1 5 2 -4 3 0
```

Output

```
3
```

Explanation: Left sum of index 3 is $-7 + 1 + 5 = -1$ and the right sum is $-4 + 3 + 0 = -1$, so print 3.

Constraints

- $1 \leq n \leq 10^5$
- $-10^4 \leq a[i] \leq 10^4$

Test Cases

Test 1 — Basic

```
Input:
6
1 7 3 6 5 6

Output:
3
```

Test 2 — Single element

```
Input:
1
10

Output:
0
```

Test 3 — No equilibrium

```
Input:
3
1 2 3

Output:
-1
```

Test 4 — Equilibrium at index 0

```
Input:
4
5 -2 1 1

Output:
0
```

Test 5 — Equilibrium at last index

```
Input:
3
1 -1 5

Output:
2
```


Test 6 — Multiple equilibria, first reported

```
Input:
5
0 0 0 0 0

Output:
0
```

37. Lowest Common Ancestor in a BST

Topic: Binary Search Trees

Problem Statement

Given a binary search tree and two nodes p and q that are present in it, find their lowest common ancestor, meaning the deepest node that has both p and q among its descendants. A node is allowed to be a descendant of itself, so if p is an ancestor of q , then p is the answer. Use the ordering property of the BST instead of exploring the whole tree: starting from the root, if both values are smaller than the current node move to the left child, if both are larger move to the right child, and otherwise the current node is the split point and hence the answer. Print the value of that node.

Input / Output format: Line 1: the BST in level-order ('null' for missing). Line 2: p and q .
Output: value of the LCA.

Example

Input

```
6 2 8 0 4 7 9 null null 3 5
2 8
```

Output

```
6
```

Explanation: Node 2 is in the left subtree and node 8 in the right subtree of 6, so the LCA is 6.

Constraints

- $2 \leq \text{number of nodes} \leq 10^5$
- All values are unique
- $p \neq q$ and both exist in the tree

Test Cases

Test 1 — Basic

```
Input:
6 2 8 0 4 7 9 null null 3 5
3 5

Output:
4
```

Test 2 — Ancestor is one of the nodes

```
Input:
6 2 8 0 4 7 9 null null 3 5
```

```
2 4
```

```
Output:  
2
```

Test 3 — Root is the LCA

```
Input:  
6 2 8 0 4 7 9  
0 9
```

```
Output:  
6
```

Test 4 — Two-node tree

```
Input:  
2 1  
2 1
```

```
Output:  
2
```

Test 5 — Both nodes in right subtree

```
Input:  
6 2 8 0 4 7 9  
7 9
```

```
Output:  
8
```

Test 6 — Deep nodes in left subtree

```
Input:
```

```
6 2 8 0 4 7 9 null null 3 5
0 3
```

```
Output:
2
```

38. Single Number – II

Topic: Arrays

Problem Statement

In an array of integers, every element appears exactly three times except for one element that appears exactly once. Find the element that appears only once. A hash map of counts would solve it, but the intended solution uses constant extra space and linear time by working on individual bits: for each bit position, the total number of set bits across all numbers modulo three equals the corresponding bit of the unique element. A pair of state variables, often called ones and twos, can track these counts modulo three for all bit positions at once, including negative numbers stored in two's complement form.

Input / Output format: Line 1: n. Line 2: n integers. Output: the element that appears once.

Example

Input

```
4
2 2 3 2
```

Output

3

Explanation: 2 appears three times and 3 appears once, so the answer is 3.

Constraints

- $1 \leq n \leq 3 \cdot 10^4$ ($n \equiv 1 \pmod 3$)
- $-2^{31} \leq a[i] \leq 2^{31} - 1$

Test Cases

Test 1 — Basic

```
Input:
7
0 1 0 1 0 1 99

Output:
99
```

Test 2 — Single element

```
Input:
1
-4

Output:
-4
```

Test 3 — Unique value is negative

```
Input:
4
5 5 -7 5

Output:
-7
```

Test 4 — Unique value is zero

```
Input:
4
3 0 3 3

Output:
0
```

Test 5 — Large values

```
Input:
4
2147483647 2147483647 -2147483648 2147483647

Output:
-2147483648
```

Test 6 — Unique value appears first

```
Input:
7
8 4 4 4 6 6 6

Output:
8
```

39. Best Time to Buy and Sell Stock – IV

Topic: Dynamic Programming

Problem Statement

You are given the daily price of a stock for n days and an integer k . You may complete at most k transactions, where one transaction means buying one share and later selling it. You cannot hold more than one share at a time, so you must sell before you buy again. Compute the maximum profit you can achieve; if no profitable trade exists the answer is 0. A dynamic programming solution keeps, for every number of completed transactions j , the best balance while holding a share and the best balance while not holding one, updating them for each day. Print the maximum profit.

Input / Output format: Line 1: k and n . Line 2: n prices. Output: the maximum profit.

Example

Input

```
2 6
3 2 6 5 0 3
```

Output

```
7
```

Explanation: Buy at 2, sell at 6 (profit 4), then buy at 0 and sell at 3 (profit 3). Total 7.

Constraints

- $0 \leq k \leq 100$
- $1 \leq n \leq 1000$
- $0 \leq \text{price}[i] \leq 1000$

Test Cases

Test 1 — Basic

```
Input:
2 6
3 3 5 0 0 3

Output:
5
```

Test 2 — $k = 0$, no transactions

```
Input:
0 3
1 2 3

Output:
0
```

Test 3 — Single day

```
Input:
2 1
5

Output:
0
```

Test 4 — Prices only fall


```
Input:
3 4
9 7 5 3

Output:
0
```

Test 5 — k larger than needed

```
Input:
10 5
1 2 3 4 5

Output:
4
```

Test 6 — Limited by k = 1

```
Input:
1 6
7 1 5 3 6 4

Output:
5
```

40. Number of Substrings Containing All Three Characters

Topic: Strings

Problem Statement

You are given a string made only of the characters 'a', 'b' and 'c'. Count how many substrings contain at least one occurrence of each of the three characters. Two substrings that have the same text but different positions are counted separately. Checking every substring costs quadratic or cubic time. A better idea is to move a right pointer over the string, and for every right end, count how many left starts still give a valid substring: this equals one plus the smallest of the last seen positions of 'a', 'b' and 'c'. Alternatively use a sliding window with counts. Print the total count.

Input / Output format: One line: the string. Output: the number of valid substrings.

Example

Input

```
abcabc
```

Output

```
10
```

Explanation: Valid substrings: abc, abca, abcab, abcabc, bca, bcab, bcabc, cab, cabc, abc → 10.

Constraints

- $3 \leq |s| \leq 5 \cdot 10^4$
- s consists only of 'a', 'b', 'c'

Test Cases

Test 1 — Basic

```
Input:
aaacb

Output:
3
```

Test 2 — Minimum valid string

```
Input:
abc

Output:
1
```

Test 3 — No valid substring

```
Input:
aaabbb

Output:
0
```

Test 4 — All characters valid from every start

```
Input:
cba

Output:
1
```

Test 5 — Repeated pattern

```
Input:
abcabcabc
```

```
Output:
28
```

Test 6 — Long block then rest

```
Input:
aaaabbbbcccc
```

```
Output:
16
```

41. Remove K Digits

Topic: Stack & Queues

Problem Statement

You are given a non-negative integer as a string and an integer k . Remove exactly k digits from the number so that the new number is as small as possible, and print it without leading zeros. If the result becomes empty or contains only zeros, print 0. The order of the remaining digits must not change. A greedy stack solution works well: scan the digits from left to right, and while the top of the stack is larger than the current digit and removals remain, pop the stack. After the scan, if removals are still left, remove digits from the end. Finally strip any leading zeros.

Input / Output format: Line 1: the number as a string. Line 2: k . Output: the smallest possible number.

Example

Input

```
1432219
3
```

Output

```
1219
```

Explanation: Removing 4, 3 and 2 gives 1219, the smallest possible result.

Constraints

- $1 \leq k \leq |\text{num}| \leq 10^5$
- num has no leading zeros except when it is 0

Test Cases

Test 1 — Basic

```
Input:
10200
1

Output:
200
```

Test 2 — Remove everything

```
Input:
9
1

Output:
0
```

Test 3 — Result has only zeros

```
Input:
100
2

Output:
0
```

Test 4 — Increasing digits remove from end

```
Input:
12345
2

Output:
123
```

Test 5 — Equal digits

```
Input:
1111
2

Output:
11
```

Test 6 — Leading zeros stripped

```
Input:
10001
4

Output:
0
```

42. Sum of Subarray Minimums

Topic: Stack & Queues

Problem Statement

Given an integer array, consider every contiguous subarray, take the minimum element of each of them and add all these minimums together. Print the total modulo 1,000,000,007 because it can be very large. A direct enumeration of all subarrays takes quadratic time. Instead count the contribution of each element: an element is the minimum of all subarrays that start within the range extending left until a smaller element and end within the range extending right until a smaller or equal element. Two monotonic stack passes find these boundaries, and the contribution of an element is its value times the product of the left and right counts.

Input / Output format: Line 1: n. Line 2: n integers. Output: the sum of subarray minimums modulo $10^9 + 7$.

Example

Input

```
4
3 1 2 4
```

Output

```
17
```

Explanation: Subarray minimums: 3, 1, 2, 4, 1, 1, 2, 1, 1, 1 $\rightarrow$ total 17.

Constraints

- $1 \leq n \leq 3 \cdot 10^4$
- $1 \leq a[i] \leq 3 \cdot 10^4$

Test Cases

Test 1 — Basic

```
Input:
5
11 81 94 43 3

Output:
444
```

Test 2 — Single element

```
Input:
1
7

Output:
7
```

Test 3 — All equal


```
Input:
4
2 2 2 2

Output:
20
```

Test 4 — Strictly increasing

```
Input:
4
1 2 3 4

Output:
20
```

Test 5 — Strictly decreasing

```
Input:
4
4 3 2 1

Output:
20
```

Test 6 — Large values need the modulo

[illegible]

Input

```
5 3 6 2 4 null 7
9
```

Output

```
true
```

Explanation: Nodes 2 and 7 sum to 9, so the answer is true.

Constraints

- $1 \leq \text{number of nodes} \leq 10^4$
- $-10^4 \leq \text{value} \leq 10^4$
- $-10^5 \leq k \leq 10^5$
- The tree is a valid BST

Test Cases

Test 1 — Pair exists

```
Input:
5 3 6 2 4 null 7
9
```

```
Output:
true
```

Test 2 — No pair exists

```
Input:
5 3 6 2 4 null 7
28

Output:
false
```

Test 3 — Single node cannot pair with itself

```
Input:
2
4

Output:
false
```

Test 4 — Two-node tree with valid pair

```
Input:
2 1
3

Output:
true
```

Test 5 — Negative values

```
Input:
0 -3 3 -5 -1
-4

Output:
true
```

Test 6 — Value would need to be reused

```
Input:
3 1 4
6

Output:
false
```

44. Find Out How Many Times the Array Is Rotated

Topic: Binary Search

Problem Statement

A sorted array of distinct integers was rotated to the right some unknown number of times, so an array such as 1 2 3 4 5 might become 3 4 5 1 2. Given the rotated array, find how many times it has been rotated. The number of rotations equals the index of the minimum element. Since the underlying array was sorted, this can be found in $O(\log n)$ time using binary search: compare the middle element with the last element to decide whether the minimum lies in the left or the right half. An array that is already sorted has been rotated zero times. Print the rotation count.

Input / Output format: Line 1: n . Line 2: n distinct integers (a rotated sorted array). Output: the number of rotations.

Example

Input

```
5
```

```
4 5 1 2 3
```

Output

```
2
```

Explanation: The minimum 1 is at index 2, so the array was rotated 2 times.

Constraints

- $1 \leq n \leq 10^5$
- $-10^9 \leq a[i] \leq 10^9$
- All elements are distinct

Test Cases

Test 1 — Basic

```
Input:
6
15 18 2 3 6 12

Output:
2
```

Test 2 — Single element

```
Input:
1
5

Output:
```

```
0
```

Test 3 — Not rotated

```
Input:
5
1 2 3 4 5

Output:
0
```

Test 4 — Rotated by n - 1

```
Input:
5
2 3 4 5 1

Output:
4
```

Test 5 — Two elements rotated

```
Input:
2
2 1

Output:
1
```

Test 6 — Negative values

```
Input:
5
0 3 -9 -4 -1
```

```
Output:  
2
```

45. Maximum Points You Can Obtain from Cards

Topic: Arrays

Problem Statement

Several cards are arranged in a row and each card has an integer number of points. In one step you may take a card either from the beginning or from the end of the row. You must take exactly k cards in total. Find the maximum total number of points you can obtain. Trying all sequences of choices is exponential, but notice that the cards you take always form a prefix plus a suffix of the row, which means the cards you leave behind form one contiguous window of size $n - k$. Minimise the window sum with a sliding window, or slide a boundary between prefix and suffix, and print the best total.

Input / Output format: Line 1: n and k . Line 2: n card values. Output: the maximum score.

Example

Input

```
7 3  
1 2 3 4 5 6 1
```

Output


```
12
```

Explanation: Take the last three cards, 5, 6 and 1, for a total of 12.

Constraints

- $1 \leq k \leq n \leq 10^5$
- $1 \leq \text{points}[i] \leq 10^4$

Test Cases

Test 1 — Basic

```
Input:
5 3
2 11 4 5 3

Output:
17
```

Test 2 — Take one card

```
Input:
4 1
3 9 2 5

Output:
5
```

Test 3 — Take all cards

```
Input:
3 3
1 79 80

Output:
160
```

Test 4 — All cards equal

```
Input:
5 2
5 5 5 5 5

Output:
10
```

Test 5 — Best cards in the middle

```
Input:
6 2
1 1 100 100 1 1

Output:
2
```

Test 6 — Only from the end

```
Input:
5 2
1 1 1 8 9

Output:
17
```

46. Binary Subarrays with Sum

Topic: Hashing

Problem Statement

You are given a binary array that contains only zeros and ones and an integer goal. Count the number of non-empty contiguous subarrays whose elements sum to goal. Because the array is binary, all prefix sums are non-decreasing, which allows a prefix sum with hash map solution or a two window technique that counts subarrays with sum at most goal and subtracts those with sum at most goal - 1. Pay special attention to the case where goal is zero, because runs of consecutive zeros create many valid subarrays. Print the number of matching subarrays as a single integer.

Input / Output format: Line 1: n and goal. Line 2: n values (0 or 1). Output: the number of subarrays with sum equal to goal.

Example

Input

```
5 2
1 0 1 0 1
```

Output

```
4
```

Explanation: Valid subarrays: [1,0,1], [1,0,1,0], [0,1,0,1], [1,0,1] (last) → 4.

Constraints

- $1 \leq n \leq 3 \cdot 10^4$
- $0 \leq \text{goal} \leq n$
- $a[i] \in \{0, 1\}$

Test Cases

Test 1 — Basic

```
Input:
6 2
1 1 0 0 1 1

Output:
7
```

Test 2 — Single element, goal 1

```
Input:
1 1
1

Output:
1
```

Test 3 — Goal zero with zero runs

```
Input:
5 0
0 0 0 1 0

Output:
7
```

Test 4 — Goal equals total ones

```
Input:
4 4
1 1 1 1

Output:
1
```

Test 5 — No subarray has the goal

```
Input:
3 2
1 0 0

Output:
0
```

Test 6 — All zeros with goal zero

```
Input:
4 0
0 0 0 0

Output:
10
```

47. Matrix Median

Topic: Binary Search

Problem Statement

You are given a matrix with an odd number of rows and an odd number of columns in which each row is sorted in non-decreasing order. Find the median of all elements of the matrix, that is, the middle element if all values were written in sorted order. Flattening and sorting works in $O(r \cdot c \cdot \log(r \cdot c))$ time, but the intended method applies binary search on the value range. For a candidate value x , count how many elements are less than or equal to x using binary search in every row, and shrink the range until the smallest value whose count exceeds half of the total elements is found. Print the median.

Input / Output format: Line 1: r and c (both odd). Next r lines: the rows (each sorted).
Output: the median.

Example

Input

```
3 3
1 3 5
2 6 9
3 6 9
```

Output

```
5
```

Explanation: Sorted values are 1 2 3 3 5 6 6 9 9, whose middle element is 5.

Constraints

- $1 \leq r, c \leq 400$, both odd
- $1 \leq \text{matrix}[i][j] \leq 2000$
- Each row is sorted in non-decreasing order

Test Cases

Test 1 — Basic

```
Input:
3 3
1 3 8
2 3 4
1 2 5

Output:
3
```

Test 2 — Single cell

```
Input:
1 1
7

Output:
7
```

Test 3 — Single row

```
Input:
1 5
1 2 3 4 5

Output:
3
```

Test 4 — Single column

```
Input:
3 1
1
5
9
```

```
Output:  
5
```

Test 5 — All values equal

```
Input:  
3 3  
4 4 4  
4 4 4  
4 4 4  
  
Output:  
4
```

Test 6 — Heavy duplicates

```
Input:  
3 3  
1 1 1  
2 2 2  
3 3 3  
  
Output:  
2
```

48. Find the Repeating and Missing Number

Topic: Sorting

Problem Statement

You are given an array of size n that should contain every integer from 1 to n exactly once, but one number appears twice and one number is missing. Find both the repeating number and the missing number. Print them on one line, first the repeating value and then the missing value. You can sort the array and compare each element with its expected value, use a frequency array, or apply mathematics: the difference between the actual sum and the expected sum of $1..n$, and the difference between the actual and expected sums of squares, form two equations that reveal both numbers directly without extra space.

Input / Output format: Line 1: n . Line 2: n integers. Output: 'repeating missing'.

Example

Input

```
6
4 3 6 2 1 1
```

Output

```
1 5
```

Explanation: The value 1 appears twice and 5 is missing, so the answer is 1 5.

Constraints

- $2 \leq n \leq 10^5$
- $1 \leq a[i] \leq n$
- Exactly one value repeats and exactly one is missing

Test Cases

Test 1 — Basic

```
Input:
5
3 1 3 5 2

Output:
3 4
```

Test 2 — Minimum size

```
Input:
2
2 2

Output:
2 1
```

Test 3 — Missing number is 1

```
Input:
4
2 2 3 4

Output:
2 1
```

Test 4 — Missing number is n

```
Input:
4
1 2 3 3

Output:
3 4
```

Test 5 — Repeating number is 1

```
Input:
5
1 1 2 3 4

Output:
1 5
```

Test 6 — Repeating number is n

```
Input:
5
5 5 1 2 3

Output:
5 4
```

49. Correct a BST with Two Nodes Swapped

Topic: Binary Search Trees

Problem Statement

Two nodes of a binary search tree have had their values swapped by mistake, so the tree no longer satisfies the BST property. Recover the tree by swapping those two values back, without changing the structure of the tree, and print the corrected tree in level-order. An inorder traversal of a valid BST is sorted, so in the traversal of the damaged tree there will be one or two places where a value is larger than its successor. The first violation gives the first wrong node, and the last violation gives the second wrong node. Swap their values. An $O(1)$ space solution uses Morris traversal.

Input / Output format: One line: the damaged BST in level-order ('null' for missing).
Output: the corrected tree in level-order.

Example

Input

```
1 3 null null 2
```

Output

```
3 1 null null 2
```

Explanation: Swapping 1 and 3 restores the BST: 3 1 null null 2.

Constraints

- $2 \leq \text{number of nodes} \leq 1000$
- $-2^{31} \leq \text{value} \leq 2^{31} - 1$
- Exactly two nodes were swapped
- All values are distinct

Test Cases

Test 1 — Adjacent nodes swapped

```
Input:  
3 1 4 null null 2
```

```
Output:
```

```
2 1 4 null null 3
```

Test 2 — Root swapped with a leaf

```
Input:  
9 3 8 2 4 7 5  
  
Output:  
5 3 8 2 4 7 9
```

Test 3 — Two-node tree

```
Input:  
1 2  
  
Output:  
2 1
```

Test 4 — Left subtree swap

```
Input:  
4 2 6 3 1 5 7  
  
Output:  
4 2 6 1 3 5 7
```

Test 5 — Right subtree swap

```
Input:  
4 2 6 1 3 7 5  
  
Output:  
4 2 6 1 3 5 7
```

Test 6 — Negative values

```
Input:
0 -3 5 -1 -5

Output:
0 -3 5 -5 -1
```

50. Job Sequencing Problem

Topic: Greedy Algorithms

Problem Statement

You are given n jobs, each with an identifier, a deadline and a profit. Every job takes exactly one unit of time to complete, only one job can run at a time, and a job earns its profit only if it finishes on or before its deadline, with time slots numbered starting from one. Choose and schedule jobs to maximise the total profit. Print two integers: the number of jobs that are completed and the maximum total profit. The greedy method sorts jobs by decreasing profit and places each job in the latest free slot not later than its deadline; a disjoint set union structure speeds up finding that slot.

Input / Output format: Line 1: n . Next n lines: 'id deadline profit'. Output: 'jobsDone maxProfit'.

Example

Input

```
1 4 20
2 1 10
3 1 40
4 1 30
```

Output

```
2 60
```

Explanation: Only one job fits before time 1, so pick job 3 (40) and job 1 (20): 2 jobs, profit 60.

Constraints

- $1 \leq n \leq 10^5$
- $1 \leq \text{deadline} \leq n$
- $1 \leq \text{profit} \leq 500$

Test Cases

Test 1 — Basic

```
Input:
5
1 2 100
2 1 19
3 2 27
4 1 25
5 1 15

Output:
2 127
```

Test 2 — Single job

```
Input:
1
1 1 50

Output:
1 50
```

Test 3 — All deadlines equal 1

```
Input:
3
1 1 5
2 1 9
3 1 7

Output:
1 9
```

Test 4 — All jobs can be scheduled

```
Input:
3
1 3 10
2 2 20
3 1 30

Output:
3 60
```

Test 5 — Late deadlines leave gaps

```
Input:
4
1 4 10
2 4 20
3 4 30
4 4 40
```



```
Output:
4 100
```

Test 6 — Ties in profit

```
Input:
4
1 2 10
2 2 10
3 2 10
4 1 10

Output:
2 20
```

51. LFU Cache

Topic: Linked List (Design)

Problem Statement

Design a Least Frequently Used cache with a fixed capacity supporting `get(key)` and `put(key, value)` in average constant time. The `get` operation returns the value for an existing key, or `-1` if it is missing. The `put` operation inserts or updates a key. Every successful `get` or `put` of a key increases its use counter by one. When the cache is full and a new key must be inserted, evict the key with the smallest use counter; if several keys share that counter, evict the least recently used among them. Doubly linked lists grouped by frequency together with a hash map achieve the required speed. Print the result of each `get` on its own line.

Input / Output format: Line 1: capacity. Line 2: `q`. Next `q` lines: 'put key value' or 'get key'. Output: result of each `get`, one per line ('(no output)' if none).

Example

Input

```
2
9
put 1 1
put 2 2
get 1
put 3 3
get 2
get 3
put 4 4
get 1
get 3
```

Output

```
1
-1
3
-1
3
```

Explanation: Key 2 is evicted when 3 arrives (lowest frequency). Key 1 is evicted when 4 arrives (tie on frequency, least recent). Results: 1, -1, 3, -1, 3.

Constraints

- $0 \leq \text{capacity} \leq 10^4$
- $1 \leq q \leq 2 \cdot 10^5$
- $0 \leq \text{key} \leq 10^5$
- $0 \leq \text{value} \leq 10^9$

Test Cases

Test 1 — Basic

```
Input:
2
6
put 1 10
put 2 20
get 1
put 3 30
get 2
get 3

Output:
10
-1
30
```

Test 2 — Capacity one

```
Input:
1
5
put 1 1
put 2 2
get 1
get 2
get 2

Output:
-1
2
2
```

Test 3 — Capacity zero

```
Input:
0
3
put 1 1
get 1
get 2

Output:
-1
-1
```

Test 4 — Tie broken by recency

```
Input:
2
7
put 1 1
put 2 2
put 3 3
get 1
get 2
get 3
get 1
```

```
Output:
-1
2
3
-1
```

Test 5 — Update counts as a use

```
Input:
2
8
put 1 1
put 2 2
put 1 5
put 3 3
get 1
get 2
get 3
get 1
```

```
Output:
5
-1
3
5
```

Test 6 — Only puts, no gets

```
Input:
2
```

```
3
put 1 1
put 2 2
put 3 3

Output:
(no output)
```

52. Trie Implementation and Advanced Operations

Topic: Tries

Problem Statement

Implement a trie, also called a prefix tree, that stores lowercase words and supports the following commands: `insert(word)`, `erase(word)`, `search(word)`, `startsWith(prefix)`, `countWords(word)` and `countPrefix(prefix)`. Words may be inserted several times, and each insertion counts separately. Erasing removes one occurrence and does nothing if the word is absent. `search` and `startsWith` answer true or false, while `countWords` returns how many copies of the exact word are stored and `countPrefix` returns how many stored words start with the given prefix, counting duplicates. Keep two counters in every trie node to support this efficiently. Print the result of each query command.

Input / Output format: Line 1: q . Next q lines: '`<command> <string>`' with command in {`insert`, `erase`, `search`, `startsWith`, `countWords`, `countPrefix`}. Output: one line per query command (all except `insert` and `erase`).

Example

Input

```
8
insert apple
insert app
search app
```

```
startsWith appl
countPrefix ap
erase app
search app
countWords apple
```

Output

```
true
true
2
false
1
```

Explanation: Results: true, true, 2, false, 1.

Constraints

- $1 \leq q \leq 10^5$
- Strings contain only lowercase English letters
- $1 \leq \text{length} \leq 2000$

Test Cases

Test 1 — Basic

```
Input:
6
insert cat
insert car
search cat
search ca
startsWith ca
countPrefix ca
```

```
Output:
true
false
```

```
true
2
```

Test 2 — Single word

```
Input:
3
insert a
search a
countWords a
```

```
Output:
true
1
```

Test 3 — Duplicate insertions

```
Input:
5
insert dog
insert dog
countWords dog
erase dog
countWords dog
```

```
Output:
2
1
```

Test 4 — Erase a word that is absent

```
Input:
4
erase cow
search cow
startsWith c
countPrefix c
```

```
Output:
false
false
```

0

Test 5 — Prefix is not a word

```
Input:
4
insert internet
search inter
startsWith inter
countPrefix inter
```

```
Output:
false
true
1
```

Test 6 — Word that is a prefix of another

```
Input:
5
insert to
insert tomato
countPrefix to
countWords to
erase to
```

```
Output:
2
1
```

53. Largest BST in a Binary Tree

Topic: Binary Search Trees

Problem Statement

Given the root of a binary tree, find the size of the largest subtree that is also a valid binary search tree. The size of a subtree is the number of nodes it contains. A subtree consists of a node together with all of its descendants, so you cannot drop nodes from inside it. A valid BST requires all values in the left subtree to be smaller than the node and all values in the right subtree to be larger. Solve it in linear time with a post-order traversal that returns, for each node, whether its subtree is a BST, its size, and the minimum and maximum values in it. Print the size of the largest BST subtree.

Input / Output format: One line: the tree in level-order ('null' for missing). Output: the number of nodes in the largest BST subtree.

Example

Input

```
10 5 15 1 8 null 7
```

Output

```
3
```

Explanation: The subtree rooted at 5 (nodes 5, 1, 8) is the largest BST, so the size is 3.

Constraints

- $1 \leq \text{number of nodes} \leq 10^5$
- $-10^9 \leq \text{value} \leq 10^9$

Test Cases

Test 1 — Basic

```
Input:
50 30 60 5 20 45 70 null null null null null null 65 80

Output:
5
```

Test 2 — Single node

```
Input:
7

Output:
1
```

Test 3 — Whole tree is a BST

```
Input:
4 2 6 1 3 5 7

Output:
7
```

Test 4 — No BST larger than a leaf

```
Input:
5 8 3

Output:
1
```

Test 5 — Leaf is the only BST

```
Input:
10 20 5

Output:
1
```

Test 6 — Violation deep in the tree

```
Input:
20 10 30 5 25 null null

Output:
3
```

54. KMP Algorithm / LPS Array

Topic: Strings

Problem Statement

Given a text and a pattern, use the Knuth–Morris–Pratt algorithm to find every occurrence of the pattern in the text. First build the LPS array of the pattern, where $\text{lps}[i]$ is the length of the longest proper prefix of $\text{pattern}[0..i]$ that is also a suffix of it. Then scan the text; whenever a mismatch occurs, use the LPS array to fall back in the pattern instead of restarting, which guarantees linear time overall. Print two lines: the LPS array of the pattern on the first line, and on the second line the zero-based starting indices of all occurrences in increasing order, or -1 if the pattern does not occur. Overlapping matches are counted.

Input / Output format: Line 1: text. Line 2: pattern. Output: line 1 = LPS array; line 2 = match indices (or -1).

Example

Input

```
abxabcabcabby
abcaby
```

Output

```
0 0 0 1 2 0
6
```

Explanation: LPS of abcaby is 0 0 0 1 2 0. The pattern first appears at index 6.

Constraints

- $1 \leq |\text{pattern}| \leq |\text{text}| \leq 10^5$
- Strings contain lowercase English letters

Test Cases

Test 1 — Overlapping matches

```
Input:
aaaaa
aa
```

```
Output:
0 1
0 1 2 3
```

Test 2 — Pattern equals text

```
Input:
abc
abc

Output:
0 0 0
0
```

Test 3 — Pattern absent

```
Input:
abcdef
xyz

Output:
0 0 0
-1
```

Test 4 — Single character pattern

```
Input:
banana
a

Output:
0
1 3 5
```

Test 5 — Repeating prefix pattern

```
Input:
ababcabcbababd
ababd

Output:
0 0 1 2 0
10
```

Test 6 — All characters identical

```
Input:
aaaa
aaaa
```

```
Output:
0 1 2 3
0
```

55. Celebrity Problem

Topic: Arrays

Problem Statement

In a party of n people, a celebrity is a person who is known by everyone else at the party but does not know anybody else. You are given an n by n matrix M where $M[i][j] = 1$ means that person i knows person j , and 0 means that person i does not know person j . The diagonal entries $M[i][i]$ are ignored. Find the index of the celebrity, or print -1 if there is none. At most one celebrity can exist. A linear time method eliminates one candidate with every comparison using two pointers, and then verifies the remaining candidate against everybody in one final pass.

Input / Output format: Line 1: n . Next n lines: n integers (0 or 1). Output: celebrity index (0 -based) or -1 .

Example

Input

```
3
0 1 0
0 0 0
0 1 0
```

Output

```
1
```

Explanation: Persons 0 and 2 both know person 1, and person 1 knows nobody. Person 1 is the celebrity.

Constraints

- $1 \leq n \leq 500$
- $M[i][j] \in \{0, 1\}$

Test Cases

Test 1 — Celebrity at the last index

```
Input:
3
0 0 1
0 0 1
0 0 0

Output:
2
```

Test 2 — Single person

```
Input:
1
0

Output:
0
```

Test 3 — Nobody knows anybody

```
Input:
3
0 0 0
0 0 0
0 0 0

Output:
-1
```

Test 4 — Everyone knows everyone

```
Input:
3
0 1 1
1 0 1
1 1 0

Output:
-1
```

Test 5 — Candidate knows someone

```
Input:
3
0 1 1
0 0 1
1 0 0

Output:
-1
```


Test 6 — Celebrity at index 0

```
Input:
3
0 0 0
1 0 0
1 0 0

Output:
0
```

56. Rabin–Karp Algorithm

Topic: Strings

Problem Statement

Implement the Rabin–Karp string matching algorithm. Given a text and a pattern, find all positions where the pattern occurs in the text. The method computes a rolling polynomial hash of every window of the text with the same length as the pattern and compares it with the pattern's hash; when two hashes are equal the actual characters are compared to rule out collisions. The hash of the next window is obtained from the previous one in constant time by removing the leading character and appending the new one. Print the number of occurrences on the first line, and on the second line the zero-based starting indices in increasing order, or -1 if there are none.

Input / Output format: Line 1: text. Line 2: pattern. Output: line 1 = number of matches; line 2 = indices (or -1).

Example

Input

```
GEEKSFORGEEKS
GEEK
```

Output

```
2
0 8
```

Explanation: GEEK occurs at index 0 and at index 8, so the count is 2.

Constraints

- $1 \leq |\text{pattern}|, |\text{text}| \leq 10^5$
- Strings contain lowercase and uppercase English letters and digits

Test Cases

Test 1 — Basic

```
Input:
abracadabra
abra
```

```
Output:
2
0 7
```

Test 2 — Pattern longer than text

```
Input:
ab
abcd

Output:
0
-1
```

Test 3 — Single character text and pattern

```
Input:
a
a

Output:
1
0
```

Test 4 — No occurrence

```
Input:
hello
world

Output:
0
-1
```

Test 5 — Overlapping occurrences

```
Input:
aaaaaa
aaa

Output:
4
0 1 2 3
```

Test 6 — Case sensitive

```
Input:
aAaA
Aa

Output:
1
1
```

57. Z Function

Topic: Strings

Problem Statement

For a string s of length n , the Z array stores at index i the length of the longest substring starting at position i that is also a prefix of s . By convention, $z[0]$ is defined as n , the length of the entire string. Compute the Z array in linear time using the Z algorithm: maintain a window $[l, r)$ that is the rightmost segment matching a prefix, reuse the values already computed inside this window for new positions, and extend the match only when a position lies beyond the window. Print all n values separated by single spaces. The Z function can also be used for pattern matching by concatenating pattern, a separator and text.

Input / Output format: One line: the string s . Output: the Z array, space separated.

Example

Input

```
aabxaab
```

Output

```
7 1 0 0 3 1 0
```

Explanation: $z = 7\ 1\ 0\ 0\ 3\ 1\ 0$: for example at index 4 the substring aab matches the prefix for 3 characters.

Constraints

- $1 \leq |s| \leq 10^5$
- s contains lowercase English letters

Test Cases

Test 1 — Basic

```
Input:
aabcaabxaaaz

Output:
12 1 0 0 3 1 0 0 2 2 1 0
```

Test 2 — Single character

```
Input:
a

Output:
1
```

Test 3 — All characters identical

```
Input:
aaaaa

Output:
5 4 3 2 1
```

Test 4 — No repeated prefix

```
Input:
abcdef

Output:
6 0 0 0 0 0
```

Test 5 — Periodic string

```
Input:
abababab

Output:
8 0 6 0 4 0 2 0
```

Test 6 — Two characters

```
Input:
ab

Output:
2 0
```

58. Build Heap from a Given Array

Topic: Arrays

Problem Statement

Given an array of n integers, rearrange it in place so that it represents a binary min-heap stored in array form, where the children of the element at index i are at indices $2i + 1$ and $2i + 2$, and every parent is not larger than its children. Use the bottom-up construction of Floyd: start from the last non-leaf index $n/2 - 1$ and move toward index 0, calling the sift-down procedure on each index. Sift-down swaps the element with its smaller child while a smaller child exists. This builds the heap in $O(n)$ time, which is faster than n separate insertions. Print the resulting array in index order.

Input / Output format: Line 1: n . Line 2: n integers. Output: the array after building the min-heap (Floyd's bottom-up procedure, sift-down choosing the smaller child, left child first on ties).

Example

Input

```
6
9 4 7 1 -2 6
```

Output

```
-2 1 6 9 4 7
```

Explanation: After sift-down at indices 2, 1 and 0 the heap array is -2 1 6 9 4 7.

Constraints

- $1 \leq n \leq 10^5$
- $-10^9 \leq a[i] \leq 10^9$

Test Cases

Test 1 — Basic

```
Input:
7
5 3 8 1 9 2 7

Output:
1 3 2 5 9 8 7
```

Test 2 — Single element

```
Input:
1
42

Output:
42
```

Test 3 — Already a min-heap

```
Input:
5
1 2 3 4 5

Output:
1 2 3 4 5
```


Test 4 — Descending order

```
Input:
5
5 4 3 2 1

Output:
1 2 3 5 4
```

Test 5 — All equal

```
Input:
4
7 7 7 7

Output:
7 7 7 7
```

Test 6 — Negative values

```
Input:
6
0 -5 3 -2 -9 4

Output:
-9 -5 3 -2 0 4
```

59. Sort a Linked List of 0's, 1's and 2's

Topic: Linked List

Problem Statement

You are given a singly linked list whose nodes contain only the values 0, 1 and 2. Sort the list in ascending order and print its values from head to tail. Rather than copying the values into an array, rearrange the nodes themselves: build three separate lists for the zeros, ones and twos while traversing the original list once, and then connect them as zeros followed by ones followed by twos. Dummy head nodes for each of the three lists remove the need for special cases. The algorithm runs in linear time and uses constant extra space apart from the list itself. A list with a single node is already sorted.

Input / Output format: Line 1: n. Line 2: n values (each 0, 1 or 2). Output: the sorted list.

Example

Input

```
6
1 2 0 1 2 0
```

Output

```
0 0 1 1 2 2
```

Explanation: Grouping the nodes by value gives 0 0 1 1 2 2.

Constraints

- $1 \leq n \leq 10^5$
- $\text{value} \in \{0, 1, 2\}$

Test Cases

Test 1 — Basic

```
Input:
8
2 1 0 2 1 0 1 2

Output:
0 0 1 1 1 2 2 2
```

Test 2 — Single node

```
Input:
1
1

Output:
1
```

Test 3 — Only zeros

```
Input:
3
0 0 0

Output:
0 0 0
```

Test 4 — Already sorted

```
Input:
5
0 0 1 2 2

Output:
0 0 1 2 2
```

Test 5 — Reverse order

```
Input:
6
2 2 1 1 0 0

Output:
0 0 1 1 2 2
```

Test 6 — Two distinct values

```
Input:
5
2 0 2 0 2

Output:
0 0 2 2 2
```

60. 3 Sum

Topic: Arrays

Problem Statement

Given an integer array, find all unique triplets of values (a, b, c) such that $a + b + c = 0$. The three numbers must come from three different positions of the array, but the answer must not contain the same triplet of values twice, even if it can be formed from different positions. Print each triplet in non-decreasing order on its own line, and sort the triplets lexicographically. If no triplet exists, print []. The standard approach sorts the array, fixes the first element, and uses two pointers on the rest of the array while skipping duplicate values, giving quadratic time and no extra hash sets.

Input / Output format: Line 1: n. Line 2: n integers. Output: each triplet on its own line ([] if none).

Example

Input

```
6
-1 0 1 2 -1 -4
```

Output

```
-1 -1 2
-1 0 1
```

Explanation: Unique triplets are -1 -1 2 and -1 0 1.

Constraints

- $1 \leq n \leq 3000$
- $-10^5 \leq a[i] \leq 10^5$

Test Cases

Test 1 — Basic

```
Input:
7
-2 0 1 1 2 -1 -4

Output:
```

```
-2 0 2  
-2 1 1  
-1 0 1
```

Test 2 — Fewer than three elements

```
Input:  
2  
0 0  
  
Output:  
[]
```

Test 3 — All zeros

```
Input:  
5  
0 0 0 0 0  
  
Output:  
0 0 0
```

Test 4 — No triplet exists

```
Input:  
4  
1 2 3 4  
  
Output:  
[]
```

Test 5 — Duplicate values collapse

```
Input:  
6
```

```
-1 -1 -1 2 2 0
```

```
Output:
```

```
-1 -1 2
```

Test 6 — Large magnitudes

```
Input:
```

```
5
```

```
-100000 0 100000 50000 -50000
```

```
Output:
```

```
-100000 0 100000
```

```
-50000 0 50000
```

61. Longest Repeating Character Replacement

Topic: Strings

Problem Statement

You are given a string of uppercase English letters and an integer k . In one operation you may choose any character of the string and change it to any other uppercase letter, and you may perform at most k such operations. Find the length of the longest substring that can be made to contain a single repeated letter after the operations. Use a sliding window: the window is valid while its length minus the count of its most frequent letter does not exceed k , meaning the other letters can be replaced. Expand the right end and move the left end when the window becomes invalid. Print the maximum valid window length.

Input / Output format: Line 1: the string. Line 2: k . Output: one integer.

Example

Input

```
AABABBA
1
```

Output

```
4
```

Explanation: Replace one A with B to get BBBB... the longest window is 4.

Constraints

- $1 \leq |s| \leq 10^5$
- s contains only uppercase English letters
- $0 \leq k \leq |s|$

Test Cases

Test 1 — Basic

```
Input:
ABAB
2

Output:
4
```

Test 2 — Single character


```
Input:
A
0

Output:
1
```

Test 3 — $k = 0$, no replacements

```
Input:
ABCDE
0

Output:
1
```

Test 4 — All characters identical

```
Input:
AAAA
2

Output:
4
```

Test 5 — k covers the whole string

```
Input:
ABCD
3

Output:
4
```

Test 6 — Multiple candidate windows

```
Input:
AAABBBCCCA
2
```

```
Output:
5
```

62. 4 Sum

Topic: Arrays

Problem Statement

Given an integer array and a target value, find all unique quadruplets of values (a, b, c, d) taken from four different positions such that their sum equals the target. The answer must not contain duplicate quadruplets. Print every quadruplet in non-decreasing order on its own line, and print the quadruplets in lexicographic order. If none exist, print []. Sort the array, fix the first two elements with nested loops and use two pointers for the remaining pair, skipping repeated values at every level to avoid duplicates. This reduces the running time to cubic instead of the quartic time of a naive approach. Watch out for integer overflow with large values.

Input / Output format: Line 1: n. Line 2: n integers. Line 3: target. Output: each quadruplet on its own line ([] if none).

Example

Input

```
6
1 0 -1 0 -2 2
0
```

Output

```
-2 -1 1 2
-2 0 0 2
-1 0 0 1
```

Explanation: Unique quadruplets: -2 -1 1 2, -2 0 0 2, -1 0 0 1.

Constraints

- $1 \leq n \leq 200$
- $-10^9 \leq a[i], \text{target} \leq 10^9$

Test Cases

Test 1 — Basic

```
Input:
6
2 2 2 2 2 2
8

Output:
2 2 2 2
```

Test 2 — Fewer than four elements

```
Input:
3
1 2 3
6

Output:
```

```
[]
```

Test 3 — No quadruplet

```
Input:
5
1 2 3 4 5
100

Output:
[]
```

Test 4 — Duplicates collapse

```
Input:
6
0 0 0 0 0 0
0

Output:
0 0 0 0
```

Test 5 — Large values

```
Input:
4
1000000000 1000000000 1000000000 1000000000
4000000000

Output:
1000000000 1000000000 1000000000 1000000000
```

Test 6 — Negative target

```
Input:
```

```
5
-3 -1 0 -2 -4
-7
```

```
Output:
-4 -2 -1 0
```

63. Longest Subarray with Sum K

Topic: Arrays

Problem Statement

Given an array of integers that can contain positive numbers, negative numbers and zeros, and an integer k , find the length of the longest contiguous subarray whose elements sum to exactly k . If there is no such subarray, print 0. Because negative values are allowed, the two pointer sliding window technique cannot be applied. Instead, keep a hash map from each prefix sum to the earliest index at which it occurred. When the current prefix sum minus k has been seen at an earlier index, the subarray between the two positions sums to k , and its length is a candidate for the answer. The overall complexity is linear.

Input / Output format: Line 1: n and k . Line 2: n integers. Output: the maximum length (0 if none).

Example

Input

```
5 3
1 -1 5 -2 3
```

Output

```
4
```

Explanation: The subarray [1, -1, 5, -2] sums to 3, and its length 4 is the longest.

Constraints

- $1 \leq n \leq 10^5$
- $-10^4 \leq a[i] \leq 10^4$
- $-10^9 \leq k \leq 10^9$

Test Cases

Test 1 — Basic

```
Input:
6 15
10 5 2 7 1 9

Output:
4
```

Test 2 — Single element match

```
Input:
1 5
5

Output:
1
```

Test 3 — No such subarray

```
Input:
3 100
1 2 3

Output:
0
```

Test 4 — $k = 0$ with negatives

```
Input:
6 0
1 -1 2 -2 3 -3

Output:
6
```

Test 5 — Zeros extend the subarray

```
Input:
6 3
3 0 0 0 0 0

Output:
6
```

Test 6 — Whole array

```
Input:
4 6
1 2 3 0

Output:
4
```

64. Longest Substring with At Most K Distinct Characters

Topic: Strings

Problem Statement

Given a string s and an integer k , find the length of the longest substring of s that contains at most k different characters. If k is zero, the answer is zero because no character may appear. A sliding window handles the problem in linear time: extend the right end one character at a time while counting each character in a hash map, and whenever the number of distinct characters exceeds k , move the left end forward, decreasing counts and removing characters whose count reaches zero. Track the largest window length seen. Print this maximum length as a single integer. The string may include letters, digits and other symbols.

Input / Output format: Line 1: the string. Line 2: k . Output: the maximum length.

Example

Input

```
eceba
2
```

Output

```
3
```

Explanation: The substring "ece" has two distinct characters and length 3.

Constraints

- $1 \leq |s| \leq 5 \cdot 10^4$
- $0 \leq k \leq 50$
- s has no whitespace

Test Cases

Test 1 — Basic

```
Input:
aabbcc
2

Output:
4
```

Test 2 — $k = 0$

```
Input:
abc
0

Output:
0
```

Test 3 — $k = 1$

```
Input:
aaabbb
1

Output:
3
```

Test 4 — k larger than distinct characters

```
Input:
abc
10

Output:
3
```

Test 5 — Single character string

```
Input:
z
1

Output:
1
```

Test 6 — Distinct characters at every position

```
Input:
abcdef
3

Output:
3
```

65. Next Greater Element – II

Topic: Stack & Queues

Problem Statement

You are given a circular array, which means that the element after the last element is the first element again. For every element, find the next strictly greater element by scanning forward in circular order, possibly wrapping around to the beginning, and print -1 if no greater element exists anywhere in the array. Print the answers for all positions in the original order. A monotonic stack solves the task in linear time: iterate over the indices from 0 to $2n - 1$, using the index modulo n to access the array, and push indices only during the first pass while resolving the pending indices with greater values.

Input / Output format: Line 1: n . Line 2: n integers. Output: n integers.

Example

Input

```
3
1 2 1
```

Output

```
2 -1 2
```

Explanation: The first $1 \rightarrow 2$, the 2 has no greater element, and the last 1 wraps to 2: 2 -1 2.

Constraints

- $1 \leq n \leq 10^4$
- $-10^9 \leq a[i] \leq 10^9$

Test Cases

Test 1 — Basic

```
Input:
5
5 4 3 2 1

Output:
-1 5 5 5 5
```

Test 2 — Single element

```
Input:
1
8

Output:
-1
```

Test 3 — All equal

```
Input:
4
6 6 6 6

Output:
-1 -1 -1 -1
```

Test 4 — Maximum in the middle

```
Input:
5
2 10 3 1 4

Output:
10 -1 4 4 10
```

Test 5 — Increasing then wrap

```
Input:
4
1 2 3 4

Output:
2 3 4 -1
```

Test 6 — Negative values

```
Input:
4
-1 -3 -2 -5

Output:
-1 -2 -1 -1
```

66. Max Consecutive Ones – III

Topic: Arrays

Problem Statement

You are given a binary array and an integer k . You are allowed to flip at most k zeros into ones. Find the maximum number of consecutive ones that can appear in the array after performing these flips. This is equivalent to finding the longest subarray that contains at most k zeros, which suggests a sliding window: expand the right end, count the zeros inside the window, and when the count exceeds k advance the left end until the window becomes valid again. Keep the best window length seen. The algorithm needs linear time and constant space. Print the maximum length as a single integer.

Input / Output format: Line 1: n and k. Line 2: n values (0 or 1). Output: the maximum number of consecutive ones.

Example

Input

```
11 2
1 1 1 0 0 0 1 1 1 1 0
```

Output

```
6
```

Explanation: Flip the zeros at indices 5 and 10 to get six consecutive ones: 6.

Constraints

- $1 \leq n \leq 10^5$
- $a[i] \in \{0, 1\}$
- $0 \leq k \leq n$

Test Cases

Test 1 — Basic

```
Input:
10 3
0 0 1 1 0 0 0 1 1 1
```

```
Output:  
8
```

Test 2 — Single zero, k = 1

```
Input:  
1 1  
0  
  
Output:  
1
```

Test 3 — Single zero, k = 0

```
Input:  
1 0  
0  
  
Output:  
0
```

Test 4 — All ones

```
Input:  
5 0  
1 1 1 1 1  
  
Output:  
5
```

Test 5 — All zeros

```
Input:  
4 2  
0 0 0 0
```

```
Output:  
2
```

Test 6 — k covers all zeros

```
Input:  
6 3  
0 1 0 1 0 1  
  
Output:  
6
```

67. Majority Element – II

Topic: Hashing

Problem Statement

Given an integer array of size n , find all elements that appear strictly more than $n/3$ times, where the division is an integer division rounded down. There can be at most two such elements. Print them in increasing order on one line, or print `[]` if none exist. A hash map of counts solves the task in linear time and linear space, but the extended Boyer–Moore voting algorithm reduces the space to $O(1)$: keep two candidates with two counters, decrease both counters when a third distinct value appears, and finally verify the two candidates by counting their actual occurrences in a second pass through the array.

Input / Output format: Line 1: n . Line 2: n integers. Output: the qualifying elements in increasing order (`[]` if none).

Example

Input

```
6
1 1 1 3 3 2
```

Output

```
1
```

Explanation: $n/3 = 2$. Value 1 appears 3 times, value 3 appears 2 times (not more than 2).
Answer: 1.

Constraints

- $1 \leq n \leq 5 \cdot 10^4$
- $-10^9 \leq a[i] \leq 10^9$

Test Cases

Test 1 — Two majority elements

```
Input:
9
5 5 5 5 7 7 7 7 1

Output:
5 7
```

Test 2 — Single element

```
Input:
1
4

Output:
4
```

Test 3 — No majority element

```
Input:
6
1 2 3 4 5 6

Output:
[]
```

Test 4 — All equal

```
Input:
5
9 9 9 9 9

Output:
9
```

Test 5 — Negative values

```
Input:
7
-1 -1 -1 2 2 2 3

Output:
-1 2
```

Test 6 — Exactly $n/3$ is not enough

```
Input:
6
1 1 2 2 3 4

Output:
[]
```

68. Kth Element of Two Sorted Arrays

Topic: Binary Search

Problem Statement

Given two sorted arrays of sizes n and m and an integer k , find the element that would be at position k (1-indexed) if the two arrays were merged into a single sorted array. Merging costs $O(k)$ time, but you should aim for $O(\log(\min(n, m)))$ using binary search on the number of elements taken from the shorter array. Choose a split so that i elements come from the first array and $k - i$ from the second, check that the largest element on the left side of each array is not greater than the smallest element on the right side of the other array, and adjust the split otherwise. Print the k -th element.

Input / Output format: Line 1: n . Line 2: array a . Line 3: m . Line 4: array b . Line 5: k .
Output: the k -th element of the merged array.

Example

Input

```
5
2 3 6 7 9
4
1 4 8 10
5
```

Output

```
6
```

Explanation: Merged: 1 2 3 4 6 7 8 9 10. The 5th element is 6.

Constraints

- $1 \leq n, m \leq 10^5$
- $1 \leq k \leq n + m$
- $0 \leq \text{elements} \leq 10^9$
- Both arrays are sorted

Test Cases

Test 1 — Basic

```
Input:
4
2 3 6 7
3
1 4 8
4

Output:
4
```

Test 2 — $k = 1$

```
Input:
3
5 6 7
```

```
2
1 9
1
```

```
Output:
1
```

Test 3 — $k = n + m$

```
Input:
3
5 6 7
2
1 9
5
```

```
Output:
9
```

Test 4 — Single element arrays

```
Input:
1
4
1
2
2
```

```
Output:
4
```

Test 5 — Duplicates across arrays

```
Input:
3
1 2 2
3
2 2 3
4
```

```
Output:
```

Test 6 — All of a is smaller

```
Input:
3
1 2 3
3
10 11 12
4

Output:
10
```

69. Best Time to Buy and Sell Stock with Transaction Fee

Topic: Dynamic Programming

Problem Statement

You are given the price of a stock on each of n days and a non-negative integer fee. You may complete as many transactions as you like, where a transaction is a buy followed by a sell, but you must pay the fee once for every transaction, and you cannot hold more than one share at any time. Find the maximum profit you can earn. A two-state dynamic programming solution works in linear time: maintain the best profit while holding a share and the best profit while not holding one, and update them for each day, subtracting the fee when a share is sold. If no profitable transaction exists, the answer is 0.

Input / Output format: Line 1: n and fee. Line 2: n prices. Output: the maximum profit.

Example

Input

```
6 2
1 3 2 8 4 9
```

Output

```
8
```

Explanation: Buy at 1, sell at 8 (profit 5 after fee), buy at 4, sell at 9 (profit 3 after fee): total 8.

Constraints

- $1 \leq n \leq 5 \cdot 10^4$
- $1 \leq \text{price}[i] < 5 \cdot 10^4$
- $0 \leq \text{fee} < 5 \cdot 10^4$

Test Cases

Test 1 — Basic

```
Input:
5 3
1 4 6 2 8

Output:
5
```

Test 2 — Single day

```
Input:
1 2
5

Output:
0
```

Test 3 — Fee larger than any gain

```
Input:
4 10
1 3 7 5

Output:
0
```

Test 4 — Zero fee, take every rise

```
Input:
5 0
1 3 2 4 5

Output:
5
```

Test 5 — Prices only fall

```
Input:
4 1
9 8 7 6

Output:
0
```

Test 6 — Constant prices


```
Input:
4 1
5 5 5 5

Output:
0
```

70. Find Peak Element – II

Topic: Binary Search

Problem Statement

A peak element in a two dimensional grid is a cell whose value is strictly greater than each of its neighbours in the four directions up, down, left and right. Cells outside the grid are treated as having value negative infinity. Given an m by n grid in which no two adjacent cells have equal values, find the position of any peak cell and print its row and column indices, zero-based. To do it in $O(m \log n)$ time, pick the middle column, find the maximum element in that column, compare it with its left and right neighbours, and continue the binary search on the side with the larger neighbour. Any valid peak is accepted; the expected output shows the first peak in row-major order.

Input / Output format: Line 1: m and n . Next m lines: the grid. Output: 'row col' of a peak (the first one in row-major order is shown).

Example

Input

```
2 2
1 4
3 2
```

Output

```
0 1
```

Explanation: The value 4 at row 0, column 1 beats 1 (left) and 2 (below): print 0 1.

Constraints

- $1 \leq m, n \leq 500$
- $1 \leq \text{grid}[i][j] \leq 10^5$
- Adjacent cells never have equal values

Test Cases

Test 1 — Basic

```
Input:
3 3
10 20 15
21 30 14
7 16 32
```

```
Output:
1 1
```

Test 2 — Single cell

```
Input:
1 1
5
```

```
Output:
```

```
0 0
```

Test 3 — Single row

```
Input:
1 5
1 3 2 5 4

Output:
0 1
```

Test 4 — Single column

```
Input:
4 1
1
2
9
3

Output:
2 0
```

Test 5 — Peak in the corner

```
Input:
2 3
9 5 4
6 3 2

Output:
0 0
```

Test 6 — Maximum in the last row

```
Input:
```

```
3 2
```

```
1 2
```

```
3 4
```

```
5 6
```

```
Output:
```

```
2 1
```

71. Longest Word with All Prefixes

Topic: Tries

Problem Statement

You are given a list of words. Find the longest word in the list such that every one of its non-empty prefixes, including the word itself, is also present in the list. For example, `ninja` qualifies only if `n`, `ni`, `nin`, `ninj` and `ninja` all appear as separate words. If several qualifying words have the same maximum length, print the lexicographically smallest one. If no word qualifies, print `None`. A trie makes this efficient: insert all words, mark word endings, and then explore the trie by moving only through nodes that are marked as the end of a word, tracking the deepest valid word encountered.

Input / Output format: Line 1: `n`. Line 2: `n` words separated by spaces. Output: the answer word, or `None`.

Example

Input

```
6
n ni nin ninj ninja ninga
```

Output

```
ninja
```

Explanation: Every prefix of ninja is present, while ninga lacks ning. The answer is ninja.

Constraints

- $1 \leq n \leq 10^5$
- Total length of all words $\leq 10^6$
- Words contain lowercase English letters only

Test Cases

Test 1 — Basic

```
Input:
5
g gr gre gree green

Output:
green
```

Test 2 — Single word

```
Input:
1
a

Output:
a
```

Test 3 — No word has all prefixes

```
Input:
3
ab abc abcd

Output:
None
```

Test 4 — Tie broken lexicographically

```
Input:
4
a b ab ba

Output:
ab
```

Test 5 — Only single-letter words

```
Input:
3
c a b

Output:
a
```

Test 6 — Duplicate words in the list

```
Input:
6
k ki ki kin kin king

Output:
king
```

72. Shortest Palindrome

Topic: Strings

Problem Statement

You are given a string s . You may add characters only in front of it, and you want to turn it into a palindrome by adding as few characters as possible. Print the shortest palindrome that can be produced this way. The key observation is that the answer is determined by the longest prefix of s that is already a palindrome; the remaining suffix, reversed, must be placed in front of the string. To find that longest palindromic prefix in linear time, build the string $s + \# + \text{reverse}(s)$ and compute its LPS array as in the KMP algorithm, then use the last value. Rolling hashes can be used as an alternative.

Input / Output format: One line: the string s . Output: the shortest palindrome obtained by adding characters in front of s .

Example

Input

```
aacecaaa
```

Output

```
aaacecaaa
```

Explanation: The longest palindromic prefix is aacecaa; adding one 'a' in front gives aaacecaaa.

Constraints

- $1 \leq |s| \leq 5 \cdot 10^4$
- s contains lowercase English letters

Test Cases

Test 1 — Basic

```
Input:  
abcd
```

```
Output:  
dcbabcd
```

Test 2 — Single character

```
Input:  
a
```

```
Output:  
a
```

Test 3 — Already a palindrome

```
Input:  
racecar
```

```
Output:  
racecar
```


Test 4 — All characters identical

```
Input:  
aaaa  
  
Output:  
aaaa
```

Test 5 — Two characters

```
Input:  
ab  
  
Output:  
bab
```

Test 6 — Palindromic prefix of length 3

```
Input:  
abacxyz  
  
Output:  
zyxcabacxyz
```

73. 3Sum Smaller

Topic: Arrays

Problem Statement

Given an array of n integers and a target value, count the number of index triplets (i, j, k) with $0 \leq i < j < k < n$ such that $\text{nums}[i] + \text{nums}[j] + \text{nums}[k]$ is strictly less than the target. Triplets are distinguished by their positions, so equal values at different positions count separately. A brute force solution with three nested loops takes cubic time. Sort the array first, fix the smallest index and use two pointers on the remaining part: whenever the sum with the current pair is smaller than the target, every element between the two pointers also works with the left pointer, so add that whole range at once. This gives quadratic time.

Input / Output format: Line 1: n and target. Line 2: n integers. Output: the number of valid triplets.

Example

Input

```
4 2
-2 0 1 3
```

Output

```
2
```

Explanation: Triplets $(-2, 0, 1)$ and $(-2, 0, 3)$ have sums -1 and 1 , both below 2 . Answer: 2 .

Constraints

- $1 \leq n \leq 3500$
- $-100 \leq \text{nums}[i] \leq 100$
- $-100 \leq \text{target} \leq 100$

Test Cases

Test 1 — Basic

```
Input:
5 4
3 1 0 -2 4

Output:
5
```

Test 2 — Fewer than three elements

```
Input:
2 5
1 2

Output:
0
```

Test 3 — No triplet qualifies

```
Input:
4 -10
1 2 3 4

Output:
0
```

Test 4 — All triplets qualify

```
Input:
4 100
1 2 3 4

Output:
```

```
4
```

Test 5 — Duplicates count by position

```
Input:
5 0
-1 -1 -1 0 2

Output:
4
```

Test 6 — Target exactly equals a sum

```
Input:
3 6
1 2 3

Output:
0
```

74. Alt and Tab Simulation

Topic: Hashing

⚠ **Note:** Statement reconstructed: the original problem text for this title could not be found publicly; verify against your senior's copy.

Problem Statement

Simulate the Alt+Tab window switcher of an operating system. There are n open windows numbered 1 to n , and they are kept in a list ordered from the most recently used window to

the least recently used one. Initially the list is 1, 2, ..., n. You are given q switch commands, each containing a position p; the command selects the window currently at position p in the list (1-indexed) and brings it to the front, keeping the relative order of the other windows unchanged. Print the final order of the windows from the most recent to the least recent. A linked list with a hash map from window to node performs each move in constant time.

Input / Output format: Line 1: n and q. Line 2: q positions. Output: the final order of window numbers, space separated.

Example

Input

```
5 3
3 2 5
```

Output

```
5 1 3 2 4
```

Explanation: [1 2 3 4 5] → [3 1 2 4 5] → [1 3 2 4 5] → [5 1 3 2 4].

Constraints

- $1 \leq n \leq 10^5$
- $1 \leq q \leq 10^5$
- $1 \leq p \leq n$ for every command

Test Cases

Test 1 — Basic

```
Input:
4 2
2 4

Output:
4 2 1 3
```

Test 2 — Single window

```
Input:
1 3
1 1 1

Output:
1
```

Test 3 — Position 1 changes nothing

```
Input:
4 2
1 1

Output:
1 2 3 4
```

Test 4 — Always the last window

```
Input:
4 3
4 4 4

Output:
2 3 4 1
```

Test 5 — Alternating positions

```
Input:
3 4
2 2 2 2

Output:
1 2 3
```

Test 6 — All windows brought forward once

```
Input:
4 4
4 4 4 4

Output:
1 2 3 4
```

75. Armstrong Number

Topic: Mathematics

Problem Statement

A positive integer is an Armstrong number, also known as a narcissistic number, if it equals the sum of its own digits each raised to the power of the number of digits. For example, 153 has three digits and $1^3 + 5^3 + 3^3 = 153$, so it is an Armstrong number, while 154 is not. Given an integer n , determine whether it is an Armstrong number. Extract the digits by repeatedly taking the remainder and integer division by ten, count how many digits there are, and then accumulate the powered digits. Print true if n is an Armstrong number and false otherwise. Every single digit number is an Armstrong number.

Input / Output format: One line: the integer n . Output: true or false.

Example

Input

```
153
```

Output

```
true
```

Explanation: $1^3 + 5^3 + 3^3 = 1 + 125 + 27 = 153$, so the answer is true.

Constraints

- $0 \leq n \leq 10^9$

Test Cases

Test 1 — Three-digit Armstrong

```
Input:  
371
```

```
Output:  
true
```

Test 2 — Zero


```
Input:  
0
```

```
Output:  
true
```

Test 3 — Single digit

```
Input:  
7
```

```
Output:  
true
```

Test 4 — Not Armstrong

```
Input:  
123
```

```
Output:  
false
```

Test 5 — Four-digit Armstrong

```
Input:  
9474
```

```
Output:  
true
```

Test 6 — Large non-Armstrong

```
Input:  
1000000000
```

```
Output:  
false
```

76. Array Upper Bound

Topic: Binary Search

⚠ **Note:** Statement reconstructed: this title is not a standard named problem; interpreted as the value returned by an upper-bound search. Verify with your senior.

Problem Statement

Given a sorted array of integers and a value x , find the upper bound of x in the array, defined as the smallest element that is strictly greater than x . If every element of the array is less than or equal to x , print -1 . Do not scan the array linearly; use binary search to locate the boundary in $O(\log n)$ time. Maintain a search range, and at each step compare the middle element with x : if it is greater than x , it is a candidate and the answer might lie further to the left, otherwise the search continues in the right half. Duplicate values may be present, and x itself does not have to occur in the array.

Input / Output format: Line 1: n . Line 2: n sorted integers. Line 3: x . Output: the smallest element greater than x , or -1 .

Example

Input

```
5  
1 2 4 4 7  
4
```

Output

```
7
```

Explanation: The smallest element greater than 4 is 7.

Constraints

- $1 \leq n \leq 10^5$
- $-10^9 \leq a[i], x \leq 10^9$
- The array is sorted in non-decreasing order

Test Cases

Test 1 — Basic

```
Input:
6
2 3 5 7 11 13
6

Output:
7
```

Test 2 — Single element greater than x

```
Input:
1
9
3

Output:
```

```
9
```

Test 3 — Single element not greater

```
Input:
1
3
3

Output:
-1
```

Test 4 — x smaller than all

```
Input:
4
5 6 7 8
-1

Output:
5
```

Test 5 — x larger than all

```
Input:
4
5 6 7 8
8

Output:
-1
```

Test 6 — Many duplicates of x

```
Input:
```

```
6
2 4 4 4 4 9
4

Output:
9
```

77. Number of Jumps

Topic: Mathematics

⚠ **Note:** Statement reconstructed: original text unavailable; interpreted as the minimum-jumps-of-size-up-to-k problem. Verify with your senior.

Problem Statement

A frog wants to travel along a straight line from position 0 to position d . In a single jump it can cover any distance from 1 up to k units forward, but it cannot go backwards. Find the minimum number of jumps the frog needs to reach exactly position d . Because every jump can be of the maximum length k except possibly the last one, the answer is the ceiling of d divided by k , which can be computed with integer arithmetic as $(d + k - 1) / k$ without using floating point operations. If d is zero, the frog is already at the destination and needs zero jumps. Print the minimum number of jumps as a single integer.

Input / Output format: Line 1: d and k . Output: the minimum number of jumps.

Example

Input

```
10 3
```

Output

```
4
```

Explanation: Jumps of 3, 3, 3 and 1 cover 10 units, so 4 jumps are needed.

Constraints

- $0 \leq d \leq 10^{18}$
- $1 \leq k \leq 10^{18}$

Test Cases

Test 1 — Basic

```
Input:
17 5

Output:
4
```

Test 2 — Zero distance

```
Input:
0 4

Output:
0
```

Test 3 — $k = 1$

```
Input:
9 1

Output:
9
```

Test 4 — d divisible by k

```
Input:
12 4

Output:
3
```

Test 5 — d smaller than k

```
Input:
3 10

Output:
1
```

Test 6 — Very large values

```
Input:
1000000000000000000 3

Output:
333333333333333334
```

78. Set Difference of Two Arrays

Topic: Hashing

Problem Statement

Given two arrays a and b , compute the set difference a minus b : the collection of distinct values that occur in a but do not occur in b . Each value must be printed only once, even when it appears several times in a , and the values must be printed in increasing order. If every value of a also appears in b , print []. A hash set of the elements of b lets you test membership in constant time, so the entire task can be solved in linear time plus the cost of sorting the final answer. The arrays are not sorted, may contain duplicates and may have different lengths, including zero elements.

Input / Output format: Line 1: n . Line 2: array a . Line 3: m . Line 4: array b . Output: the distinct values of a not in b , ascending ([] if none).

Example

Input

```
5
1 2 3 4 4
3
2 4 6
```

Output

```
1 3
```

Explanation: Values 1 and 3 are in a but not in b . Answer: 1 3.

Constraints

- $1 \leq n \leq 10^5$
- $0 \leq m \leq 10^5$
- $-10^9 \leq \text{elements} \leq 10^9$

Test Cases

Test 1 — Basic

```
Input:
6
9 5 1 7 5 3
3
5 7 8

Output:
1 3 9
```

Test 2 — Empty b

```
Input:
3
3 1 2
0

Output:
1 2 3
```

Test 3 — Everything removed

```
Input:
3
1 2 3
4
3 2 1 0
```

```
Output:  
[]
```

Test 4 — No common values

```
Input:  
3  
1 2 3  
3  
4 5 6  
  
Output:  
1 2 3
```

Test 5 — Duplicates in a collapse

```
Input:  
5  
2 2 2 3 3  
1  
9  
  
Output:  
2 3
```

Test 6 — Negative values

```
Input:  
4  
-1 -2 0 5  
2  
-2 5  
  
Output:  
-1 0
```

79. Rearrange Array Elements by Sign

Topic: Arrays

Problem Statement

You are given an array of even length containing an equal number of positive and negative integers, and no zeros. Rearrange the elements so that positive and negative numbers alternate, starting with a positive number. The relative order of the positive numbers among themselves must stay the same as in the original array, and the same is required for the negative numbers. Print the rearranged array. A straightforward method uses two separate lists, and a more space efficient variant fills a result array using two write positions, one at even indices for positive values and one at odd indices for negative values, in a single pass over the input.

Input / Output format: Line 1: n (even). Line 2: n integers. Output: the rearranged array.

Example

Input

```
6
3 1 -2 -5 2 -4
```

Output

```
3 -2 1 -5 2 -4
```

Explanation: Positives in order: 3 1 2. Negatives in order: -2 -5 -4. Interleaved: 3 -2 1 -5 2 -4.

Constraints

- $2 \leq n \leq 2 \cdot 10^5$, n even
- $-10^4 \leq a[i] \leq 10^4$, $a[i] \neq 0$
- Exactly $n/2$ positives and $n/2$ negatives

Test Cases

Test 1 — Basic

```
Input:
4
1 2 -3 -4

Output:
1 -3 2 -4
```

Test 2 — Minimum size

```
Input:
2
-5 5

Output:
5 -5
```

Test 3 — Already alternating

```
Input:
4
1 -1 2 -2

Output:
```

```
1 -1 2 -2
```

Test 4 — All negatives first

```
Input:
6
-1 -2 -3 4 5 6

Output:
4 -1 5 -2 6 -3
```

Test 5 — Large magnitudes

```
Input:
4
10000 -10000 9999 -9999

Output:
10000 -10000 9999 -9999
```

Test 6 — Duplicates

```
Input:
6
2 2 -1 -1 2 -1

Output:
2 -1 2 -1 2 -1
```

80. Intersection of Two Sorted Arrays

Topic: Arrays

Problem Statement

Given two arrays sorted in non-decreasing order, print their intersection, meaning the elements that are present in both arrays. If a value appears x times in the first array and y times in the second, it should appear $\min(x, y)$ times in the answer. The result must also be in non-decreasing order. Use two pointers, one for each array: if the two current elements are equal, add the value to the answer and advance both pointers; otherwise advance the pointer at the smaller element. This solves the problem in $O(n + m)$ time without extra data structures. If there are no common elements, print `[]`.

Input / Output format: Line 1: n . Line 2: array a . Line 3: m . Line 4: array b . Output: the intersection in non-decreasing order (`[]` if empty).

Example

Input

```
5
1 2 2 3 5
4
2 2 2 5
```

Output

```
2 2 5
```

Explanation: Value 2 appears twice in the first array and three times in the second, so min is 2; 5 appears once. Answer: 2 2 5.

Constraints

- $1 \leq n, m \leq 10^5$
- $-10^9 \leq \text{elements} \leq 10^9$
- Both arrays are sorted

Test Cases

Test 1 — Basic

```
Input:
5
1 3 4 5 7
4
2 3 5 8

Output:
3 5
```

Test 2 — Single elements match

```
Input:
1
4
1
4

Output:
4
```

Test 3 — No common elements

```
Input:
3
1 2 3
3
4 5 6

Output:
```

```
[]
```

Test 4 — Identical arrays

```
Input:
4
1 2 3 4
4
1 2 3 4

Output:
1 2 3 4
```

Test 5 — Heavy duplicates

```
Input:
6
1 1 1 2 2 3
4
1 1 2 2

Output:
1 1 2 2
```

Test 6 — Negative values

```
Input:
4
-3 -1 0 2
3
-3 0 5

Output:
-3 0
```

81. Move Zeros to End

Topic: Arrays

Problem Statement

Given an integer array, move all zeros to the end of the array while keeping the relative order of the non-zero elements unchanged. The operation must be done in place, which means you should not allocate another array of the same size. A simple two pointer technique achieves this in one pass: keep a write index for the next position of a non-zero element, copy every non-zero value there while scanning, and finally fill the remaining positions with zeros. Print the modified array on a single line with values separated by spaces. An array with no zeros, or with only zeros, is left unchanged.

Input / Output format: Line 1: n. Line 2: n integers. Output: the array after moving zeros to the end.

Example

Input

```
5
0 1 0 3 12
```

Output

```
1 3 12 0 0
```

Explanation: Non-zeros keep their order and zeros go last: 1 3 12 0 0.

Constraints

- $1 \leq n \leq 10^5$
- $-10^9 \leq a[i] \leq 10^9$

Test Cases

Test 1 — Basic

```
Input:
6
4 0 5 0 0 6

Output:
4 5 6 0 0 0
```

Test 2 — Single zero

```
Input:
1
0

Output:
0
```

Test 3 — Single non-zero

```
Input:
1
8

Output:
8
```

Test 4 — No zeros

```
Input:
4
1 2 3 4

Output:
1 2 3 4
```

Test 5 — All zeros

```
Input:
4
0 0 0 0

Output:
0 0 0 0
```

Test 6 — Zeros already at the end

```
Input:
5
7 8 0 0 0

Output:
7 8 0 0 0
```

82. Remove Duplicates from Sorted Array

Topic: Arrays

Problem Statement

Given an array sorted in non-decreasing order, remove the duplicates in place so that each distinct value appears exactly once, keeping the original relative order of the values. Print the number k of distinct elements on the first line, followed on the second line by the first k elements of the modified array. The elements beyond position k do not matter. Use the two pointer method: one pointer marks the end of the unique prefix, and the other scans the array, and whenever it meets a value different from the last unique value, it is written to the next position of the unique prefix. No extra array is needed and the running time is linear.

Input / Output format: Line 1: n . Line 2: n sorted integers. Output: line 1 = k ; line 2 = the k unique values.

Example

Input

```
7
0 0 1 1 1 2 3
```

Output

```
4
0 1 2 3
```

Explanation: There are 4 distinct values: 0 1 2 3.

Constraints

- $1 \leq n \leq 10^5$
- $-10^9 \leq a[i] \leq 10^9$
- The array is sorted in non-decreasing order

Test Cases

Test 1 — Basic

```
Input:
6
1 1 2 2 3 3

Output:
3
1 2 3
```

Test 2 — Single element

```
Input:
1
5

Output:
1
5
```

Test 3 — All equal

```
Input:
5
4 4 4 4 4

Output:
1
4
```

Test 4 — Already unique

```
Input:
4
1 2 3 4
```

```
Output:  
4  
1 2 3 4
```

Test 5 — Negative values

```
Input:  
5  
-3 -3 -1 0 0  
  
Output:  
3  
-3 -1 0
```

Test 6 — Duplicates only at the end

```
Input:  
5  
1 2 3 4 4  
  
Output:  
4  
1 2 3 4
```

83. Union of Two Sorted Arrays

Topic: Arrays

Problem Statement

Given two sorted arrays, print their union: all distinct values that occur in at least one of the two arrays, listed in increasing order. A value that appears in both arrays or many times inside one array must be printed only once. Since both arrays are already sorted, you can

produce the union in linear time by merging them with two pointers, always taking the smaller current element and skipping it if it equals the last value that was added to the answer. This avoids sets and extra sorting. The arrays may have different lengths, and either of them may contain repeated values. Print the union as space separated integers on one line.

Input / Output format: Line 1: n. Line 2: array a. Line 3: m. Line 4: array b. Output: the sorted union without duplicates.

Example

Input

```
5
1 2 2 4 6
4
2 3 6 7
```

Output

```
1 2 3 4 6 7
```

Explanation: Distinct values from both arrays: 1 2 3 4 6 7.

Constraints

- $1 \leq n, m \leq 10^5$
- $-10^9 \leq \text{elements} \leq 10^9$
- Both arrays are sorted

Test Cases

Test 1 — Basic

```
Input:
4
1 3 5 7
4
2 4 6 8

Output:
1 2 3 4 5 6 7 8
```

Test 2 — Single elements, equal

```
Input:
1
5
1
5

Output:
5
```

Test 3 — Identical arrays

```
Input:
3
1 2 3
3
1 2 3

Output:
1 2 3
```

Test 4 — Disjoint ranges

```
Input:
3
1 2 3
3
```



```
7 8 9
```

```
Output:
```

```
1 2 3 7 8 9
```

Test 5 — Duplicates inside arrays

```
Input:
```

```
5
```

```
1 1 1 2 2
```

```
4
```

```
2 2 3 3
```

```
Output:
```

```
1 2 3
```

Test 6 — Negative values

```
Input:
```

```
3
```

```
-5 -2 0
```

```
3
```

```
-3 -2 4
```

```
Output:
```

```
-5 -3 -2 0 4
```

84. Automorphic Number

Topic: Mathematics

Problem Statement

A number is called automorphic if its square ends with the same digits as the number itself. For example, 25 is automorphic because 25 squared is 625, which ends with 25, and 76 is automorphic because 76 squared is 5776. Given a non-negative integer n , check whether it is automorphic. Compare the last d digits of n squared with n , where d is the number of digits of n , which you can do with the modulo operator and a power of ten. Use 64 bit integers to avoid overflow for large inputs. Print true if the number is automorphic and false otherwise. Both 0 and 1 are automorphic.

Input / Output format: One line: the integer n . Output: true or false.

Example

Input

```
76
```

Output

```
true
```

Explanation: $76 \times 76 = 5776$ and the last two digits are 76: true.

Constraints

- $0 \leq n \leq 10^9$

Test Cases

Test 1 — Two-digit automorphic

```
Input:
25
```

```
Output:
true
```

Test 2 — Zero

```
Input:
0
```

```
Output:
true
```

Test 3 — One

```
Input:
1
```

```
Output:
true
```

Test 4 — Not automorphic

```
Input:
12
```

```
Output:
false
```

Test 5 — Large automorphic

```
Input:
890625
```

```
Output:  
true
```

Test 6 — Single digit automorphic

```
Input:  
6  
  
Output:  
true
```

85. Find Row with Maximum 1's

Topic: Binary Search

Problem Statement

You are given a binary matrix in which every row is sorted, so each row consists of some zeros followed by some ones. Find the index of the row that contains the maximum number of ones. If several rows tie for the maximum, print the smallest row index. If the matrix contains no ones at all, print -1. Counting the ones in every row takes $O(r \cdot c)$ time. Since each row is sorted, binary search can find the position of the first one in each row in $O(\log c)$, giving $O(r \log c)$ overall, and an even better staircase walk from the top right corner uses $O(r + c)$ time. Row indices are zero-based.

Input / Output format: Line 1: r and c . Next r lines: the rows (0/1, each sorted). Output: the row index or -1.

Example

Input

```
3 4
0 1 1 1
0 0 1 1
1 1 1 1
```

Output

```
2
```

Explanation: Row 2 has four ones, more than any other row, so the answer is 2.

Constraints

- $1 \leq r, c \leq 1000$
- $\text{matrix}[i][j] \in \{0, 1\}$
- Each row is sorted in non-decreasing order

Test Cases

Test 1 — Basic

```
Input:
4 4
0 0 0 1
0 1 1 1
0 0 1 1
0 0 0 0

Output:
1
```

Test 2 — Single cell equals one

```
Input:
1 1
1

Output:
0
```

Test 3 — No ones at all

```
Input:
2 3
0 0 0
0 0 0

Output:
-1
```

Test 4 — Tie, smallest index wins

```
Input:
3 3
0 1 1
0 1 1
0 0 1

Output:
0
```

Test 5 — All ones

```
Input:
2 2
1 1
1 1

Output:
```

0

Test 6 — Single column

```
Input:
3 1
0
1
1

Output:
1
```

86. Search in a 2D Matrix

Topic: Binary Search

Problem Statement

You are given an m by n matrix of integers with two properties: every row is sorted in non-decreasing order, and the first element of each row is greater than the last element of the previous row. Determine whether a target value exists in the matrix and print true or false. These properties mean that if you imagine the rows written one after another, the matrix behaves like a single sorted array of $m \cdot n$ elements, so you can run an ordinary binary search on the virtual index range and convert a middle index k into the row k / n and the column $k \% n$. The running time is $O(\log(m \cdot n))$.

Input / Output format: Line 1: m and n . Next m lines: the rows. Last line: target. Output: true or false.

Example

Input

```
3 4
1 3 5 7
10 11 16 20
23 30 34 60
3
```

Output

```
true
```

Explanation: The value 3 is present in the first row: true.

Constraints

- $1 \leq m, n \leq 100$
- $-10^4 \leq \text{matrix}[i][j], \text{target} \leq 10^4$

Test Cases

Test 1 — Target absent

```
Input:
3 4
1 3 5 7
10 11 16 20
23 30 34 60
13

Output:
false
```


Test 2 — Single cell match

```
Input:
1 1
4
4

Output:
true
```

Test 3 — Single cell no match

```
Input:
1 1
4
5

Output:
false
```

Test 4 — Target is the last element

```
Input:
2 2
1 2
3 4
4

Output:
true
```

Test 5 — Target smaller than all

```
Input:
2 2
5 6
7 8
-3
```

```
Output:  
false
```

Test 6 — Target between two rows

```
Input:  
2 3  
1 2 3  
10 20 30  
5  
  
Output:  
false
```

87. Lower Bound

Topic: Binary Search

Problem Statement

Given a sorted array of n integers and a value x , find the lower bound of x , which is the smallest index i such that $a[i]$ is greater than or equal to x . If every element in the array is smaller than x , the lower bound is n , the position just after the last element. Indices are zero-based. The array may contain duplicates, in which case the answer is the index of the first occurrence of x , if x is present. Use binary search: keep a range $[low, high]$, and if the middle element is at least x , remember it as a candidate and search the left half, otherwise search the right half. The time complexity is $O(\log n)$.

Input / Output format: Line 1: n . Line 2: n sorted integers. Line 3: x . Output: the lower bound index.

Example

Input

```
5
1 2 2 3 5
2
```

Output

```
1
```

Explanation: The first element that is at least 2 is at index 1.

Constraints

- $1 \leq n \leq 10^5$
- $-10^9 \leq a[i], x \leq 10^9$
- The array is sorted in non-decreasing order

Test Cases

Test 1 — x not present

```
Input:
5
1 3 5 7 9
4

Output:
2
```

Test 2 — Single element, equal

```
Input:
1
5
5

Output:
0
```

Test 3 — Single element, smaller

```
Input:
1
5
9

Output:
1
```

Test 4 — x smaller than all elements

```
Input:
4
3 4 5 6
0

Output:
0
```

Test 5 — x larger than all elements

```
Input:
4
3 4 5 6
10

Output:
```

Test 6 — Duplicates of x

```
Input:
7
1 4 4 4 4 4 9
4

Output:
1
```

88. Search X in a Sorted Array

Topic: Binary Search

Problem Statement

You are given an array of distinct integers sorted in ascending order and a target value x . Print the zero-based index of x in the array, or -1 if x does not occur. A linear scan works but takes $O(n)$ time, so use binary search instead: repeatedly compare x with the middle element of the current range, return its index when they are equal, and discard the left or the right half depending on the comparison. Take care to compute the middle index as $\text{low} + (\text{high} - \text{low}) / 2$ to avoid integer overflow in languages with fixed size integers. Report -1 when the range becomes empty without finding the value.

Input / Output format: Line 1: n . Line 2: n sorted distinct integers. Line 3: x . Output: the index of x or -1.

Example

Input

```
6
-1 0 3 5 9 12
9
```

Output

```
4
```

Explanation: The value 9 is stored at index 4.

Constraints

- $1 \leq n \leq 10^5$
- $-10^9 \leq a[i], x \leq 10^9$
- All $a[i]$ are distinct

Test Cases

Test 1 — Target absent

```
Input:
6
-1 0 3 5 9 12
2
```

```
Output:
-1
```

Test 2 — Single element found

```
Input:
1
8
8

Output:
0
```

Test 3 — Single element missing

```
Input:
1
8
3

Output:
-1
```

Test 4 — Target is the first element

```
Input:
5
2 4 6 8 10
2

Output:
0
```

Test 5 — Target is the last element

```
Input:
5
2 4 6 8 10
10

Output:
4
```

Test 6 — Target outside the range

```
Input:
3
5 6 7
100

Output:
-1
```

89. Upper Bound

Topic: Binary Search

Problem Statement

Given a sorted array of n integers and a value x , find the upper bound of x , defined as the smallest index i such that $a[i]$ is strictly greater than x . If no element is greater than x , the upper bound is n . Indices are zero-based, and the array can contain duplicate values, so if x occurs several times the upper bound points just past its last occurrence. Use binary search: if the middle element is greater than x , it becomes the current answer and the search continues in the left half; otherwise the search continues in the right half. The running time is $O(\log n)$ and the extra space is constant. Print a single integer.

Input / Output format: Line 1: n . Line 2: n sorted integers. Line 3: x . Output: the upper bound index.

Example

Input

```
5
```



```
1 2 2 3 5
2
```

Output

```
3
```

Explanation: The first element greater than 2 is 3, at index 3.

Constraints

- $1 \leq n \leq 10^5$
- $-10^9 \leq a[i], x \leq 10^9$
- The array is sorted in non-decreasing order

Test Cases

Test 1 — x not present

```
Input:
5
1 3 5 7 9
4

Output:
2
```

Test 2 — Single element equal to x

```
Input:
1
5
```

```
5

Output:
1
```

Test 3 — Single element smaller than x

```
Input:
1
2
5

Output:
1
```

Test 4 — x smaller than all elements

```
Input:
4
3 4 5 6
0

Output:
0
```

Test 5 — x larger than or equal to all

```
Input:
4
3 4 5 6
6

Output:
4
```

Test 6 — Duplicates of x

```
Input:
7
1 4 4 4 4 4 9
4

Output:
6
```

90. Calculate Frequency of Characters in a String

Topic: Hashing

Year / Source: Feb 2024 (Code360 on-campus experience: 'Count Frequency'); also on the past-year list

Problem Statement

You are given a string made of lowercase English letters. For every letter from 'a' to 'z', count how many times it occurs in the string, and print the 26 counts in alphabetical order separated by single spaces. Letters that do not appear must be reported with a count of zero, so the output always contains exactly 26 numbers. A fixed size array of 26 integers is the ideal data structure: for each character, subtract the code of 'a' to get an index and increment the counter at that index. The algorithm needs one pass through the string and constant extra space, and it is a building block for anagram and pattern problems.

Input / Output format: One line: the string. Output: 26 integers, the frequencies of 'a' to 'z'.

Example

Input

```
abcdg
```

Output

```
1 1 1 1 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
```

Explanation: a, b, c, d and g each occur once: 1 1 1 1 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0.

Constraints

- $1 \leq |s| \leq 10^5$
- s contains only lowercase English letters

Test Cases

Test 1 — Basic

```
Input:
hello

Output:
0 0 0 0 1 0 0 1 0 0 0 2 0 0 1 0 0 0 0 0 0 0 0 0
```

Test 2 — Single character

```
Input:
z

Output:
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1
```

Test 3 — All identical characters

```
Input:
aaaaaaa

Output:
8 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
```

Test 4 — Whole alphabet once

```
Input:
abcdefghijklmnopqrstuvwxyz

Output:
1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
```

Test 5 — Only the last letters

```
Input:
zzyyxx

Output:
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 2 2 2
```

Test 6 — Palindrome

```
Input:
abcba

Output:
2 2 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
```

91. Candy Crush

Topic: Arrays

Problem Statement

A board is an m by n grid of positive integers representing candy types, where 0 means an empty cell. Simulate the crushing rule until the board becomes stable. First, mark every candy that belongs to a horizontal or vertical run of three or more identical candies; all marked candies are crushed simultaneously and become 0. Then apply gravity: in every column, remaining candies fall down to fill the empty cells, keeping their order, and zeros rise to the top. Repeat the process while any run of three or more exists. Print the final stable board with m lines, each containing n integers separated by spaces.

Input / Output format: Line 1: m and n . Next m lines: the board. Output: the stable board, m lines of n integers.

Example

Input

```
3 3
1 1 1
2 2 3
3 2 3
```

Output

```
0 0 0
2 2 3
3 2 3
```

Explanation: The top row of 1s is crushed; nothing else forms a run afterwards. Result rows: 0 0 0 / 2 2 3 / 3 2 3.

Constraints

- $3 \leq m, n \leq 50$
- $0 \leq \text{board}[i][j] \leq 2000$ (0 = empty)

Test Cases

Test 1 — Basic

```
Input:
5 5
110 5 112 113 114
210 211 5 213 214
310 311 3 313 314
410 411 412 5 414
5 1 512 3 3

Output:
110 5 112 113 114
210 211 5 213 214
310 311 3 313 314
410 411 412 5 414
5 1 512 3 3
```

Test 2 — Horizontal crush only

```
Input:
3 3
5 5 5
1 2 3
4 6 7

Output:
0 0 0
1 2 3
4 6 7
```

Test 3 — Vertical crush only

```
Input:
3 3
1 7 3
2 7 4
5 7 6

Output:
1 0 3
2 0 4
5 0 6
```

Test 4 — Nothing to crush

```
Input:
3 3
1 2 3
4 5 6
7 8 9

Output:
1 2 3
4 5 6
7 8 9
```

Test 5 — Chain reaction

```
Input:
5 3
1 4 2
2 4 1
7 7 7
3 4 2
1 9 3

Output:
0 0 0
1 0 2
2 0 1
3 0 2
1 9 3
```


Test 6 — Entire board crushed

```
Input:
```

```
3 3
4 4 4
4 4 4
4 4 4
```

```
Output:
```

```
0 0 0
0 0 0
0 0 0
```

92. Check if a Number is a Harshad (Niven) Number

Topic: Mathematics

Problem Statement

A Harshad number, also called a Niven number, is a positive integer that is divisible by the sum of its own decimal digits. For example, 18 is a Harshad number because the digit sum is $1 + 8 = 9$ and 18 is divisible by 9, while 19 is not because 19 is not divisible by 10. Given a positive integer n , determine whether it is a Harshad number. Extract the digits with repeated division by ten, add them to obtain the digit sum, and then test whether n modulo the digit sum equals zero. Print true if the number is a Harshad number and false otherwise. Every single digit number is trivially a Harshad number.

Input / Output format: One line: the integer n . Output: true or false.

Example

Input

```
18
```

Output

```
true
```

Explanation: Digit sum is 9 and $18 \bmod 9 = 0$, so the answer is true.

Constraints

- $1 \leq n \leq 10^9$

Test Cases

Test 1 — Harshad number

```
Input:  
21
```

```
Output:  
true
```

Test 2 — Smallest input

```
Input:  
1
```

```
Output:  
true
```

Test 3 — Not Harshad

```
Input:  
19  
  
Output:  
false
```

Test 4 — Power of ten

```
Input:  
1000000000  
  
Output:  
true
```

Test 5 — Two digit non-Harshad

```
Input:  
11  
  
Output:  
false
```

Test 6 — Larger Harshad

```
Input:  
1729  
  
Output:  
true
```

93. Check if a Number is a Strong Number

Topic: Mathematics

Problem Statement

A strong number is a number whose value equals the sum of the factorials of its digits. For example, 145 is strong because $1! + 4! + 5! = 1 + 24 + 120 = 145$, while 146 is not. Given a positive integer n , decide whether it is a strong number. Precompute the factorials of the digits 0 to 9 in a small array, then split n into digits with repeated modulo and division by ten and add up the corresponding factorials. Compare the total with the original number. Print true if the number is strong and false otherwise. The numbers 1 and 2 are both strong, because $1! = 1$ and $2! = 2$.

Input / Output format: One line: the integer n . Output: true or false.

Example

Input

```
145
```

Output

```
true
```

Explanation: $1! + 4! + 5! = 145$, so the answer is true.

Constraints

- $1 \leq n \leq 10^9$

Test Cases

Test 1 — Strong number

```
Input:
40585

Output:
true
```

Test 2 — One

```
Input:
1

Output:
true
```

Test 3 — Two

```
Input:
2

Output:
true
```

Test 4 — Not strong

```
Input:
123

Output:
```

```
false
```

Test 5 — Three is not strong

```
Input:
3

Output:
false
```

Test 6 — Large non-strong

```
Input:
999999999

Output:
false
```

94. Check if a Number is an Abundant Number

Topic: Mathematics

Problem Statement

An abundant number is a positive integer whose proper divisors add up to more than the number itself. A proper divisor of n is any positive divisor of n other than n . For example, the proper divisors of 12 are 1, 2, 3, 4 and 6, and their sum is 16, which is greater than 12, so 12 is abundant. A number whose proper divisors sum exactly to the number, such as 6, is perfect and not abundant. Given n , print true if it is abundant and false otherwise. Enumerate the divisors only up to the square root of n , adding both the divisor and its pair, and be careful with perfect squares and with n equal to 1.

Input / Output format: One line: the integer n. Output: true or false.

Example

Input

```
12
```

Output

```
true
```

Explanation: $1 + 2 + 3 + 4 + 6 = 16 > 12$, so the answer is true.

Constraints

- $1 \leq n \leq 10^9$

Test Cases

Test 1 — Abundant number

```
Input:  
18
```

```
Output:  
true
```

Test 2 — One has no proper divisors

```
Input:  
1
```

```
Output:  
false
```

Test 3 — Perfect number is not abundant

```
Input:  
6
```

```
Output:  
false
```

Test 4 — Prime number

```
Input:  
13
```

```
Output:  
false
```

Test 5 — Perfect square that is deficient

```
Input:  
49
```

```
Output:  
false
```

Test 6 — Smallest odd abundant number


```
Input:  
945
```

```
Output:  
true
```

95. Check if an Array Is a Subset of Another

Topic: Hashing

Problem Statement

Given two arrays a_1 and a_2 , check whether a_2 is a subset of a_1 . Array a_2 is a subset of a_1 if every value that occurs in a_2 also occurs somewhere in a_1 . The arrays are unsorted and their sizes can differ. Only the presence of a value matters, so a value that appears twice in a_2 needs just one occurrence in a_1 . Solve the problem in linear expected time by inserting all elements of a_1 into a hash set and then checking that each element of a_2 is contained in it. Sorting both arrays and using two pointers is an alternative with $O(n \log n)$ time. Print true if a_2 is a subset of a_1 , otherwise false.

Input / Output format: Line 1: n . Line 2: array a_1 . Line 3: m . Line 4: array a_2 . Output: true or false.

Example

Input

```
6  
11 1 13 21 3 7  
4  
11 3 7 1
```

Output

```
true
```

Explanation: All of 11, 3, 7 and 1 occur in the first array: true.

Constraints

- $1 \leq n, m \leq 10^5$
- $-10^9 \leq \text{elements} \leq 10^9$

Test Cases

Test 1 — Not a subset

```
Input:
6
1 2 3 4 5 6
3
1 2 9
```

```
Output:
false
```

Test 2 — Single elements match

```
Input:
1
5
1
5
```

```
Output:
true
```

Test 3 — Single elements differ

```
Input:
1
5
1
6

Output:
false
```

Test 4 — Identical arrays

```
Input:
3
3 1 2
3
1 2 3

Output:
true
```

Test 5 — Duplicates in a2

```
Input:
4
1 2 3 4
4
2 2 2 2

Output:
true
```

Test 6 — a2 larger than a1 in count

```
Input:
```

```
2
1 2
5
1 2 1 2 1
```

```
Output:
true
```

96. Check Whether a Floating-Point Number Is Even or Odd

Topic: Mathematics

⚠ **Note:** Statement reconstructed: exact original semantics unknown; interpreted as parity of the integer part. Verify with your senior.

Problem Statement

You are given a floating-point number. Decide whether it should be treated as even or odd by looking at its integer part, which is the value obtained by discarding the fractional digits and truncating toward zero. If the integer part is divisible by two, print Even; otherwise print Odd. For instance, 4.0 and 10.999 are Even, while 7.9 and -3.2 are Odd. Convert the value to an integer type using truncation, then apply the remainder operator with two, remembering that the remainder of a negative odd integer can be negative in some languages, so compare it with zero rather than with one. Print exactly Even or Odd.

Input / Output format: One line: a floating-point number. Output: Even or Odd.

Example

Input

```
7.9
```

Output

```
Odd
```

Explanation: The integer part of 7.9 is 7, which is odd: Odd.

Constraints

- $-10^9 \leq x \leq 10^9$
- x is given with at most 6 digits after the decimal point

Test Cases

Test 1 — Even integral value

```
Input:
4.0

Output:
Even
```

Test 2 — Odd with fraction

```
Input:
15.75

Output:
Odd
```

Test 3 — Zero

```
Input:
0.0

Output:
Even
```

Test 4 — Small positive fraction

```
Input:
0.5

Output:
Even
```

Test 5 — Negative odd integer part

```
Input:
-3.2

Output:
Odd
```

Test 6 — Large even value

```
Input:
100000000.999999

Output:
Even
```

97. Check Whether a Given Number Is Even or Odd

Topic: Mathematics

Problem Statement

You are given an integer n , which may be negative or zero. Print Even if n is divisible by 2 and Odd otherwise. Zero is considered an even number. This is one of the simplest programming exercises, but it still contains a few traps. The remainder operator can return a negative value for negative inputs in languages such as C, C++ and Java, so testing whether $n \% 2$ equals 1 would wrongly treat -3 as even; test whether $n \% 2$ is zero, or use the bitwise operation $n \& 1$ which works for negative numbers in two's complement form. Print exactly Even or Odd with the first letter capitalised.

Input / Output format: One line: the integer n . Output: Even or Odd.

Example

Input

```
10
```

Output

```
Even
```

Explanation: 10 is divisible by 2: Even.

Constraints

- $-10^{18} \leq n \leq 10^{18}$

Test Cases

Test 1 — Positive odd

```
Input:
7

Output:
Odd
```

Test 2 — Zero

```
Input:
0

Output:
Even
```

Test 3 — Negative odd

```
Input:
-3

Output:
Odd
```

Test 4 — Negative even

```
Input:
-8
```



```
Output:
Even
```

Test 5 — Large odd

```
Input:  
999999999999999999  
  
Output:  
Odd
```

Test 6 — One

```
Input:
1

Output:
Odd
```

98. Check Whether a Number Is Positive or Negative

Topic: Mathematics

Problem Statement

You are given an integer n . Determine its sign and print Positive if n is greater than zero, Negative if n is less than zero, and Zero if n is exactly zero. Even though the task looks trivial, be careful about the third case, since zero is neither positive nor negative, and about very large magnitudes, which need a 64 bit integer type. Use a chain of conditional statements: first check whether n is greater than zero, then whether it is less than zero, and

treat everything else as zero. The program should read a single integer and print exactly one of the three words, with only the first letter in capitals.

Input / Output format: One line: the integer n. Output: Positive, Negative or Zero.

Example

Input

```
-15
```

Output

```
Negative
```

Explanation: -15 is less than zero: Negative.

Constraints

- $-10^{18} \leq n \leq 10^{18}$

Test Cases

Test 1 — Positive number

```
Input:  
42
```

```
Output:
```

Positive

Test 2 — Zero

```
Input:
0

Output:
Zero
```

Test 3 — Negative one

```
Input:
-1

Output:
Negative
```

Test 4 — Positive one

```
Input:
1

Output:
Positive
```

Test 5 — Very large positive

```
Input:  
1000000000000000000  
  
Output:  
Positive
```

Test 6 — Very large negative

```
Input:  
-1000000000000000000  
  
Output:  
Negative
```

99. Confusing Number

Topic: Mathematics

Problem Statement

A confusing number is a number that becomes a different valid number when it is rotated by 180 degrees. When rotated, the digits 0, 1, 8 stay the same, the digit 6 becomes 9, and the digit 9 becomes 6, while the digits 2, 3, 4, 5 and 7 become invalid. The rotated number is read in the reversed order of the digits, and leading zeros in the rotated result are ignored. Given a non-negative integer n , determine whether it is a confusing number: all its digits must be rotatable, and the rotated value must differ from n . Print true if n is confusing and false otherwise. For example, 6 becomes 9, but 69 rotates back to 69.

Input / Output format: One line: the integer n. Output: true or false.

Example

Input

6

Output

```
true
```

Explanation: 6 rotates into 9, which is a valid different number: true.

Constraints

- $0 \leq n \leq 10^9$

Test Cases

Test 1 — Two digit confusing number

```
Input:
89

Output:
true
```

Test 2 — Zero rotates to itself

```
Input:
0

Output:
false
```

Test 3 — Invalid digit

```
Input:  
25
```

```
Output:  
false
```

Test 4 — Self-symmetric number

```
Input:  
69
```

```
Output:  
false
```

Test 5 — Rotation equals the number

```
Input:  
1001
```

```
Output:  
false
```

Test 6 — Leading zero after rotation

```
Input:  
10
```

```
Output:  
true
```

100. Convert a Number to Hexadecimal

Topic: Mathematics

Problem Statement

Given a 32 bit signed integer, convert it to its hexadecimal representation and print it using lowercase letters a to f, without any leading zeros except when the number itself is zero. Negative numbers are represented using the two's complement method on 32 bits, so for example -1 becomes ffffffff. Do not use the built-in conversion or formatting functions of your language. Repeatedly extract the lowest four bits with a bitwise AND with 15, map the value 0 to 15 to its character, shift the number right by four bits using a logical shift, and finally reverse the collected characters. For negative inputs, at most eight iterations are needed.

Input / Output format: One line: the integer n. Output: the hexadecimal string (lowercase, no prefix).

Example

Input

```
26
```

Output

```
1a
```

Explanation: $26 = 1 \times 16 + 10$, so the hexadecimal form is 1a.

Constraints

- $-2^{31} \leq n \leq 2^{31} - 1$

Test Cases

Test 1 — Positive number

```
Input:
255

Output:
ff
```

Test 2 — Zero

```
Input:
0

Output:
0
```

Test 3 — Minus one

```
Input:
-1

Output:
ffffffff
```

Test 4 — Power of sixteen

```
Input:
16

Output:
```



```
10
```

Test 5 — Maximum 32-bit value

```
Input:
2147483647

Output:
7fffffff
```

Test 6 — Minimum 32-bit value

```
Input:
-2147483648

Output:
80000000
```

101. Convert Decimal to Binary Number

Topic: Mathematics

Problem Statement

Given a non-negative decimal integer n , print its binary representation without leading zeros. Zero should be printed as a single 0. Do not use built-in conversion functions. The standard method repeatedly divides the number by two and records the remainder, which is either 0 or 1, and since the first remainder is the least significant bit, the remainders must be printed in reverse order of their generation. You can store the bits in an array or a string and reverse it at the end, or build the number by pushing each remainder onto a stack. Alternatively use right shifts and bitwise AND with one to read the bits one by one.

Input / Output format: One line: the integer n. Output: the binary representation.

Example

Input

```
10
```

Output

```
1010
```

Explanation: $10 = 8 + 2$, so the binary form is 1010.

Constraints

- $0 \leq n \leq 10^{18}$

Test Cases

Test 1 — Basic

```
Input:  
37
```

```
Output:  
100101
```

Test 2 — Zero

```
Input:
0

Output:
0
```

Test 3 — One

```
Input:
1

Output:
1
```

Test 4 — Power of two

```
Input:
1024

Output:
10000000000
```

Test 5 — All ones

```
Input:
255

Output:
11111111
```

Test 6 — Very large value

```
Input:  
10000000000000000000  
  
Output:  
11011110000010110110101100111010011101100100000000000000000
```

```
Input:  
10000000000000000000  
  
Output:  
1101111000001011011010110011101001110110010000000000000000
```

102. Convert Decimal to Octal Number

Topic: Mathematics

Problem Statement

Given a non-negative decimal integer n , print its octal representation, which uses only the digits 0 to 7 and has no leading zeros. Zero is printed as 0. Avoid built-in base conversion routines and implement the conversion yourself: repeatedly take the remainder of n divided by eight, which gives the next digit starting from the least significant one, then replace n by n divided by eight using integer division until it becomes zero. Because the digits are produced in reverse order, store them and print them backwards, or accumulate the answer as a number using increasing powers of ten. Ensure that your program uses 64 bit integers for large inputs.

Input / Output format: One line: the integer n. Output: the octal representation.

Example

Input

Output

```
100
```

Explanation: $64 = 1 \times 64$, so the octal form is 100.

Constraints

- $0 \leq n \leq 10^{18}$

Test Cases

Test 1 — Basic

```
Input:
100

Output:
144
```

Test 2 — Zero

```
Input:
0

Output:
0
```

Test 3 — Single octal digit

```
Input:
7

Output:
```

7

Test 4 — Exactly eight

Input:
8

Output:
10

Test 5 — Large value

Input:
123456789012345

Output:
3404420603357571

Test 6 — Maximum input

Input:
10000000000000000000

Output:
67405553164731000000

103. Convert Digits / Numbers to Words

Topic: Mathematics

Problem Statement

Convert a non-negative integer into its English words representation. Use capitalised words separated by single spaces, without the word 'and' and without hyphens, so 123 becomes One Hundred Twenty Three and 1000010 becomes One Million Ten. Zero is written as Zero. Group the number into chunks of three digits from the right, convert each chunk of up to three digits using arrays for the words of 0 to 19 and of the multiples of ten, and append the scale word Thousand, Million or Billion for every non-empty chunk. Chunks that are entirely zero contribute nothing, so 1000000 is just One Million. The largest input is 2147483647.

Input / Output format: One line: the integer n. Output: the number in words.

Example

Input

```
123
```

Output

```
One Hundred Twenty Three
```

Explanation: Hundreds: One Hundred, remainder 23: Twenty Three.

Constraints

- $0 \leq n \leq 2^{31} - 1$

Test Cases

Test 1 — Basic

```
Input:
12345

Output:
Twelve Thousand Three Hundred Forty Five
```

Test 2 — Zero

```
Input:
0

Output:
Zero
```

Test 3 — Single digit

```
Input:
7

Output:
Seven
```

Test 4 — Teens

```
Input:
19

Output:
Nineteen
```

Test 5 — Zero chunks are skipped


```
Input:
1000010

Output:
One Million Ten
```

Test 6 — Maximum input

```
Input:
2147483647

Output:
Two Billion One Hundred Forty Seven Million Four Hundred Eighty
Three Thousand Six Hundred Forty Seven
```

104. Convex Polygon

Topic: Mathematics

Problem Statement

You are given the vertices of a simple polygon in the order in which they appear along its boundary, either clockwise or counterclockwise. Determine whether the polygon is convex, which means that every interior angle is at most 180 degrees and the polygon never turns back on itself. A reliable test uses the cross product of consecutive edge vectors: for each triple of consecutive vertices compute the sign of the cross product, ignore zero values that come from three collinear points, and check that all the non-zero signs are identical. If both a positive and a negative turn appear, the polygon is concave. Print true for a convex polygon and false otherwise.

Input / Output format: Line 1: n . Next n lines: 'x y' (vertices in boundary order). Output: true or false.

Example

Input

```
4
0 0
0 5
5 5
5 0
```

Output

```
true
```

Explanation: A square has four turns in the same direction: true.

Constraints

- $3 \leq n \leq 10^4$
- $-10^4 \leq x, y \leq 10^4$
- The polygon is simple (no self-intersections)

Test Cases

Test 1 — Concave polygon

```
Input:
5
0 0
4 0
2 1
4 4
0 4
```

```
Output:  
false
```

Test 2 — Triangle

```
Input:  
3  
0 0  
4 0  
0 3  
  
Output:  
true
```

Test 3 — Collinear extra vertex

```
Input:  
5  
0 0  
2 0  
4 0  
4 4  
0 4  
  
Output:  
true
```

Test 4 — Counterclockwise convex

```
Input:  
4  
0 0  
5 0  
5 5  
0 5  
  
Output:  
true
```

Test 5 — Concave arrow shape

```
Input:
4
0 0
4 0
1 1
0 4

Output:
false
```

Test 6 — Hexagon

```
Input:
6
0 0
2 -1
4 0
4 2
2 3
0 2

Output:
true
```

105. Count Prefix Appearances

Topic: Hashing

Problem Statement

You are given an array of words and a string s . Count how many of the words in the array are prefixes of s . A word is a prefix of s if s begins with exactly the characters of that word. If the

same word appears several times in the array, every occurrence is counted separately, and a word that is longer than s can never be a prefix. Comparing each word with the beginning of s directly takes time proportional to the total length of the words. A more scalable approach stores all prefixes of s in a hash set, since there are only $|s|$ of them, and looks up each word in that set. Print the number of matching words.

Input / Output format: Line 1: n . Line 2: n words separated by spaces. Line 3: the string s .
Output: the count.

Example

Input

```
5
a b c ab bc
abc
```

Output

```
2
```

Explanation: a and ab are prefixes of abc ; b , c and bc are not. Answer: 2.

Constraints

- $1 \leq n \leq 1000$
- $1 \leq |\text{word}|, |s| \leq 10$
- Lowercase English letters only

Test Cases

Test 1 — Basic

```
Input:
4
app a ap apple
apple

Output:
4
```

Test 2 — Single word that matches

```
Input:
1
x
xyz

Output:
1
```

Test 3 — No prefix

```
Input:
3
b c d
abc

Output:
0
```

Test 4 — Repeated words count separately

```
Input:
4
a a ab ab
abc

Output:
```

Test 5 — Word longer than s

```
Input:
2
abcd abcde
abc
```

```
Output:
0
```

Test 6 — Word equals s

```
Input:
3
abc abc ab
abc
```

```
Output:
3
```

106. Count the Number of Good Subsequences

Topic: Hashing

Problem Statement

A subsequence of a string is good if it is non-empty and every character that appears in it appears the same number of times. For example, from the string aabb, the subsequence ab is good, aabb is good, but aab is not, because a appears twice and b once. Count the number of good subsequences of the given string, where subsequences are counted by the positions chosen, and print the answer modulo 1,000,000,007. Fix a common frequency f ; for every

distinct letter with count c you may either skip it or choose exactly f of its c occurrences in $C(c, f)$ ways. Multiply these choices, subtract the empty selection, and sum over all f .

Input / Output format: One line: the string s . Output: the number of good subsequences modulo $10^9 + 7$.

Example

Input

```
aabb
```

Output

```
11
```

Explanation: $f = 1$: $(1+2)(1+2) - 1 = 8$; $f = 2$: $(1+1)(1+1) - 1 = 3$. Total 11.

Constraints

- $1 \leq |s| \leq 10^4$
- s contains lowercase English letters

Test Cases

Test 1 — Basic

```
Input :  
abcd
```



```
Output:  
15
```

Test 2 — Single character

```
Input:  
a  
  
Output:  
1
```

Test 3 — All characters identical

```
Input:  
aaaa  
  
Output:  
15
```

Test 4 — Two letters, unequal counts

```
Input:  
aaabb  
  
Output:  
19
```

Test 5 — Three letters

```
Input:  
aabbcc  
  
Output:
```

Test 6 — Modulo needed

```
Input:
abcdefghijklmnopqrstuvwxyzabcdefghijklmnopqrstuvwxyz

Output:
932919404
```

107. Counting Elements

Topic: Hashing

Problem Statement

Given an integer array, count how many elements x have the property that $x + 1$ is also present somewhere in the array. If the array contains duplicates, every duplicate is counted separately, so an array holding two copies of 1 and one copy of 2 contributes two to the answer. Print the total count. The intended solution inserts all elements into a hash set, then makes a second pass over the array and counts the elements whose successor value is in the set, which takes linear time. An alternative approach sorts the array and compares runs of equal values, but the hash based method is simpler and faster.

Input / Output format: Line 1: n . Line 2: n integers. Output: the count.

Example

Input

```
5
1 2 3 3 5
```

Output

```
2
```

Explanation: 1 has 2 present and 2 has 3 present; both 3s and the 5 have no successor.
Answer: 2.

Constraints

- $1 \leq n \leq 1000$
- $0 \leq a[i] \leq 1000$

Test Cases

Test 1 — Basic

```
Input:
6
1 3 2 3 5 0

Output:
3
```

Test 2 — Single element

```
Input:
1
7
```

```
Output:  
0
```

Test 3 — All equal

```
Input:  
4  
1 1 1 1
```

```
Output:  
0
```

Test 4 — Duplicates each count

```
Input:  
5  
1 1 2 2 3
```

```
Output:  
4
```

Test 5 — Consecutive chain

```
Input:  
5  
1 2 3 4 5
```

```
Output:  
4
```

Test 6 — Gaps only

```
Input:  
4
```

```
2 4 6 8
```

```
Output:  
0
```

108. Counting Frequencies of Array Elements

Topic: Hashing

Problem Statement

Given an array of integers, print the number of times each distinct value occurs in the array. Output one line for every distinct value in the order in which the value first appears in the input, using the exact format '<value> occurs <count> times', even when the count is one. Use a hash map from value to count: iterate once over the array to accumulate counts, then iterate again over the original array and print each value only the first time it is encountered, marking it as printed or removing it from the map. An ordered map keyed by first appearance is another convenient option. The running time is linear in the size of the array.

Input / Output format: Line 1: n. Line 2: n integers. Output: one line per distinct value: '<value> occurs <count> times'.

Example

Input

```
10  
1 2 3 3 4 1 4 5 1 2
```

Output

```
1 occurs 3 times
2 occurs 2 times
3 occurs 2 times
4 occurs 2 times
5 occurs 1 times
```

Explanation: 1 occurs 3 times / 2 occurs 2 times / 3 occurs 2 times / 4 occurs 2 times / 5 occurs 1 times (one per line).

Constraints

- $1 \leq n \leq 10^5$
- $-10^9 \leq a[i] \leq 10^9$

Test Cases

Test 1 — Basic

```
Input:
6
5 5 7 7 7 2

Output:
5 occurs 2 times
7 occurs 3 times
2 occurs 1 times
```

Test 2 — Single element

```
Input:
1
9

Output:
9 occurs 1 times
```

Test 3 — All identical

```
Input:
4
3 3 3 3

Output:
3 occurs 4 times
```

Test 4 — All distinct

```
Input:
4
4 3 2 1

Output:
4 occurs 1 times
3 occurs 1 times
2 occurs 1 times
1 occurs 1 times
```

Test 5 — Negative values

```
Input:
5
-1 -1 0 -2 0

Output:
-1 occurs 2 times
0 occurs 2 times
-2 occurs 1 times
```

Test 6 — Order of first appearance

```
Input:
6
```

```
3 1 3 2 1 2
```

Output:

```
3 occurs 2 times
```

```
1 occurs 2 times
```

```
2 occurs 2 times
```

109. Divide Array Into Increasing Sequences

Topic: Hashing

Problem Statement

You are given an array sorted in non-decreasing order and an integer k . Determine whether it is possible to divide the array into one or more disjoint strictly increasing subsequences such that each subsequence has length at least k . Every element must belong to exactly one subsequence. The key observation is that copies of the same value can never be in the same strictly increasing subsequence, so the number of subsequences must be at least the maximum frequency of any value. With that many subsequences, each of length at least k , you need at least k times the maximum frequency elements. Print true if the division is possible and false otherwise.

Input / Output format: Line 1: n and k . Line 2: n sorted integers. Output: true or false.

Example

Input

```
6 3
1 2 2 3 3 4
```

Output


```
true
```

Explanation: Two sequences are needed (value 2 and 3 appear twice), and $2 \times 3 = 6 \leq 6$: true, e.g. 1 2 3 and 2 3 4.

Constraints

- $1 \leq k \leq n \leq 10^5$
- $1 \leq a[i] \leq 10^5$
- The array is sorted in non-decreasing order

Test Cases

Test 1 — Not possible

```
Input:
5 3
5 6 6 7 8

Output:
false
```

Test 2 — Single element with $k = 1$

```
Input:
1 1
4

Output:
true
```

Test 3 — $k = n$ with distinct values

```
Input:
4 4
1 2 3 4

Output:
true
```

Test 4 — All equal, $k = 1$

```
Input:
3 1
2 2 2

Output:
true
```

Test 5 — All equal, $k = 2$

```
Input:
3 2
2 2 2

Output:
false
```

Test 6 — Exactly enough elements

```
Input:
6 2
1 1 2 2 3 3

Output:
true
```

110. Dot Product of Two Sparse Vectors

Topic: Arrays

Problem Statement

A sparse vector is a vector in which most entries are zero. Given two sparse vectors of the same length, compute their dot product, which is the sum of the products of the corresponding entries. Storing and multiplying every entry wastes time when most values are zero, so the efficient representation keeps only the non-zero entries as pairs of index and value, or in a hash map. To compute the dot product, iterate over the smaller collection and look up matching indices in the other, or walk both index-sorted lists with two pointers. The result can be large, so use a 64 bit integer. Print a single integer.

Input / Output format: Line 1: n. Line 2: vector a. Line 3: vector b. Output: the dot product.

Example

Input

```
5
1 0 0 2 3
0 3 0 4 0
```

Output

```
8
```

Explanation: Only index 3 has non-zero entries in both: $2 \times 4 = 8$.

Constraints

- $1 \leq n \leq 10^5$
- $0 \leq a[i], b[i] \leq 100$

Test Cases

Test 1 — Basic

```
Input:
4
1 2 0 3
2 0 5 4

Output:
14
```

Test 2 — Single element

```
Input:
1
5
6

Output:
30
```

Test 3 — All zeros

```
Input:
3
0 0 0
0 0 0

Output:
```

```
0
```

Test 4 — No overlapping non-zero entries

```
Input:
4
1 0 2 0
0 3 0 4

Output:
0
```

Test 5 — Dense vectors

```
Input:
4
1 2 3 4
5 6 7 8

Output:
70
```

Test 6 — Maximum values

```
Input:
3
100 100 100
100 100 100

Output:
30000
```

111. MEXX (Minimum Excluded Value)

Topic: Hashing

⚠ **Note:** Statement reconstructed: the exact original problem could not be located; interpreted as the standard MEX of an array. Verify with your senior.

Problem Statement

The MEX of an array is the smallest non-negative integer that does not appear in the array. For example, the MEX of 0, 1, 2, 4 is 3 and the MEX of 1, 2, 3 is 0. Given an array of non-negative integers, print its MEX. A simple approach inserts all elements into a hash set and then checks the integers 0, 1, 2, and so on until one is missing; the answer can never exceed the array length, so this loop is short. You can also use a boolean array of size $n + 1$ that marks every value not larger than n , and then scan for the first unmarked index. The time complexity is linear.

Input / Output format: Line 1: n . Line 2: n integers. Output: the MEX.

Example

Input

```
5
0 1 2 4 5
```

Output

```
3
```

Explanation: The values 0, 1 and 2 are present but 3 is missing: 3.

Constraints

- $1 \leq n \leq 10^5$
- $0 \leq a[i] \leq 10^9$

Test Cases

Test 1 — Basic

```
Input:
6
2 0 1 5 6 3

Output:
4
```

Test 2 — Single element zero

```
Input:
1
0

Output:
1
```

Test 3 — Single element non-zero

```
Input:
1
5

Output:
0
```

Test 4 — No zero present

```
Input:
4
1 2 3 4

Output:
0
```

Test 5 — Complete range

```
Input:
5
4 3 2 1 0

Output:
5
```

Test 6 — Duplicates and large values

```
Input:
6
0 0 1 1 1000000000 3

Output:
2
```

112. Highest Occurring Element in an Array

Topic: Hashing

Problem Statement

Given an array of integers, find the element that occurs the highest number of times. If several elements share the maximum frequency, print the smallest of them. Build a frequency map with one pass over the array, then scan the map keeping the best count and, in case of a tie, the smaller value. The solution takes linear time and linear extra space. If all the elements are distinct, every element has frequency one, and the answer is therefore the smallest element of the array. Print only the value of the element, not its frequency. The input can contain negative numbers and zeros.

Input / Output format: Line 1: n. Line 2: n integers. Output: the most frequent element (smallest value on ties).

Example

Input

```
7
4 1 4 2 2 4 1
```

Output

```
4
```

Explanation: Value 4 occurs three times, more than any other value: 4.

Constraints

- $1 \leq n \leq 10^5$
- $-10^9 \leq a[i] \leq 10^9$

Test Cases

Test 1 — Basic

```
Input:
6
5 3 5 3 5 1

Output:
5
```

Test 2 — Single element

```
Input:
1
8

Output:
8
```

Test 3 — All distinct

```
Input:
4
9 3 7 5

Output:
3
```

Test 4 — Tie, smallest wins

```
Input:
6
4 4 2 2 9 9

Output:
2
```

Test 5 — Negative values

```
Input:
5
-2 -2 -2 3 3

Output:
-2
```

Test 6 — All equal

```
Input:
3
6 6 6

Output:
6
```

113. Second Highest Occurring Element

Topic: Hashing

Problem Statement

Given an array of integers, find the element whose frequency is the second highest among the distinct frequency values that occur in the array. Elements that share the same frequency are on the same level, so if the highest frequency is 4 and the next lower distinct frequency is 3, the answer is chosen from the elements that occur exactly three times. If several elements occur at that level, print the smallest. If the array has fewer than two distinct frequency values, print -1. A hash map of counts followed by sorting the distinct frequencies in decreasing order gives a clean solution in near linear time.

Input / Output format: Line 1: n . Line 2: n integers. Output: the element with the second highest frequency, or -1.

Example

Input

```
8
1 1 1 2 2 3 3 4
```

Output

```
2
```

Explanation: Frequencies: $1 \rightarrow 3$, $2 \rightarrow 2$, $3 \rightarrow 2$, $4 \rightarrow 1$. The second highest frequency is 2, shared by 2 and 3; the smaller is 2.

Constraints

- $1 \leq n \leq 10^5$
- $-10^9 \leq a[i] \leq 10^9$

Test Cases

Test 1 — Basic

```
Input:
7
5 5 5 6 6 7 8

Output:
```

```
6
```

Test 2 — Single element

```
Input:
1
4

Output:
-1
```

Test 3 — All frequencies equal

```
Input:
4
1 1 2 2

Output:
-1
```

Test 4 — Two levels only

```
Input:
5
3 3 3 9 9

Output:
9
```

Test 5 — Ties at the second level

```
Input:
9
1 1 1 1 5 5 3 3 8
```

```
Output:  
3
```

Test 6 — Negative values

```
Input:  
6  
-1 -1 -1 -2 -2 0  
  
Output:  
-2
```

114. Sum of Highest and Lowest Frequency

Topic: Hashing

Problem Statement

Given an array of integers, count how many times each distinct value appears. Let the highest frequency be the largest of these counts and the lowest frequency the smallest of these counts. Print the sum of the highest frequency and the lowest frequency. For instance, in the array 1 1 1 2 2 3 the counts are 3, 2 and 1, so the answer is $3 + 1 = 4$. If only one distinct value exists, the highest and lowest frequencies are the same, and both are added. Use a hash map for counting, then scan its values to find the maximum and the minimum. The running time is linear in the array size.

Input / Output format: Line 1: n . Line 2: n integers. Output: highest frequency + lowest frequency.

Example

Input

```
6
1 1 1 2 2 3
```

Output

```
4
```

Explanation: Counts are 3, 2, 1, so $3 + 1 = 4$.

Constraints

- $1 \leq n \leq 10^5$
- $-10^9 \leq a[i] \leq 10^9$

Test Cases

Test 1 — Basic

```
Input:
8
4 4 4 4 5 5 6 6

Output:
6
```

Test 2 — Single element

```
Input:
1
```

```
9
```

```
Output:  
2
```

Test 3 — All identical

```
Input:  
5  
2 2 2 2 2
```

```
Output:  
10
```

Test 4 — All distinct

```
Input:  
4  
1 2 3 4
```

```
Output:  
2
```

Test 5 — Two values

```
Input:  
5  
7 7 7 8 8
```

```
Output:  
5
```

Test 6 — Negative values

```
Input:
```



```
6
-1 -1 -1 -1 0 5

Output:
5
```

115. Reverse an Array – 2 (Reverse in Groups)

Topic: Arrays

⚠ **Note:** Statement reconstructed: 'Reverse an array 2' has no standard public definition; interpreted as reversal in groups of size k. Verify with your senior.

Problem Statement

Given an array and an integer k, reverse the array in groups of size k. Divide the array from left to right into consecutive blocks of k elements, and reverse the order of the elements inside each block. If the last block contains fewer than k elements, reverse whatever elements it holds. When k is one the array is unchanged, and when k is at least the array length the whole array is simply reversed. Implement it in place with two pointers for each block, swapping the elements at the ends of the block and moving inwards. The total time complexity is linear and the extra space is constant. Print the resulting array on a single line.

Input / Output format: Line 1: n and k. Line 2: n integers. Output: the array after group-wise reversal.

Example

Input

```
5 3
1 2 3 4 5
```

Output

```
3 2 1 5 4
```

Explanation: Blocks [1 2 3] and [4 5] become [3 2 1] and [5 4]: 3 2 1 5 4.

Constraints

- $1 \leq k \leq n \leq 10^5$
- $-10^9 \leq a[i] \leq 10^9$

Test Cases

Test 1 — Basic

```
Input:
6 2
1 2 3 4 5 6

Output:
2 1 4 3 6 5
```

Test 2 — Single element

```
Input:
1 1
5

Output:
5
```

Test 3 — $k = 1$, no change

```
Input:
4 1
1 2 3 4

Output:
1 2 3 4
```

Test 4 — $k = n$, full reversal

```
Input:
4 4
1 2 3 4

Output:
4 3 2 1
```

Test 5 — Last block shorter

```
Input:
7 3
1 2 3 4 5 6 7

Output:
3 2 1 6 5 4 7
```

Test 6 — Negative values

```
Input:
5 2
-1 -2 -3 -4 -5

Output:
-2 -1 -4 -3 -5
```

116. Isomorphic Strings

Topic: Hashing

Problem Statement

Two strings s and t are isomorphic if the characters of s can be replaced by characters of t to obtain t . All occurrences of a character must be replaced with the same character, and the mapping must be one-to-one, so two different characters of s cannot map to the same character of t . A character is allowed to map to itself. Given the strings, print true if they are isomorphic and false otherwise. Strings of different lengths can never be isomorphic. Maintain two hash maps, one from s to t and one from t to s , and check both directions for consistency at every position, or compare the index of the first occurrence of each character.

Input / Output format: Line 1: string s . Line 2: string t . Output: true or false.

Example

Input

```
egg  
add
```

Output

```
true
```

Explanation: $e \rightarrow a$ and $g \rightarrow d$ consistently, so the strings are isomorphic: true.

Constraints

- $1 \leq |s|, |t| \leq 5 \cdot 10^4$
- Strings contain letters, digits and symbols with no spaces

Test Cases

Test 1 — Not isomorphic (inconsistent mapping)

```
Input:
foo
bar

Output:
false
```

Test 2 — Single characters

```
Input:
a
b

Output:
true
```

Test 3 — Two characters map to one

```
Input:
ab
aa

Output:
false
```

Test 4 — Identical strings

```
Input:
abc
abc

Output:
true
```

Test 5 — Longer isomorphic pair

```
Input:
paper
title

Output:
true
```

Test 6 — Different lengths

```
Input:
ab
abc

Output:
false
```

117. Sort Characters by Frequency

Topic: Hashing

Problem Statement

Given a string, sort its characters in decreasing order of frequency and print the resulting string. Characters that occur more often must appear before characters that occur less often, and all copies of the same character must stay together. To make the result unique, characters with the same frequency should be arranged in increasing order of their ASCII values, so uppercase letters come before lowercase letters. The task is case sensitive, so the characters A and a are different. Count frequencies with a hash map or a fixed array of size 128, sort the distinct characters with the described comparator, and build the output by repeating each character according to its count.

Input / Output format: One line: the string. Output: the string sorted by decreasing frequency (ties by ASCII order).

Example

Input

```
tree
```

Output

```
eert
```

Explanation: e appears twice; r and t once each, ordered by ASCII: eert.

Constraints

- $1 \leq |s| \leq 5 \cdot 10^5$
- s consists of uppercase and lowercase letters and digits

Test Cases

Test 1 — Basic

```
Input:
cccaa

Output:
aaaccc
```

Test 2 — Single character

```
Input:
z

Output:
z
```

Test 3 — Case sensitive

```
Input:
Aabb

Output:
bbAa
```

Test 4 — All distinct

```
Input:
dcba

Output:
abcd
```

Test 5 — Digits and letters


```
Input:
2a2a1b
```

```
Output:
22aa1b
```

Test 6 — Ties broken by ASCII

```
Input:
bbaaBB
```

```
Output:
BBaabb
```

118. Valid Anagram

Topic: Hashing

Problem Statement

Given two strings s and t , determine whether t is an anagram of s . Two strings are anagrams when they contain exactly the same characters with exactly the same frequencies, possibly in a different order. The comparison is case sensitive, and strings of different lengths cannot be anagrams. Sorting both strings and comparing them works in $O(n \log n)$ time. A faster approach uses a frequency array of 26 integers for lowercase letters or a hash map for general characters: increase the counters while scanning s , decrease them while scanning t , and verify that all counters return to zero. Print true if t is an anagram of s and false otherwise.

Input / Output format: Line 1: string s . Line 2: string t . Output: true or false.

Example

Input

```
anagram
nagaram
```

Output

```
true
```

Explanation: Both strings contain a×3, n, g, r, m: true.

Constraints

- $1 \leq |s|, |t| \leq 5 \cdot 10^4$
- Strings contain English letters

Test Cases

Test 1 — Not an anagram

```
Input:
rat
car

Output:
false
```

Test 2 — Single character equal

```
Input:
a
a

Output:
true
```

Test 3 — Single character different

```
Input:
a
b

Output:
false
```

Test 4 — Different lengths

```
Input:
abc
ab

Output:
false
```

Test 5 — Same letters, different counts

```
Input:
aacc
ccac

Output:
false
```

Test 6 — Case sensitive

```
Input:
Listen
silent
```

```
Output:
false
```

119. Furthest Empty Plot from Buildings

Topic: Arrays

⚠ **Note:** Statement reconstructed: the original text could not be located; interpreted as maximising the distance to the nearest building. Verify with your senior.

Problem Statement

A street has n plots in a row, and each plot is described by a character: 1 means a building stands on it and 0 means it is empty. Among all empty plots, find the one that is furthest from the buildings, that is, the empty plot whose distance to its nearest building is the largest. The distance between plots i and j is the absolute difference of their indices. Print the index of that plot; if several plots share the largest distance, print the smallest index. If the street has no buildings or no empty plot, print -1. Compute the nearest building distance for every plot with a left-to-right and a right-to-left pass.

Input / Output format: Line 1: n . Line 2: a string of n characters (0 or 1). Output: the index of the furthest empty plot, or -1.

Example

Input

```
1000001
```

Output

```
3
```

Explanation: Plot 3 is three plots away from both buildings, the largest possible distance: 3.

Constraints

- $1 \leq n \leq 10^5$
- Each character is 0 or 1

Test Cases

Test 1 — Basic

```
Input:
8
10010000

Output:
7
```

Test 2 — No buildings

```
Input:
4
0000

Output:
```

```
-1
```

Test 3 — No empty plot

```
Input:
3
111

Output:
-1
```

Test 4 — Single building at the start

```
Input:
5
10000

Output:
4
```

Test 5 — Tie between plots, smallest index

```
Input:
6
100001

Output:
2
```

Test 6 — Building in the middle

```
Input:
5
00100
```

```
Output:  
0
```

120. Faulty Sensor

Topic: Arrays

⚠ **Note:** Corresponds to the LeetCode 'Faulty Sensor' problem.

Problem Statement

Two sensors record the same sequence of n values, so ideally their readings are identical. Exactly one sensor may be defective. A defective sensor drops one reading somewhere; all later readings shift one position to the left and the last position is filled with an arbitrary value that differs from the dropped reading. Given both arrays, find out which sensor is defective. Print 1 if it is the first sensor, 2 if it is the second sensor, and -1 if the defective one cannot be determined, which also covers identical arrays. Find the first position where the arrays differ and test both possible explanations by comparing the shifted remainders.

Input / Output format: Line 1: n . Line 2: readings of sensor 1. Line 3: readings of sensor 2.
Output: 1, 2 or -1.

Example

Input

```
4  
2 3 4 5  
2 1 3 4
```

Output

```
1
```

Explanation: Sensor 1 is missing the reading 1: from index 1 on, its values equal sensor 2's values shifted by one place, so sensor 1 is defective: 1.

Constraints

- $2 \leq n \leq 100$
- $1 \leq \text{reading} \leq 100$

Test Cases

Test 1 — Sensor 1 defective

```
Input:
4
2 3 2 3
2 3 4 2

Output:
1
```

Test 2 — Identical arrays

```
Input:
3
5 6 7
5 6 7

Output:
-1
```


Test 3 — Both sensors fit (ambiguous)

```
Input:
5
5 1 2 1 2
5 2 1 2 1

Output:
-1
```

Test 4 — Sensor 2 defective

```
Input:
3
1 2 3
1 3 9

Output:
2
```

Test 5 — Difference only in the last position

```
Input:
3
1 2 3
1 2 4

Output:
-1
```

Test 6 — Sensor 1 defective, early drop

```
Input:
4
1 3 4 9
1 2 3 4
```

```
Output:  
1
```

121. Find All Non-Repeating Elements in an Array

Topic: Hashing

Problem Statement

Given an array of integers, find all the elements that appear exactly once in the array. Print these elements in the same order in which they appear in the input, on a single line separated by spaces. If every element occurs more than once, print []. A frequency map built in one pass gives the counts, and a second pass over the original array selects the values whose count is exactly one, preserving the order without extra sorting. Elements that appear twice or more, even if they are separated by other values, are not part of the answer. The array can contain negative values and zeros, and its length can be large, so aim for linear time.

Input / Output format: Line 1: n. Line 2: n integers. Output: the elements that occur exactly once, in input order ([] if none).

Example

Input

```
7  
4 5 4 6 7 5 8
```

Output

```
6 7 8
```

Explanation: Values 6, 7 and 8 appear exactly once: 6 7 8.

Constraints

- $1 \leq n \leq 10^5$
- $-10^9 \leq a[i] \leq 10^9$

Test Cases

Test 1 — Basic

```
Input:
8
1 2 3 2 4 3 5 1

Output:
4 5
```

Test 2 — Single element

```
Input:
1
9

Output:
9
```

Test 3 — All distinct

```
Input:
4
4 3 2 1

Output:
4 3 2 1
```

Test 4 — Every value repeats

```
Input:
6
2 2 3 3 4 4

Output:
[]
```

Test 5 — Negative values and zero

```
Input:
6
0 -1 0 -2 -1 5

Output:
-2 5
```

Test 6 — All identical

```
Input:
4
7 7 7 7

Output:
[]
```

122. Find All Repeating Elements in an Array

Topic: Hashing

Problem Statement

Given an array of integers, print every value that occurs more than once. Each such value must be printed exactly once, in the order of its first appearance in the array. If there is no repeating value, print []. Use a hash map or a hash set: scan the array while counting occurrences, and record a value the moment its count becomes two, so that it is added to the answer only one time regardless of how many extra copies exist. This gives linear time and space complexity. The array may contain negative numbers and zeros, and the values can be large, so do not rely on a small index range.

Input / Output format: Line 1: n. Line 2: n integers. Output: the repeating values, each once, in order of first appearance ([] if none).

Example

Input

```
8
4 5 4 6 7 5 4 9
```

Output

```
4 5
```

Explanation: Values 4 and 5 occur more than once: 4 5.

Constraints

- $1 \leq n \leq 10^5$
- $-10^9 \leq a[i] \leq 10^9$

Test Cases

Test 1 — Basic

```
Input:
8
1 2 3 2 4 3 5 1

Output:
1 2 3
```

Test 2 — Single element

```
Input:
1
9

Output:
[]
```

Test 3 — All distinct

```
Input:
4
4 3 2 1

Output:
[]
```

Test 4 — All identical

```
Input:
4
7 7 7 7

Output:
7
```

Test 5 — Negative values and zero

```
Input:
6
0 -1 0 -2 -1 5

Output:
0 -1
```

Test 6 — Late repetition

```
Input:
5
3 8 6 1 3

Output:
3
```

123. Find All Symmetric Pairs

Topic: Hashing

Problem Statement

You are given a list of pairs of integers. Two pairs (a, b) and (c, d) are symmetric when c equals b and d equals a, that is, one is the reverse of the other. Find all symmetric pairs in the list and print each symmetric pair once, written with the smaller number first as 'a b' where $a < b$, in increasing lexicographic order. Pairs whose two numbers are equal are not reported. If no symmetric pair exists, print []. Store every pair in a hash set, and for each pair check whether its reverse is present, which makes the whole algorithm linear in the number of pairs. Repeated identical pairs do not create new answers.

Input / Output format: Line 1: n. Next n lines: 'a b'. Output: each symmetric pair ($a < b$) on its own line ([] if none).

Example

Input

```
5
11 20
30 40
5 10
40 30
10 5
```

Output

```
5 10
30 40
```

Explanation: (30, 40) and (40, 30) are symmetric, and so are (5, 10) and (10, 5). Sorted output: 5 10, then 30 40.

Constraints

- $1 \leq n \leq 10^5$
- $-10^9 \leq a, b \leq 10^9$

Test Cases

Test 1 — Basic

```
Input:
4
1 2
3 4
2 1
5 6

Output:
1 2
```

Test 2 — Single pair

```
Input:
1
1 2

Output:
[]
```

Test 3 — No symmetric pair

```
Input:
3
1 2
3 4
5 6

Output:
[]
```

Test 4 — Equal numbers are ignored

```
Input:
```

```
3  
2 2  
2 2  
3 4
```

```
Output:
```

```
[]
```

Test 5 — Duplicate pairs

```
Input:
```

```
4  
1 2  
1 2  
2 1  
2 1
```

```
Output:
```

```
1 2
```

Test 6 — Negative values

```
Input:
```

```
4  
-1 -2  
-2 -1  
0 5  
5 0
```

```
Output:
```

```
-2 -1  
0 5
```

124. Find Distinct Expressions

Topic: Hashing

⚠ **Note:** Statement reconstructed: the original text could not be located; interpreted as counting expressions up to commutativity. Verify with your senior.

Problem Statement

You are given a list of simple expressions, each written as 'a op b', where a and b are integers and op is one of +, -, _ or /. Two expressions are considered the same when they are identical, or when they differ only by swapping the operands of a commutative operator, which means + and _. For example, 3 + 5 and 5 + 3 are the same expression, but 3 - 5 and 5 - 3 are different. Count the number of distinct expressions in the list. Convert each expression to a canonical form, ordering the operands numerically for + and * and leaving them unchanged for - and /, then insert it into a hash set and report the size of the set.

Input / Output format: Line 1: n. Next n lines: 'a op b' (separated by spaces). Output: the number of distinct expressions.

Example

Input

```
4
3 + 5
5 + 3
3 - 5
5 - 3
```

Output

```
3
```

Explanation: 3 + 5 and 5 + 3 match; the two subtractions are different: 3 distinct expressions.

Constraints

- $1 \leq n \leq 10^5$
- $-10^9 \leq a, b \leq 10^9$
- $op \in \{+, -, *, /\}$

Test Cases

Test 1 — Basic

```
Input:
5
2 * 3
3 * 2
2 + 3
3 + 2
2 * 3

Output:
2
```

Test 2 — Single expression

```
Input:
1
1 + 1

Output:
1
```

Test 3 — All identical

```
Input:
3
4 / 2
```

```
4 / 2
4 / 2

Output:
1
```

Test 4 — Division is not commutative

```
Input:
2
8 / 2
2 / 8

Output:
2
```

Test 5 — Negative operands

```
Input:
3
-1 + 4
4 + -1
-1 - 4

Output:
2
```

Test 6 — Same operands, different operators

```
Input:
4
6 + 2
6 - 2
6 * 2
6 / 2

Output:
4
```

125. Flatten 2D Vector

Topic: Arrays

Problem Statement

Design an iterator over a two dimensional vector, a list of rows where each row is a list of integers and some rows may be empty. The iterator must visit the values row by row, from left to right, as if the vector were flattened into one long list. It supports two operations: `next`, which returns the next integer and moves the iterator forward, and `hasNext`, which reports whether any elements remain. Empty rows must be skipped transparently. Keep two pointers, a row index and a column index, and advance the row index past empty rows whenever needed. Process the given operations and print the result of each one on its own line, using `true` or `false` for `hasNext`.

Input / Output format: Line 1: `r`. Next `r` lines: '`len v1 v2 ... vlen`' (a row; '`len`' = 0 for an empty row). Then a line with `q`, followed by `q` lines: '`next`' or '`hasNext`'. Output: one line per operation.

Example

Input

```
3
2 1 2
1 3
2 4 5
7
next
next
hasNext
next
next
next
next
hasNext
```

Output

```
1
2
true
3
4
5
false
```

Explanation: Flattened order is 1 2 3 4 5. Results: 1, 2, true, 3, 4, 5, false.

Constraints

- $0 \leq r \leq 200$
- $0 \leq \text{total elements} \leq 500$
- $-500 \leq \text{value} \leq 500$
- next is never called when no element remains

Test Cases

Test 1 — Basic

```
Input:
2
2 1 2
1 3
4
next
next
next
hasNext

Output:
1
2
3
false
```

Test 2 — Single element

```
Input:
1
1 9
3
hasNext
next
hasNext

Output:
true
9
false
```

Test 3 — Empty rows are skipped

```
Input:
4
0
2 4 5
0
1 6
5
next
next
hasNext
next
hasNext

Output:
4
5
true
6
false
```

Test 4 — Only empty rows

```
Input:
2
0
0
```



```
1
hasNext

Output:
false
```

Test 5 — hasNext called repeatedly

```
Input:
1
2 7 8
4
hasNext
hasNext
next
hasNext

Output:
true
true
7
true
```

Test 6 — Negative values

```
Input:
2
2 -1 -2
1 0
3
next
next
next

Output:
-1
-2
0
```

126. Intersection of Three Sorted Arrays

Topic: Arrays

Problem Statement

You are given three arrays that are each sorted in strictly increasing order. Return a sorted list of the integers that appear in all three arrays, and print it on one line separated by spaces. If no integer appears in all three arrays, print []. The arrays are already sorted, so instead of building hash sets you can walk through them with three pointers: when all three current values are equal, record the value and advance all three pointers; otherwise advance the pointer or pointers that point to the smallest value. This uses constant extra space and takes time proportional to the total number of elements.

Input / Output format: Line 1: n1. Line 2: array 1. Line 3: n2. Line 4: array 2. Line 5: n3. Line 6: array 3. Output: the common elements in increasing order ([] if none).

Example

Input

```
5
1 2 3 4 5
5
1 2 5 7 9
5
1 3 4 5 8
```

Output

```
1 5
```

Explanation: Only 1 and 5 occur in all three arrays: 1 5.

Constraints

- $1 \leq n_1, n_2, n_3 \leq 1000$
- $1 \leq \text{values} \leq 2000$
- Each array is strictly increasing

Test Cases

Test 1 — Basic

```
Input:
4
2 4 6 8
4
4 8 10 12
4
1 4 8 9

Output:
4 8
```

Test 2 — Single-element arrays that match

```
Input:
1
5
1
5
1
5

Output:
5
```

Test 3 — No common element

```
Input:
```

```
3
1 2 3
3
4 5 6
3
7 8 9
```

```
Output:
[]
```

Test 4 — Common to two arrays only

```
Input:
3
1 2 3
3
2 3 4
3
5 6 7
```

```
Output:
[]
```

Test 5 — Identical arrays

```
Input:
3
1 2 3
3
1 2 3
3
1 2 3
```

```
Output:
1 2 3
```

Test 6 — Different lengths

```
Input:
2
3 9
5
```

```
1 3 5 7 9
3
3 4 9

Output:
3 9
```

127. Line Reflection

Topic: Hashing

Problem Statement

Given a set of points on a two dimensional plane, determine whether there exists a vertical line, parallel to the y-axis, that reflects the set of points onto itself. Reflecting each point across the line must produce a point that is also in the set, so the set has to be symmetric about that line. If such a line exists it must pass through the midpoint of the smallest and the largest x coordinate, so its position is $(\min X + \max X) / 2$. Insert all points in a hash set and check that for every point (x, y) the mirror point $(\min X + \max X - x, y)$ exists. Duplicate points can be ignored. Print true or false.

Input / Output format: Line 1: n. Next n lines: 'x y'. Output: true or false.

Example

Input

```
2
1 1
-1 1
```

Output

```
true
```

Explanation: The line $x = 0$ reflects one point onto the other: true.

Constraints

- $1 \leq n \leq 10^4$
- $-10^8 \leq x, y \leq 10^8$

Test Cases

Test 1 — Not symmetric

```
Input:
2
1 1
-1 -1

Output:
false
```

Test 2 — Single point

```
Input:
1
5 5

Output:
true
```

Test 3 — Points on the line itself

```
Input:
```

```
3  
0 0  
0 1  
0 2
```

```
Output:
```

```
true
```

Test 4 — Symmetric with duplicates

```
Input:
```

```
4  
1 1  
-1 1  
1 1  
0 3
```

```
Output:
```

```
true
```

Test 5 — Different y for the mirror

```
Input:
```

```
3  
0 0  
4 0  
2 5
```

```
Output:
```

```
true
```

Test 6 — Multiple rows symmetric

```
Input:
```

```
4  
0 0  
4 0
```

```
1 1
3 1

Output:
true
```

128. Match Alphanumerical Pattern in Matrix – I

Topic: Hashing

Problem Statement

You are given a two dimensional integer matrix board containing digits and a smaller pattern matrix whose cells are either digits or lowercase letters. A submatrix of board matches the pattern if the same-sized cells can be paired so that every digit in the pattern equals the corresponding board value, and every letter stands for a single digit: the same letter always corresponds to the same digit, and different letters correspond to different digits. Find the top-left corner of the first matching submatrix in row-major order and print its row and column. If no submatrix matches, print -1 -1. Try every position and validate with two hash maps, letter to digit and digit to letter.

Input / Output format: Line 1: m and n. Next m lines: the board. Next line: p and q. Next p lines: the pattern (q tokens each; a token is a digit or a lowercase letter). Output: 'row col' of the first match, or -1 -1.

Example

Input

```
3 3
1 2 2
2 2 3
2 3 3
2 2
a b
```



```
b b
```

Output

```
0 0
```

Explanation: At (0, 0) the block $[[1,2],[2,2]]$ fits the pattern with $a = 1$ and $b = 2$, and the two letters map to different digits. Output: 0 0.

Constraints

- $1 \leq p \leq m \leq 50$
- $1 \leq q \leq n \leq 50$
- $0 \leq \text{board}[i][j] \leq 9$

Test Cases

Test 1 — Basic

```
Input:
2 3
1 3 1
3 1 3
2 2
a b
b a
```

```
Output:
0 0
```

Test 2 — Single cell pattern

```
Input:
```

```
1 1
```

```
7
```

```
1 1
```

```
a
```

```
Output:
```

```
0 0
```

Test 3 — Digits must match exactly

```
Input:
```

```
2 3
```

```
1 2 3
```

```
4 5 6
```

```
2 2
```

```
2 3
```

```
a 3
```

```
Output:
```

```
-1 -1
```

Test 4 — Different letters need different digits

```
Input:
```

```
2 2
```

```
5 5
```

```
5 5
```

```
1 2
```

```
a b
```

```
Output:
```

```
-1 -1
```

Test 5 — Same letter needs the same digit

```
Input:
```

```
2 2
```

```
1 2
```

```
3 4
```

```
1 2
a a

Output:
-1 -1
```

Test 6 — No match at all

```
Input:
3 3
1 2 3
4 5 6
7 8 9
2 2
1 1
1 1

Output:
-1 -1
```

129. Minimum Unlocked Indices to Sort Nums

Topic: Hashing

⚠ **Note:** Corresponds to LeetCode 3431; statement and expected outputs follow the official definition.

Problem Statement

You are given an array `nums` whose values are only 1, 2 or 3, and a binary array `locked` of the same length. The array is sortable if it can be sorted in non-decreasing order using adjacent swaps, where swapping positions i and $i + 1$ is allowed only when $\text{nums}[i] - \text{nums}[i + 1] == 1$ and $\text{locked}[i] == 0$. In one operation you may unlock any index i , setting $\text{locked}[i]$ to 0. Find the minimum number of operations needed to make `nums` sortable, or -1 if it is impossible. Track the first positions of 2 and 3 and the last positions of 1 and 2, and count the locked indices that lie in the ranges that must be unlocked.

Input / Output format: Line 1: n. Line 2: nums. Line 3: locked. Output: the minimum number of unlock operations, or -1.

Example

Input

```
7
1 2 1 1 3 2 2
1 0 1 1 0 1 0
```

Output

```
2
```

Explanation: Unlocking indices 2 and 5 makes the array sortable: 2.

Constraints

- $1 \leq n \leq 10^5$
- $1 \leq \text{nums}[i] \leq 3$
- $\text{locked}[i] \in \{0, 1\}$

Test Cases

Test 1 — Already sortable

```
Input:
6
```

```
1 1 2 2 3 3
1 1 1 1 1 1
```

```
Output:
0
```

Test 2 — Single element

```
Input:
1
2
1
```

```
Output:
0
```

Test 3 — Impossible: a 3 comes before a 1

```
Input:
7
1 2 1 2 3 2 1
0 0 0 0 0 0 0
```

```
Output:
-1
```

Test 4 — Already sorted, needs nothing

```
Input:
5
1 2 2 3 3
1 1 1 1 1
```

```
Output:
0
```

Test 5 — Needs several unlocks

```
Input:
6
2 1 1 3 2 2
1 1 1 1 1 1

Output:
4
```

Test 6 — Already unlocked

```
Input:
5
2 1 3 2 2
0 0 0 0 0

Output:
0
```

130. Remove Duplicates from an Unsorted Linked List

Topic: Hashing

Problem Statement

You are given the head of an unsorted singly linked list. Remove every node whose value has already appeared earlier in the list, so that only the first occurrence of each value remains, and print the resulting list from head to tail. The relative order of the surviving nodes must not change. Traverse the list once while storing the values seen so far in a hash set; keep a pointer to the previous surviving node, and when the current node's value is already in the set, unlink it by connecting the previous node to the next one. This takes linear time and linear extra space. Without extra space, a quadratic nested traversal is also possible.

Input / Output format: Line 1: n. Line 2: n integers (list values). Output: the list after removing duplicates.

Example

Input

```
6
12 11 12 21 41 43
```

Output

```
12 11 21 41 43
```

Explanation: The second 12 is removed: 12 11 21 41 43.

Constraints

- $1 \leq n \leq 10^5$
- $-10^9 \leq \text{value} \leq 10^9$

Test Cases

Test 1 — Basic

```
Input:
7
10 12 11 11 12 11 10

Output:
```

```
10 12 11
```

Test 2 — Single node

```
Input:
1
5

Output:
5
```

Test 3 — All identical

```
Input:
5
3 3 3 3 3

Output:
3
```

Test 4 — All distinct

```
Input:
4
1 2 3 4

Output:
1 2 3 4
```

Test 5 — Duplicate at the end

```
Input:
4
1 2 3 1
```



```
Output:  
1 2 3
```

Test 6 — Negative values

```
Input:  
5  
-1 -1 0 -2 0  
  
Output:  
-1 0 -2
```

131. Remove Duplicates from an Unsorted Array

Topic: Hashing

Problem Statement

Given an unsorted array of integers, remove the duplicates so that only the first occurrence of every value is kept, and print the remaining elements in their original relative order. For example, the array 4 5 4 6 5 becomes 4 5 6. Do not sort the array, because the order of first appearance has to be preserved. Iterate through the array once and maintain a hash set of the values that have already been output; append a value to the result only if it is not in the set yet, then insert it. The algorithm has linear expected time and linear extra space, which is better than the quadratic nested loop solution that compares every pair.

Input / Output format: Line 1: n. Line 2: n integers. Output: the array without duplicates, in order of first appearance.

Example

Input

```
5
4 5 4 6 5
```

Output

```
4 5 6
```

Explanation: The second 4 and the second 5 are dropped: 4 5 6.

Constraints

- $1 \leq n \leq 10^5$
- $-10^9 \leq a[i] \leq 10^9$

Test Cases

Test 1 — Basic

```
Input:
8
1 2 3 2 4 1 5 3

Output:
1 2 3 4 5
```

Test 2 — Single element

```
Input:
1
```

```
9
```

```
Output:  
9
```

Test 3 — All identical

```
Input:  
5  
7 7 7 7 7
```

```
Output:  
7
```

Test 4 — All distinct

```
Input:  
4  
4 3 2 1
```

```
Output:  
4 3 2 1
```

Test 5 — Negative values and zero

```
Input:  
6  
0 -1 0 -1 5 -5
```

```
Output:  
0 -1 5 -5
```

Test 6 — Duplicates only at the end

```
Input:
```

```
5
1 2 3 3 3

Output:
1 2 3
```

132. Analyze User Website Visit Pattern

Topic: Sorting

Problem Statement

You are given a log of website visits. Every record has a username, a timestamp and a website. A pattern is a sequence of three websites, repetition allowed, that a user visited in that order, not necessarily consecutively, when the user's visits are sorted by timestamp. The score of a pattern is the number of distinct users who followed it. Find the pattern with the highest score. If several patterns share the highest score, return the lexicographically smallest one, comparing the three websites in order. Group the visits by user, sort each group by time, enumerate every triple, collect each user's distinct triples in a set, count them globally and pick the best. Every user has at least three visits.

Input / Output format: Line 1: n. Next n lines: 'username timestamp website'. Output: the three websites of the best pattern, space separated.

Example

Input

```
10
joe 1 home
joe 2 about
joe 3 career
james 4 home
james 5 cart
james 6 maps
```

```
james 7 home
mary 8 home
mary 9 about
mary 10 career
```

Output

```
home about career
```

Explanation: joe and mary both visit home, about, career in that order (score 2). Every pattern of james has score 1. Output: home about career.

Constraints

- $3 \leq n \leq 50$
- $1 \leq \text{timestamp} \leq 10^9$, all timestamps distinct
- Every user has at least 3 visits
- Usernames and websites are lowercase strings of length 1–10

Test Cases

Test 1 — Basic, two users share a pattern

```
Input:
6
u1 1 a
u1 2 b
u1 3 c
u2 4 a
u2 5 b
u2 6 c

Output:
a b c
```

Test 2 — Single user, exactly three visits

```
Input:
3
z 5 x
z 6 y
z 7 w

Output:
x y w
```

Test 3 — Records not sorted by timestamp

```
Input:
6
b 30 c
a 10 x
b 10 a
a 30 z
b 20 b
a 20 y

Output:
a b c
```

Test 4 — Tie broken lexicographically

```
Input:
6
p 1 b
p 2 b
p 3 b
q 4 a
q 5 a
q 6 a

Output:
a a a
```

Test 5 — Repeated website in the pattern

```
Input:
5
k 1 s
k 2 s
k 3 s
k 4 s
k 5 s

Output:
s s s
```

Test 6 — Same user must count once per pattern

```
Input:
7
m 1 a
m 2 b
m 3 c
m 4 a
m 5 b
m 6 c
n 7 x

Output:
a a b
```

133. Before and After Puzzle

Topic: Sorting

Problem Statement

Given a list of phrases, generate every Before and After puzzle. A phrase is a string of lowercase words separated by single spaces with no leading or trailing space. A puzzle is formed by merging two different phrases, taken from different positions in the list, where the last word of the first phrase equals the first word of the second phrase; the common word appears only once in the merged phrase. Note that phrases[i] followed by phrases[j] and

phrases[j] followed by phrases[i] are different attempts and both may be valid. Return all distinct puzzles sorted lexicographically, one per line, or [] if none exist. Compare every ordered pair, merge when the words match, insert into a set, then sort.

Input / Output format: Line 1: n. Next n lines: one phrase each. Output: distinct puzzles sorted lexicographically, one per line, or [].

Example

Input

```
4
writing code
code rocks
mission statement
a quick bite to eat
```

Output

```
writing code rocks
```

Explanation: Only 'writing code' and 'code rocks' share the word code, so the puzzle is 'writing code rocks'.

Constraints

- $1 \leq n \leq 100$
- $1 \leq \text{words per phrase} \leq 5$
- Each word has 1–10 lowercase letters

Test Cases

Test 1 — Several puzzles, sorted

```
Input:
4
writing code
code rocks
rocks and roll
roll over

Output:
code rocks and roll
rocks and roll over
writing code rocks
```

Test 2 — No possible puzzle

```
Input:
3
abc def
ghi jkl
mno pqr

Output:
[]
```

Test 3 — Single phrase only

```
Input:
1
hello world

Output:
[]
```

Test 4 — Single-word phrases merge to themselves

```
Input:
2
a
```

```
a

Output:
a
```

Test 5 — Duplicate puzzles are reported once

```
Input:
3
x y
y z
y z

Output:
x y z
```

Test 6 — Chain in both directions

```
Input:
2
sun moon
moon sun

Output:
moon sun moon
sun moon sun
```

134. Best Meeting Point

Topic: Sorting

Problem Statement

You are given an $m \times n$ binary grid in which every 1 marks the home of one friend. The friends want to meet at a single point of the grid, which may be any cell. The travel distance of a friend is the Manhattan distance $|x_1 - x_2| + |y_1 - y_2|$ between the meeting point and the friend's home. Find the minimum possible total travel distance of all friends. Because the row and column distances are independent, collect the row indices and the column indices of all friends, sort both lists, take the median of each as the meeting coordinates, and add up the distances to those medians. There are at least two friends.

Input / Output format: Line 1: m and n . Next m lines: n values of 0/1 each. Output: the minimum total travel distance.

Example

Input

```
3 5
1 0 0 0 1
0 0 0 0 0
0 0 1 0 0
```

Output

```
6
```

Explanation: Meeting at (0, 2): $2 + 2 + 2 = 6$. No other cell gives a smaller sum.

Constraints

- $1 \leq m, n \leq 200$
- $\text{grid}[i][j] \in \{0, 1\}$
- At least two 1s in the grid

Test Cases

Test 1 — Two friends far apart

```
Input:
1 5
1 0 0 0 1

Output:
4
```

Test 2 — Two adjacent friends

```
Input:
2 1
1
1

Output:
1
```

Test 3 — Full 2x2 grid

```
Input:
2 2
1 1
1 1

Output:
4
```

Test 4 — Single column

```
Input:
5 1
1
0
1
```

```
0
1

Output:
4
```

Test 5 — Friends in corners of a 3x3 grid

```
Input:
3 3
1 0 1
0 0 0
1 0 1

Output:
8
```

Test 6 — Even number of friends with many medians

```
Input:
1 6
1 1 0 0 1 1

Output:
8
```

135. Best Time to Buy and Sell Stock with Cooldown

Topic: Dynamic Programming

Problem Statement

You are given an array `prices` where `prices[i]` is the price of a stock on day `i`. You may complete as many transactions as you like, buying and selling one share at a time, but you cannot hold more than one share at once and you must sell before you buy again. After you sell the stock you cannot buy on the next day, because a one-day cooldown is enforced. Find the maximum profit you can achieve. Use a three-state dynamic programme over the days: holding a share, having just sold, and resting with no share. Update the states in constant time per day, so the whole algorithm runs in linear time with constant extra space.

Input / Output format: Line 1: `n`. Line 2: `n` prices. Output: the maximum profit.

Example

Input

```
5
1 2 3 0 2
```

Output

```
3
```

Explanation: Buy on day 0, sell on day 1, cooldown on day 2, buy on day 3, sell on day 4. Profit = $1 + 2 = 3$.

Constraints

- $1 \leq n \leq 5000$
- $0 \leq \text{prices}[i] \leq 1000$

Test Cases

Test 1 — Basic

```
Input:
6
1 2 4 3 0 5

Output:
8
```

Test 2 — Single day

```
Input:
1
7

Output:
0
```

Test 3 — Decreasing prices

```
Input:
5
9 7 5 3 1

Output:
0
```

Test 4 — Increasing prices

```
Input:
5
1 2 3 4 5

Output:
4
```

Test 5 — Constant prices

```
Input:
4
4 4 4 4

Output:
0
```

Test 6 — Cooldown blocks the tempting trade

```
Input:
5
1 4 2 7 5

Output:
6
```

136. Better Compression of String

Topic: Sorting

Problem Statement

You are given a compressed string made of lowercase letters, each immediately followed by a positive integer that gives its frequency, for example `a3c9b2c1`. The same letter can appear several times in different groups. Produce a better compression of the string in which every distinct letter appears exactly once, the frequencies of its groups are added together, and the letters are arranged in alphabetical order. For `a3c9b2c1` the answer is `a3b2c10`, because `c` occurs $9 + 1 = 10$ times. Parse the string into letter and number pairs, accumulate the totals in a map, then sort the keys and print each letter followed by its total. Numbers can have several digits.

Input / Output format: Line 1: the compressed string. Output: the better compression, letters in alphabetical order.

Example

Input

```
a3c9b2c1
```

Output

```
a3b2c10
```

Explanation: a occurs 3 times, b 2 times and c $9 + 1 = 10$ times. Output: a3b2c10.

Constraints

- $1 \leq |s| \leq 10^5$
- s consists of lowercase letters each followed by a number
- $1 \leq \text{each frequency} \leq 10^4$

Test Cases

Test 1 — Basic

```
Input:  
c2a1b3a2
```

```
Output:
```

```
a3b3c2
```

Test 2 — Single group

```
Input:  
z12
```

```
Output:  
z12
```

Test 3 — Same letter repeated

```
Input:  
a1a1a1a1
```

```
Output:  
a4
```

Test 4 — Already alphabetical

```
Input:  
a1b2c3
```

```
Output:  
a1b2c3
```

Test 5 — Multi-digit counts

```
Input:  
x100y200x50
```

```
Output:  
x150y200
```

Test 6 — Reverse alphabetical order

```
Input:
z1y2x3

Output:
x3y2z1
```

137. Bold Words in String

Topic: Tries

Problem Statement

You are given a string s and a list of keywords. Add a pair of bold tags `<b>` and `</b>` around every substring of s that matches a keyword. If two matched substrings overlap, or if they are adjacent with no character between them, they must be merged into a single bold section so that the result uses as few tags as possible. Return the marked string. A simple approach marks a boolean array: for each keyword find all of its occurrences and mark those positions. Then scan the array and open a tag where a marked run begins and close it where the run ends. A Trie or Aho-Corasick automaton makes the matching faster for large keyword lists.

Input / Output format: Line 1: the string s . Line 2: the keywords separated by spaces.
Output: s with the bold tags inserted.

Example

Input

```
aabcd
```

```
ab cd
```

Output

```
a<b>abcd</b>
```

Explanation: ab matches at index 1 and cd at index 3; they are adjacent, so they merge.
Output: aabcd.

Constraints

- $1 \leq |s| \leq 1000$
- $1 \leq \text{number of keywords} \leq 50$
- $1 \leq |\text{keyword}| \leq 10$
- s and keywords are lowercase letters

Test Cases

Test 1 — Overlapping keywords

```
Input:
abcxyz123
abc 123

Output:
<b>abc</b>xyz<b>123</b>
```

Test 2 — No keyword occurs

```
Input:
hello
```

```
xyz
```

```
Output:  
hello
```

Test 3 — Whole string bold

```
Input:  
abcabc  
abc
```

```
Output:  
<b>abcabc</b>
```

Test 4 — Overlap merges into one section

```
Input:  
aaabbcc  
aaa aab bc
```

```
Output:  
<b>aaabbcc</b>c
```

Test 5 — Repeated occurrences

```
Input:  
abab  
ab
```

```
Output:  
<b>abab</b>
```

Test 6 — Keyword at the very end

```
Input:
```

```
xyzabc
abc

Output:
xyz<b>abc</b>
```

138. Campus Bikes

Topic: Sorting

Problem Statement

A campus is modelled as a 2D grid with n workers and m bikes, where $n \leq m$. Each worker must be assigned exactly one distinct bike. Repeatedly choose the unassigned worker and unassigned bike pair with the smallest Manhattan distance and assign them. If several pairs have the same distance, choose the one with the smallest worker index; if there is still a tie, choose the pair with the smallest bike index. Continue until every worker has a bike, and return the bike index assigned to each worker. Build all worker and bike pairs with their distances, sort them by distance, worker index and bike index, and scan them greedily, skipping pairs whose worker or bike is already taken.

Input / Output format: Line 1: n and m . Next n lines: worker coordinates 'x y'. Next m lines: bike coordinates 'x y'. Output: n integers, the bike index for every worker.

Example

Input

```
2 2
0 0
2 1
1 2
3 3
```

Output

```
1 0
```

Explanation: Distances: $w_0-b_0 = 3$, $w_0-b_1 = 6$, $w_1-b_0 = 2$, $w_1-b_1 = 3$. The smallest is w_1-b_0 (2), so worker 1 gets bike 0. Worker 0 then takes the remaining bike 1. Output: 1 0.

Constraints

- $1 \leq n \leq m \leq 1000$
- $0 \leq x, y < 1000$
- All coordinates of workers and bikes are distinct

Test Cases

Test 1 — Basic

```
Input:
3 3
0 0
1 1
2 0
1 0
2 2
2 1

Output:
0 2 1
```

Test 2 — One worker one bike

```
Input:
1 1
```

```
5 5
0 0

Output:
0
```

Test 3 — More bikes than workers

```
Input:
2 4
0 0
5 5
9 9
4 4
1 1
6 6

Output:
2 1
```

Test 4 — Distance ties resolved by worker index

```
Input:
2 2
0 0
2 0
1 0
3 0

Output:
0 1
```

Test 5 — Distance ties resolved by bike index

```
Input:
1 2
0 0
0 1
1 0

Output:
```


0

Test 6 — Workers stacked in a line

```
Input:
3 3
0 0
0 1
0 2
0 3
0 4
0 5

Output:
2 1 0
```

139. Capitalize First and Last Character of Each Word

Topic: Strings

Problem Statement

Given a sentence made of words separated by single spaces, convert the first and the last character of every word to uppercase and leave all the remaining characters unchanged. A word that has only one character is simply converted to uppercase once, since its first and last character are the same. Characters that are already uppercase stay uppercase. The number of spaces and the order of the words must stay exactly as in the input. Traverse the sentence word by word, either by splitting it on spaces or by scanning with a pointer, rebuild each word with the changed end characters, and join the words back together with single spaces.

Input / Output format: Line 1: the sentence. Output: the transformed sentence.

Example

Input

```
hello world
```

Output

```
Hello World
```

Explanation: hello becomes Hello and world becomes World. Output: Hello World.

Constraints

- $1 \leq |s| \leq 10^5$
- s contains letters and single spaces between words
- No leading or trailing spaces

Test Cases

Test 1 — Several words

```
Input:  
the quick brown fox
```

```
Output:  
ThE QuicK BrowN FoX
```

Test 2 — Single word

```
Input:
programming

Output:
ProgramminG
```

Test 3 — Single-character words

```
Input:
a b c

Output:
A B C
```

Test 4 — Two-letter words

```
Input:
go to it

Output:
GO TO IT
```

Test 5 — Already uppercase input

```
Input:
HELLO WORLD

Output:
HELLO WORLD
```

Test 6 — Mixed with digits

```
Input:
room 101 is open
```

```
Output:  
RoOm 101 IS OpEn
```

140. Change Case of Each Character in a String

Topic: Strings

Problem Statement

Given a string, change the case of every alphabetic character: each lowercase letter must be converted to uppercase and each uppercase letter must be converted to lowercase. Digits, spaces and punctuation marks are not letters and must be copied to the output without any change. The length of the string does not change. You can solve it with the ASCII values by toggling the sixth bit, that is, adding or subtracting 32 from the code of a letter, or by using the built-in swapcase style function of the language. Scan the string once from left to right and build the answer character by character, which takes linear time and linear space for the output string.

Input / Output format: Line 1: the string. Output: the string with the case of every letter toggled.

Example

Input

```
Hello World 123
```

Output

```
hELLO wORLD 123
```

Explanation: $H \rightarrow h$, $e \rightarrow E$, $l \rightarrow L$, $l \rightarrow L$, $o \rightarrow O$, $W \rightarrow w$, $o \rightarrow O$, $r \rightarrow R$, $l \rightarrow L$, $d \rightarrow D$; the digits stay. Output: hELLO wORLD 123.

Constraints

- $1 \leq |s| \leq 10^5$
- s has printable ASCII characters and no leading or trailing spaces

Test Cases

Test 1 — Mixed case

```
Input:
GeeksForGeeks

Output:
gEEKSfORgEEKS
```

Test 2 — All lowercase

```
Input:
programming

Output:
PROGRAMMING
```

Test 3 — All uppercase

```
Input:
```

```
CAPGEMINI
```

```
Output:  
capgemini
```

Test 4 — Digits and symbols only

```
Input:  
2024-#$$%
```

```
Output:  
2024-#$$%
```

Test 5 — Single letter

```
Input:  
q
```

```
Output:  
Q
```

Test 6 — Sentence with punctuation

```
Input:  
It's 9 AM, Go!
```

```
Output:  
iT'S 9 am, gO!
```

141. Change Every Letter with the Next Lexicographic

Topic: Strings

Problem Statement

Given a string, replace every letter by the letter that comes immediately after it in the alphabet. The alphabet is circular, so the letter z is replaced by a and the letter Z is replaced by A. The case of every letter must be preserved: lowercase letters stay lowercase and uppercase letters stay uppercase. Any character that is not a letter, such as a digit, a space or a punctuation mark, must remain unchanged. For example, the string abcz becomes bcda. Process the string from left to right, apply the rule to each character using its ASCII value and modular arithmetic, and append the result to the output. The time complexity is linear in the length of the string.

Input / Output format: Line 1: the string. Output: the transformed string.

Example

Input

```
abcz
```

Output

```
bcda
```

Explanation: $a \rightarrow b$, $b \rightarrow c$, $c \rightarrow d$, $z \rightarrow a$. Output: bcda.

Constraints

- $1 \leq |s| \leq 10^5$
- s has printable ASCII characters and no leading or trailing spaces

Test Cases

Test 1 — Mixed case

```
Input:
Hello World

Output:
Ifmmp Xpsme
```

Test 2 — Wrap-around of z and Z

```
Input:
zZ

Output:
aA
```

Test 3 — Digits and symbols unchanged

```
Input:
a1b2-c3!

Output:
b1c2-d3!
```

Test 4 — Only symbols

```
Input:
123 456

Output:
```



```
123 456
```

Test 5 — Whole alphabet

```
Input:
abcdefghijklmnopqrstuvwxyz

Output:
bcdefghijklmnopqrstuvwxyzabc
```

Test 6 — Single letter

```
Input:
y

Output:
z
```

142. Check if String Is Decomposable Into Value-Equal Substrings

Topic: Strings

Problem Statement

A value-equal string is a string in which all characters are the same. You are given a digit string s . Decide whether s can be split into consecutive substrings so that every substring is value-equal, exactly one substring has length 2, and all the other substrings have length 3. Return true if such a decomposition exists, otherwise false. Split the string into maximal runs of equal characters and inspect the length of each run. A run whose length modulo 3 equals 1

makes the decomposition impossible. A run whose length modulo 3 equals 2 must contribute the one substring of length 2, so exactly one such run may exist in the whole string.

Input / Output format: Line 1: the digit string s . Output: true or false.

Example

Input

```
00011111222
```

Output

```
true
```

Explanation: The runs are 000, 11111 and 222. Split them as 000, 111, 11, 222: exactly one piece of length 2. Output: true.

Constraints

- $2 \leq |s| \leq 1000$
- s consists of digits '0'-'9'

Test Cases

Test 1 — All runs are multiples of 3

```
Input :  
000111000
```

```
Output:  
false
```

Test 2 — Exactly one length-2 run

```
Input:  
77  
  
Output:  
true
```

Test 3 — Run of length 4 cannot be split

```
Input:  
1111  
  
Output:  
false
```

Test 4 — Two runs with remainder 2

```
Input:  
0011  
  
Output:  
false
```

Test 5 — Long run of remainder 2

```
Input:  
888888888888  
  
Output:
```

```
true
```

Test 6 — Run of 5 plus run of 2 (two pieces of length 2)

```
Input:
1111100

Output:
false
```

143. Concatenate One String to Another

Topic: Strings

Problem Statement

You are given two strings. Append the second string at the end of the first string and print the resulting string. For example, if the first string is Cap and the second string is gemini, the answer is Capgemini. The characters of the first string must appear before the characters of the second string, in their original order, and no separator is inserted between them. Do not use a library function such as strcat if the interviewer asks for the manual method: create a result buffer, copy every character of the first string, then copy every character of the second string. The time complexity is $O(n + m)$ where n and m are the lengths of the two strings.

Input / Output format: Line 1: the first string. Line 2: the second string. Output: the concatenation.

Example

Input

```
Cap
gemini
```

Output

```
Capgemini
```

Explanation: Cap followed by gemini gives Capgemini.

Constraints

- $1 \leq |s1|, |s2| \leq 10^5$
- Strings have printable ASCII characters and no leading or trailing spaces

Test Cases

Test 1 — Basic

```
Input:
Hello
World

Output:
HelloWorld
```

Test 2 — Single characters

```
Input:
a
b
```

```
Output:  
ab
```

Test 3 — Words with internal spaces

```
Input:  
good morning  
to all  
  
Output:  
good morningto all
```

Test 4 — Digits

```
Input:  
123  
456  
  
Output:  
123456
```

Test 5 — Different lengths

```
Input:  
x  
abcdefghij  
  
Output:  
xabcdefghij
```

Test 6 — Symbols

```
Input:  
a#b  
$c%
```

```
Output :  
a#b$c%
```

144. Count Substrings with Only One Distinct Letter

Topic: Strings

Problem Statement

Given a string s , return the number of substrings that contain only one distinct letter. Substrings are counted by their position, so equal substrings that occur at different places are counted separately. For example, in the string `aaba` the valid substrings are `a`, `a`, `aa`, `b` and `a`, so the answer is 5. Do not generate all substrings. Instead, split the string into maximal groups of identical consecutive characters. A group of length k contributes k times k plus one divided by two substrings. Adding the contribution of every group gives the answer in a single linear pass over the string with constant extra space.

Input / Output format: Line 1: the string s . Output: the number of substrings with one distinct letter.

Example

Input

```
aaaba
```

Output

8

Explanation: Group aaa contributes $3 \cdot 4/2 = 6$, group b contributes 1, group a contributes 1. Output: 8.

Constraints

- $1 \leq |s| \leq 10^5$
- s consists of lowercase English letters

Test Cases

Test 1 — All letters equal

```
Input:
aaaaa

Output:
15
```

Test 2 — Single character

```
Input:
z

Output:
1
```

Test 3 — All different

```
Input:
```



```
abcde

Output:
5
```

```
abcde

Output:
5
```

Test 4 — Two long groups

```
Input:
aabbbb

Output:
13
```

```
Input:
aabbabb

Output:
13
```

Test 5 — Alternating

```
Input:
ababab

Output:
6
```

```
Input:
ababab

Output:
6
```

Test 6 — Large groups

```
Input:  
xxxxxxxxxxxxyyyyyyyyyy  
  
Output:  
110
```

```
Input:  
xxxxxxxxxxxxyyyyyyyyyy  
  
Output:  
110
```

145. Count the Number of Words in a Given String

Topic: Strings

Problem Statement

You are given a string that contains words separated by one or more space characters. A word is a maximal sequence of non-space characters. Count how many words the string contains. The string can contain several consecutive spaces between two words, so counting spaces and adding one gives a wrong answer. Scan the string with a flag that tells whether the previous character was a space. Every time you meet a non-space character right after a space, or at the very start of the string, a new word begins, so increase the counter. The scan needs one pass and constant extra space. Leading spaces are allowed, trailing spaces are not present in the input.

Input / Output format: Line 1: the string. Output: the number of words.

Example

Input

```
Welcome to  Capgemini
```

Output

```
3
```

Explanation: The words are Welcome, to and Capgemini; the double space does not create an extra word. Output: 3.

Constraints

- $1 \leq |s| \leq 10^5$

- s has printable ASCII characters
- s contains at least one non-space character

Test Cases

Test 1 — Simple sentence

```
Input:
the quick brown fox jumps

Output:
5
```

Test 2 — Single word

```
Input:
hello

Output:
1
```

Test 3 — Multiple spaces between words

```
Input:
a  b  c

Output:
3
```

Test 4 — Leading spaces

```
Input:
  leading spaces
```

```
Output:  
2
```

Test 5 — Words with punctuation

```
Input:  
Hi, how are you?  
  
Output:  
4
```

Test 6 — Single character

```
Input:  
x  
  
Output:  
1
```

146. Count Vowels, Consonants, and Spaces in a String

Topic: Strings

Problem Statement

Given a string, count the number of vowels, the number of consonants and the number of spaces it contains, and print the three counts separated by single spaces in that order. The vowels are a, e, i, o, u in both lowercase and uppercase. A consonant is any English letter that is not a vowel. Digits and punctuation marks are neither vowels, consonants nor spaces, so they are ignored. Traverse the string once. For each character, check whether it is a space, a

vowel or another letter, using the lowercase form of the character for comparison, and increment the corresponding counter. The algorithm runs in linear time and constant space.

Input / Output format: Line 1: the string. Output: 'vowels consonants spaces'.

Example

Input

```
Hello World 2024
```

Output

```
3 7 2
```

Explanation: Vowels: e, o, o = 3. Consonants: H, l, l, W, r, l, d = 7. Spaces: 2. Digits are ignored. Output: 3 7 2.

Constraints

- $1 \leq |s| \leq 10^5$
- s has printable ASCII characters

Test Cases

Test 1 — Basic

```
Input:
Programming is fun
```

```
Output:  
5 11 2
```

Test 2 — Only vowels

```
Input:  
AEIOUaeiou
```

```
Output:  
10 0 0
```

Test 3 — Only consonants

```
Input:  
bcd fg
```

```
Output:  
0 5 0
```

Test 4 — No letters

```
Input:  
1234 5678 !!
```

```
Output:  
0 0 2
```

Test 5 — Uppercase letters

```
Input:  
CAPGEMINI IS GREAT
```

```
Output:
```

```
7 9 2
```

Test 6 — Single space in the middle

```
Input:
a b

Output:
1 1 1
```

147. Count Pairs of Equal Substrings with Minimum Difference

Topic: Greedy Algorithms

Problem Statement

You are given two strings, `firstString` and `secondString`. Count the quadruples (i, j, a, b) such that $0 \leq i \leq j < |\text{firstString}|$, $0 \leq a \leq b < |\text{secondString}|$, the substring `firstString[i..j]` equals the substring `secondString[a..b]`, and the value $j - a$ is the minimum possible among all valid quadruples. Two quadruples differing in any index are different. To minimise $j - a$, the substring should be a single character, because a longer match can only push j up or a down. So for each common character take its first index in `firstString` and its last index in `secondString`, compute the difference, and count how many characters reach the minimum.

Input / Output format: Line 1: `firstString`. Line 2: `secondString`. Output: the number of quadruples (0 when no character is common).

Example

Input

```
abcd
dacb
```

Output

```
1
```

Explanation: Character b gives $1 - 3 = -2$, which is the smallest value (a gives -1, c gives 0, d gives 3). Only b reaches -2, so there is one quadruple. Output: 1.

Constraints

- $1 \leq |\text{firstString}|, |\text{secondString}| \leq 2 \cdot 10^5$
- Both strings consist of lowercase English letters

Test Cases

Test 1 — Basic

```
Input:
abc
cba

Output:
1
```

Test 2 — No common character

```
Input:
```



```
ab
cd

Output:
0
```

Test 3 — Single matching character

```
Input:
a
a

Output:
1
```

Test 4 — Repeated letters use first and last index

```
Input:
aabb
bbaa

Output:
1
```

Test 5 — Repeated letters, all characters tie

```
Input:
abcabc
abcabc

Output:
3
```

Test 6 — Every character gives difference 0

```
Input:
xyz
xyz

Output:
3
```

148. Find Pattern in Infinite Stream II

Topic: Strings

Problem Statement

You are given a binary array pattern and a stream of bits that can be read one bit at a time. Return the first index at which the pattern appears as a contiguous block in the stream. The stream is very long, so every bit must be read once and never revisited. KMP allows this: build the failure (LPS) array of the pattern, feed the bits into the automaton, and when the matched length equals the pattern length, the start index is the current position minus the pattern length plus one. Here the stream is a finite prefix; print -1 if it never matches.

Input / Output format: Line 1: n (stream length). Line 2: the stream bits. Line 3: m (pattern length). Line 4: the pattern bits. Output: the first start index, or -1.

Example

Input

```
9
1 1 1 0 1 1 1 0 1
2
0 1
```

Output

```
3
```

Explanation: The pattern 0 1 first occurs at indices 3 and 4. Output: 3.

Constraints

- $1 \leq m \leq 10^5$
- $m \leq n \leq 10^6$
- Every bit is 0 or 1

Test Cases

Test 1 — Pattern at the start

```
Input:
5
1 0 1 1 0
3
1 0 1

Output:
0
```

Test 2 — Pattern is one bit

```
Input:
4
1 1 1 0
1
0

Output:
```

```
3
```

Test 3 — Pattern never occurs

```
Input:
6
1 1 1 1 1 1
2
0 1
```

```
Output:
-1
```

Test 4 — Partial match then restart

```
Input:
9
1 1 0 1 1 0 1 1 1
6
1 1 0 1 1 1
```

```
Output:
3
```

Test 5 — Overlapping candidates

```
Input:
9
0 0 0 0 1 0 0 0 1
4
0 0 0 1
```

```
Output:
1
```

Test 6 — Pattern equals the whole stream

```
Input:
3
1 0 1
3
1 0 1

Output:
0
```

149. Find the ASCII Value of a Character

Topic: Strings

Problem Statement

Given a single character, print its ASCII value, that is, the integer code that represents the character in the ASCII table. For example, the uppercase letter A has the value 65, the lowercase letter a has the value 97, and the digit character 0 has the value 48. Note that the input is a character and not a number: the character 5 has the ASCII value 53. In C and C++ you can print the character with the integer format specifier or cast it to int; in Java you can cast the char to int; in Python you can call the built-in function ord. The answer is always in the range 33 to 126 for the printable characters used in the tests.

Input / Output format: Line 1: a single printable, non-space character. Output: its ASCII value.

Example

Input

```
A
```

Output

```
65
```

Explanation: The character A is stored as 65. Output: 65.

Constraints

- The input is exactly one printable ASCII character other than space

Test Cases

Test 1 — Lowercase letter

```
Input:
a

Output:
97
```

Test 2 — Uppercase letter

```
Input:
Z

Output:
90
```

Test 3 — Digit character

```
Input:
5
```

```
Output:
53
```

Test 4 — Symbol

```
Input:
#
```

```
Output:
35
```

Test 5 — Tilde, the highest printable code

```
Input:
~
```

```
Output:
126
```

Test 6 — Exclamation mark, the lowest printable code

```
Input:
!
```

```
Output:
33
```

150. Find the Largest Word in a String

Topic: Strings

Problem Statement

Given a sentence in which words are separated by one or more spaces, find and print the word with the largest number of characters. A word is a maximal sequence of non-space characters. If two or more words have the same maximum length, print the one that appears first in the sentence. Punctuation attached to a word counts as part of the word. Split the sentence into words, compare each length with the best seen so far, and update the answer only when the new word is strictly longer, which keeps the earliest word on a tie. The time complexity is linear.

Input / Output format: Line 1: the sentence. Output: the largest word (the first one on ties).

Example

Input

```
I love programming in Python
```

Output

```
programming
```

Explanation: programming has 11 characters, the longest of all. Output: programming.

Constraints

- $1 \leq |s| \leq 10^5$

- s contains at least one word

Test Cases

Test 1 — Tie: first longest word wins

```
Input:
the quick brown fox

Output:
quick
```

Test 2 — Single word

```
Input:
hello

Output:
hello
```

Test 3 — Extra spaces

```
Input:
a    bb   ccc  dd

Output:
ccc
```

Test 4 — All words of length one

```
Input:
a b c d

Output:
```

```
a
```

Test 5 — Punctuation is part of the word

```
Input:
wow! amazing, right?

Output:
amazing,
```

Test 6 — Longest word is last

```
Input:
go to school yesterday

Output:
yesterday
```

151. Flip Game

Topic: Strings

Problem Statement

You are playing a Flip Game with your friend. You are given a string `currentState` that contains only the characters `+` and `-`. In one move a player chooses two consecutive `+` characters and flips them into `--`. The game ends when a player can no longer make a move. Return all the possible states of the string after one valid move, listing them in the order of the position where the two flipped characters start, from left to right. If there is no valid move, return an empty list, printed as `[]`. Scan the string once, and whenever the characters at positions `i` and `i + 1` are both plus signs, create a copy of the string with those two positions replaced by minus signs.

Input / Output format: Line 1: currentState. Output: all states after one move, one per line, in left-to-right order of the flipped pair, or [].

Example

Input

```
++++
```

Output

```
--++  
+--+  
++--
```

Explanation: Flipping positions (0,1), (1,2) and (2,3) gives --++, +--+ and ++--. Output: those three states, one per line.

Constraints

- $1 \leq |\text{currentState}| \leq 500$
- currentState has only '+' and '-'

Test Cases

Test 1 — Mixed characters

```
Input:  
+-++-+++
```

```
Output:
+----+++
+-++---+
+-++-+--
```

Test 2 — No move possible

```
Input:
+-+-

Output:
[]
```

Test 3 — Single character

```
Input:
+

Output:
[]
```

Test 4 — Exactly one pair

```
Input:
--++--

Output:
-----
```

Test 5 — Only minus signs

```
Input:
-----
```

```
Output:  
[]
```

Test 6 — Three in a row

```
Input:  
+++  
  
Output:  
--+  
+--
```

152. Hexspeak

Topic: Strings

Problem Statement

A decimal number can be converted to its Hexspeak representation by first converting it to an uppercase hexadecimal string, then replacing every occurrence of the digit 0 with the letter O and every occurrence of the digit 1 with the letter I. Such a representation is valid only if it consists of the letters A, B, C, D, E, F, I and O. You are given a string num that represents a decimal integer N. Return the Hexspeak representation of N if it is valid, otherwise return ERROR. For example, 257 is 101 in hexadecimal, which becomes IOI. The number 3 is hexadecimal 3, which contains a digit that is not allowed, so the answer is ERROR.

Input / Output format: Line 1: the decimal integer N as a string. Output: the Hexspeak string or ERROR.

Example

Input

```
257
```

Output

```
IOI
```

Explanation: $257 = 0x101$; replacing 1 by I and 0 by O gives IOI. Output: IOI.

Constraints

- $1 \leq N \leq 10^{12}$
- N has no leading zeros

Test Cases

Test 1 — Invalid digit remains

```
Input:
3

Output:
ERROR
```

Test 2 — Only letters

```
Input:
171

Output:
```

```
AB
```

Test 3 — Letters and replaced digits

```
Input:  
2748
```

```
Output:  
ABC
```

Test 4 — Smallest valid value

```
Input:  
1
```

```
Output:  
I
```

Test 5 — Power of 16

```
Input:  
256
```

```
Output:  
IOO
```

Test 6 — Large number

```
Input:  
10000000000000
```

```
Output:  
ERROR
```

153. Number of Distinct Substrings in a String

Topic: Tries

Problem Statement

Given a string s , count the number of distinct non-empty substrings of s . Substrings that are equal as strings are counted once, even if they appear at several positions. For example, the string `abab` has the distinct substrings `a`, `b`, `ab`, `ba`, `aba`, `bab` and `abab`, so the answer is 7. A straightforward method inserts every suffix of the string into a Trie; every newly created node of the Trie represents a new distinct substring, so the number of nodes, excluding the root, is the answer. This takes $O(n^2)$ time and space. Suffix arrays with the LCP array, or a suffix automaton, solve the same problem in near linear time for longer strings.

Input / Output format: Line 1: the string s . Output: the number of distinct non-empty substrings.

Example

Input

```
abab
```

Output

```
7
```


Explanation: The distinct substrings are a, b, ab, ba, aba, bab and abab. Output: 7.

Constraints

- $1 \leq |s| \leq 3000$
- s consists of lowercase English letters

Test Cases

Test 1 — All characters equal

```
Input:
aaaa

Output:
4
```

Test 2 — Single character

```
Input:
z

Output:
1
```

Test 3 — All different characters

```
Input:
abcd

Output:
10
```

Test 4 — Repeated pattern

```
Input:
abcabc

Output:
15
```

Test 5 — Palindrome

```
Input:
racecar

Output:
25
```

Test 6 — Two letters

```
Input:
ab

Output:
3
```

154. Range Sum in BST

Topic: Binary Search Trees

Problem Statement

Given the root node of a binary search tree and two integers low and high, return the sum of the values of all the nodes whose value lies in the inclusive range [low, high]. The tree is a valid binary search tree, so the values of the left subtree of a node are smaller than the node and the values of the right subtree are larger. Use this ordering to prune the search: if the current value is smaller than low, only the right subtree can contain valid nodes; if it is larger than high, only the left subtree can. Otherwise add the value and explore both children. The tree is given in level-order with null for missing children.

Input / Output format: Line 1: the tree in level-order ('null' for missing children). Line 2: low and high. Output: the sum of the values in [low, high].

Example

Input

```
10 5 15 3 7 null 18
7 15
```

Output

```
32
```

Explanation: The nodes in [7, 15] are 7, 10 and 15. Their sum is 32.

Constraints

- $1 \leq \text{number of nodes} \leq 2 \cdot 10^4$
- $-10^5 \leq \text{node value} \leq 10^5$
- All node values are unique
- $\text{low} \leq \text{high}$

Test Cases

Test 1 — Range covers the whole tree

```
Input:
10 5 15 3 7 null 18
0 100

Output:
58
```

Test 2 — Single node inside

```
Input:
8
8 8

Output:
8
```

Test 3 — Single node outside

```
Input:
8
9 20

Output:
0
```

Test 4 — Range that contains no node

```
Input:
10 5 15 3 7 null 18
11 14

Output:
0
```

Test 5 — Negative values

```
Input:
-5 -10 0 null null -2 3
-6 0

Output:
-7
```

Test 6 — Range touches only leaves

```
Input:
10 5 15 3 7 null 18
3 3

Output:
3
```

155. Rotate a Linked List

Topic: Linked List

Problem Statement

Given the head of a singly linked list and an integer k , rotate the list to the right by k places, so that the last k nodes move to the front while the relative order inside both parts stays the same. The value of k can be larger than the length of the list, in which case rotating by k is the same as rotating by k modulo the length. For the list $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5$ and $k = 2$, the answer is $4 \rightarrow 5 \rightarrow 1 \rightarrow 2 \rightarrow 3$. Join the tail to the head to form a circle, walk length minus k steps to the new tail, and cut the circle after it.

Input / Output format: Line 1: n . Line 2: the list values. Line 3: k . Output: the rotated list.

Example

Input

```
5
1 2 3 4 5
2
```

Output

```
4 5 1 2 3
```

Explanation: The last two nodes 4 and 5 move to the front. Output: 4 5 1 2 3.

Constraints

- $1 \leq n \leq 5 \cdot 10^4$
- $-10^9 \leq \text{value} \leq 10^9$
- $0 \leq k \leq 2 \cdot 10^9$

Test Cases

Test 1 — k larger than the length

```
Input:
3
1 2 3
4

Output:
```

```
3 1 2
```

Test 2 — k equal to zero

```
Input:
3
1 2 3
0

Output:
1 2 3
```

Test 3 — k equal to the length

```
Input:
4
5 6 7 8
4

Output:
5 6 7 8
```

Test 4 — Single node

```
Input:
1
9
100

Output:
9
```

Test 5 — Rotate by one

```
Input:
```

```
4
1 2 3 4
1
```

```
Output:
4 1 2 3
```

Test 6 — Negative values

```
Input:
5
-1 0 -2 3 4
3
```

```
Output:
-2 3 4 -1 0
```

156. Delete a Node in BST

Topic: Binary Search Trees

Problem Statement

Given the root of a binary search tree and a key, delete the node with that key and return the root of the modified tree. If the key does not exist, the tree remains unchanged. There are three cases. A leaf node is simply removed. A node with one child is replaced by that child. A node with two children is replaced by its inorder successor, which is the smallest node of its right subtree: copy the successor's value into the node and then delete the successor from the right subtree. The tree is given and printed in level-order, with null for missing children and trailing nulls removed. An empty tree is printed as []. The expected complexity is $O(\text{height})$.

Input / Output format: Line 1: the tree in level-order. Line 2: the key. Output: the resulting tree in level-order ('null' for gaps, trailing nulls removed), or [] when empty.

Example

Input

```
5 3 6 2 4 null 7
3
```

Output

```
5 4 6 2 null null 7
```

Explanation: Node 3 has two children, so it takes the value of its successor 4 and the old 4 is removed. Output: 5 4 6 2 null null 7.

Constraints

- $0 \leq \text{number of nodes} \leq 10^4$
- All node values are unique
- The tree is a valid BST

Test Cases

Test 1 — Delete a leaf

```
Input:
5 3 6 2 4 null 7
7
```

```
Output:
5 3 6 2 4
```

Test 2 — Key not present

```
Input:
5 3 6 2 4 null 7
99
```

```
Output:
5 3 6 2 4 null 7
```

Test 3 — Delete the only node

```
Input:
1
1
```

```
Output:
[]
```

Test 4 — Delete the root with two children

```
Input:
5 3 6 2 4 null 7
5
```

```
Output:
6 3 7 2 4
```

Test 5 — Delete a node with only a left child

```
Input:
5 3 null 2
3
```

```
Output:
5 2
```

Test 6 — Delete the root with only a right child

```
Input:
2 null 3
2

Output:
3
```

157. Inorder Successor and Predecessor in BST

Topic: Binary Search Trees

Problem Statement

Given a binary search tree and a key, find the inorder predecessor and the inorder successor of the key. The predecessor is the node with the largest value strictly smaller than the key, and the successor is the node with the smallest value strictly greater than the key. The key does not have to exist in the tree. If there is no predecessor or no successor, print -1 for that position. Search from the root: when the current node is greater than the key, record it as a possible successor and go left; when it is smaller, record it as a possible predecessor and go right; when it is equal, the predecessor is the maximum of the left subtree and the successor is the minimum of the right subtree.

Input / Output format: Line 1: the BST in level-order ('null' for missing children). Line 2: the key. Output: 'predecessor successor'.

Example

Input

```
8 3 10 1 6 null 14 null null 4 7 13
8
```

Output

```
7 10
```

Explanation: The inorder sequence is 1 3 4 6 7 8 10 13 14. Around 8: 7 before and 10 after.
Output: 7 10.

Constraints

- $1 \leq \text{number of nodes} \leq 10^5$
- All node values are unique positive integers
- The tree is a valid BST

Test Cases

Test 1 — Key smaller than every node

```
Input:
8 3 10 1 6 null 14 null null 4 7 13
0

Output:
-1 1
```

Test 2 — Key larger than every node

```
Input:
8 3 10 1 6 null 14 null null 4 7 13
```

```
20
```

```
Output:  
14 -1
```

Test 3 — Key not in the tree

```
Input:  
8 3 10 1 6 null 14 null null 4 7 13  
5
```

```
Output:  
4 6
```

Test 4 — Single node with the key

```
Input:  
5  
5
```

```
Output:  
-1 -1
```

Test 5 — Key is the minimum

```
Input:  
8 3 10 1 6 null 14 null null 4 7 13  
1
```

```
Output:  
-1 3
```

Test 6 — Key is the maximum

```
Input:
```

```
8 3 10 1 6 null 14 null null 4 7 13
14

Output:
13 -1
```

158. Floor and Ceil in a BST

Topic: Binary Search Trees

Problem Statement

Given a binary search tree and an integer key, find the floor and the ceil of the key in the tree. The floor is the largest value in the tree that is less than or equal to the key, and the ceil is the smallest value in the tree that is greater than or equal to the key. If the floor or the ceil does not exist, print -1 in its place. If the key is present, both equal the key. Walk down from the root: a node smaller than the key is a floor candidate and you go right; a larger node is a ceil candidate and you go left.

Input / Output format: Line 1: the BST in level-order ('null' for missing children). Line 2: the key. Output: 'floor ceil'.

Example

Input

```
8 4 12 2 6 10 14
5
```

Output

```
4 6
```

Explanation: The values around 5 are 4 and 6. Output: 4 6.

Constraints

- $1 \leq \text{number of nodes} \leq 10^5$
- All node values are unique positive integers
- The tree is a valid BST

Test Cases

Test 1 — Key present in the tree

```
Input:
8 4 12 2 6 10 14
10

Output:
10 10
```

Test 2 — Key smaller than every node

```
Input:
8 4 12 2 6 10 14
1

Output:
-1 2
```

Test 3 — Key larger than every node

```
Input:
8 4 12 2 6 10 14
15
```

```
Output:
14 -1
```

Test 4 — Single node, larger key

```
Input:
5
7
```

```
Output:
5 -1
```

Test 5 — Single node, smaller key

```
Input:
5
3
```

```
Output:
-1 5
```

Test 6 — Key between root and right child

```
Input:
8 4 12 2 6 10 14
9
```

```
Output:
8 10
```

159. Count Number of Nice Subarrays

Topic: Mathematics

Problem Statement

You are given an array of integers `nums` and an integer `k`. A continuous subarray is called nice if it contains exactly `k` odd numbers. Return the number of nice subarrays. Each subarray is counted by its position, so equal subarrays at different places count separately. The parity of the numbers matters, not their size, so you can replace each element by 1 if it is odd and by 0 if it is even and then count the subarrays whose sum is exactly `k`. Keep a running count of odd numbers and a hash map that stores how many prefixes have each count; for every position add the number of earlier prefixes whose count equals current minus `k`. The algorithm runs in linear time.

Input / Output format: Line 1: `n` and `k`. Line 2: the array. Output: the number of nice subarrays.

Example

Input

```
5 3
1 1 2 1 1
```

Output

```
2
```

Explanation: The subarrays `[1,1,2,1]` and `[1,2,1,1]` contain exactly three odd numbers.
Output: 2.

Constraints

- $1 \leq n \leq 5 \cdot 10^4$
- $1 \leq \text{nums}[i] \leq 10^5$
- $1 \leq k \leq n$

Test Cases

Test 1 — All even numbers

```
Input:
5 1
2 4 6 8 10

Output:
0
```

Test 2 — All odd numbers, k = 1

```
Input:
4 1
1 3 5 7

Output:
4
```

Test 3 — Single element

```
Input:
1 1
3

Output:
1
```

Test 4 — Evens around one odd

```
Input:
7 1
2 2 2 1 2 2 2

Output:
16
```

Test 5 — k equal to n with all odd

```
Input:
3 3
1 1 1

Output:
1
```

Test 6 — Alternating parity

```
Input:
9 2
1 2 1 2 1 2 1 2 1

Output:
12
```

160. Stock Span Problem

Topic: Stack & Queues

Problem Statement

The stock span problem asks for the span of a stock's price on every day. You are given the prices of a stock for n consecutive days. The span of the price on day i is the maximum number of consecutive days, ending on day i and going backwards, for which the price was less than or equal to the price on day i ; the day i itself counts. Print the span for every day in order. A monotonic stack of indices with strictly greater prices gives a linear solution: pop while the top price is less than or equal to today's, then the span is the distance to the new top.

Input / Output format: Line 1: n . Line 2: the n prices. Output: the n spans, space separated.

Example

Input

```
7
100 80 60 70 60 75 85
```

Output

```
1 1 1 2 1 4 6
```

Explanation: Day 3 price 70 spans two days (70, 60); day 5 price 75 spans four days; day 6 price 85 spans six. Output: 1 1 1 2 1 4 6.

Constraints

- $1 \leq n \leq 10^5$
- $1 \leq \text{price} \leq 10^5$

Test Cases

Test 1 — Increasing prices

```
Input:
5
10 20 30 40 50

Output:
1 2 3 4 5
```

Test 2 — Decreasing prices

```
Input:
5
50 40 30 20 10

Output:
1 1 1 1 1
```

Test 3 — Single day

```
Input:
1
7

Output:
1
```

Test 4 — Equal prices count together

```
Input:
4
5 5 5 5

Output:
1 2 3 4
```

Test 5 — Zigzag

```
Input:
6
3 1 4 1 5 9

Output:
1 1 3 1 5 6
```

Test 6 — Peak in the middle

```
Input:
7
2 4 8 3 2 1 9

Output:
1 2 3 1 1 1 7
```

161. Sum of Subarray Ranges

Topic: Stack & Queues

Problem Statement

You are given an integer array `nums`. The range of a subarray is the difference between its largest element and its smallest element. Return the sum of the ranges of all the non-empty contiguous subarrays of `nums`. For the array `[1, 2, 3]` the subarrays `[1]`, `[2]`, `[3]` have range 0, the subarrays `[1, 2]` and `[2, 3]` have range 1, and the subarray `[1, 2, 3]` has range 2, so the sum is 4. A brute force method that maintains the running maximum and minimum for each starting index takes quadratic time. A monotonic stack can compute, for every element, the number of subarrays where it is the maximum and where it is the minimum, giving a linear time solution.

Input / Output format: Line 1: n. Line 2: the array. Output: the sum of all subarray ranges.

Example

Input

```
3
1 2 3
```

Output

```
4
```

Explanation: Ranges: [1]=0, [2]=0, [3]=0, [1,2]=1, [2,3]=1, [1,2,3]=2. Sum = 4.

Constraints

- $1 \leq n \leq 1000$
- $-10^9 \leq \text{nums}[i] \leq 10^9$

Test Cases

Test 1 — Basic with negatives

```
Input:
5
4 -2 -3 4 1

Output:
```

59

Test 2 — Single element

```
Input:
1
5

Output:
0
```

Test 3 — All equal

```
Input:
4
7 7 7 7

Output:
0
```

Test 4 — Descending

```
Input:
4
4 3 2 1

Output:
10
```

Test 5 — Repeated values

```
Input:
3
1 3 3
```



```
Output:  
4
```

Test 6 — Two elements

```
Input:  
2  
-5 5  
  
Output:  
10
```

162. Maximum Rectangles

Topic: Stack & Queues

Problem Statement

You are given an $m \times n$ binary matrix filled with 0s and 1s. Find the largest rectangle that contains only 1s and return its area. The rectangle must be axis aligned, that is, made of full rows and columns of the matrix. Treat every row as the base of a histogram: the height of a column is the number of consecutive 1s ending at that row going upward, and it resets to zero when a 0 is met. For each row, the largest rectangle in that histogram is found in linear time with a monotonic stack. The answer is the maximum over all rows. The overall time complexity is $O(m \cdot n)$, and the extra space is $O(n)$.

Input / Output format: Line 1: m and n . Next m lines: n values (0 or 1) separated by spaces. Output: the area of the largest all-1 rectangle.

Example

Input

```
4 5
1 0 1 0 0
1 0 1 1 1
1 1 1 1 1
1 0 0 1 0
```

Output

```
6
```

Explanation: The rectangle in rows 1-2 and columns 2-4 has $2 \times 3 = 6$ ones. Output: 6.

Constraints

- $1 \leq m, n \leq 200$
- $\text{matrix}[i][j] \in \{0, 1\}$

Test Cases

Test 1 — All zeros

```
Input:
2 3
0 0 0
0 0 0

Output:
0
```

Test 2 — All ones

```
Input:
```

```
3 3
```

```
1 1 1
```

```
1 1 1
```

```
1 1 1
```

```
Output:
```

```
9
```

Test 3 — Single cell

```
Input:
```

```
1 1
```

```
1
```

```
Output:
```

```
1
```

Test 4 — Single row

```
Input:
```

```
1 6
```

```
1 1 0 1 1 1
```

```
Output:
```

```
3
```

Test 5 — Single column

```
Input:
```

```
5 1
```

```
1
```

```
1
```

```
0
```

```
1
```

```
1
```

```
Output:
```

```
2
```

Test 6 — Staircase shape

```
Input:
4 4
1 0 0 0
1 1 0 0
1 1 1 0
1 1 1 1

Output:
6
```

163. Distinct Substrings

Topic: Tries

⚠ **Note:** This list entry counts the empty substring (Coding Ninjas convention); 'Number of Distinct Substrings in a String' counts only non-empty ones.

Problem Statement

Given a string S , count the distinct substrings of S , including the empty substring. Two substrings are the same if they consist of the same characters in the same order, no matter where they occur in S . For the string sds the distinct substrings are the empty string, s , d , sd , ds and sds , so the answer is 6. Note that this variant counts the empty substring, so its answer is one larger than the count of the non-empty distinct substrings. Insert every suffix of the string into a Trie and count the number of nodes, including the root, which represents the empty string. Each node stands for exactly one distinct substring, so the count of nodes is the answer.

Input / Output format: Line 1: the string S . Output: the number of distinct substrings including the empty one.

Example

Input

```
sds
```

Output

```
6
```

Explanation: The distinct substrings are "", s, d, sd, ds and sds. Output: 6.

Constraints

- $1 \leq |S| \leq 3000$
- S consists of lowercase English letters

Test Cases

Test 1 — All characters equal

```
Input:
aaa

Output:
4
```

Test 2 — Single character

```
Input:  
a
```

```
Output:  
2
```

Test 3 — All different

```
Input:  
abc
```

```
Output:  
7
```

Test 4 — Repeated pattern

```
Input:  
abab
```

```
Output:  
8
```

Test 5 — Palindrome

```
Input:  
abba
```

```
Output:  
9
```

Test 6 — Longer string

```
Input:  
banana
```

```
Output:
16
```

164. Search in BST

Topic: Binary Search Trees

Problem Statement

You are given the root of a binary search tree and an integer `val`. Find the node in the BST whose value equals `val` and return the subtree rooted at that node. If no such node exists, return an empty tree, printed as `[]`. Because of the ordering property of a BST, you do not need to look at every node. Start from the root: if the value is equal to the current node, you are done; if `val` is smaller, move to the left child; otherwise move to the right child. Stop when you find the node or reach a null pointer. The running time is $O(h)$, where h is the height of the tree. The subtree is printed in level-order with null for missing children.

Input / Output format: Line 1: the BST in level-order. Line 2: `val`. Output: the subtree rooted at `val` in level-order (trailing nulls removed), or `[]`.

Example

Input

```
4 2 7 1 3
2
```

Output

```
2 1 3
```

Explanation: Node 2 has children 1 and 3. Output: 2 1 3.

Constraints

- $1 \leq \text{number of nodes} \leq 5000$
- All node values are unique
- $1 \leq \text{node value, val} \leq 10^7$

Test Cases

Test 1 — Value not present

```
Input:
4 2 7 1 3
5

Output:
[]
```

Test 2 — Root value

```
Input:
4 2 7 1 3
4

Output:
4 2 7 1 3
```

Test 3 — Leaf value


```
Input:
4 2 7 1 3
3
```

```
Output:
3
```

Test 4 — Single node found

```
Input:
9
9
```

```
Output:
9
```

Test 5 — Single node missing

```
Input:
9
8
```

```
Output:
[]
```

Test 6 — Node with only a right child

```
Input:
5 3 8 null 4
3
```

```
Output:
3 null 4
```

165. Insert a Given Node in BST

Topic: Binary Search Trees

Problem Statement

You are given the root of a binary search tree and a value `val` that does not yet exist in the tree. Insert a new node with this value so that the tree remains a valid binary search tree, and print the resulting tree. There can be several valid ways to insert, but the simplest is to add the new node as a leaf. Start at the root; if `val` is smaller than the current node, go to the left child, otherwise go to the right child. When the child in the required direction is empty, attach the new node there. The insertion takes $O(h)$ time, where h is the height of the tree. The tree is printed in level-order with null for missing children and trailing nulls removed.

Input / Output format: Line 1: the BST in level-order. Line 2: `val`. Output: the tree after insertion in level-order.

Example

Input

```
4 2 7 1 3
5
```

Output

```
4 2 7 1 3 5
```

Explanation: 5 is larger than 4 and smaller than 7, so it becomes the left child of 7. Output: 4 2 7 1 3 5.

Constraints

- $1 \leq \text{number of nodes} \leq 10^4$
- All node values are unique and val is not in the tree

Test Cases

Test 1 — Insert a new minimum

```
Input:
4 2 7 1 3
0

Output:
4 2 7 1 3 null null 0
```

Test 2 — Insert a new maximum

```
Input:
4 2 7 1 3
10

Output:
4 2 7 1 3 null 10
```

Test 3 — Insert below the root

```
Input:
5
3

Output:
5 3
```

Test 4 — Insert to the right of a single node

```
Input:
5
8

Output:
5 null 8
```

Test 5 — Insert into a left-skewed tree

```
Input:
5 3 null 2
1

Output:
5 3 null 2 null 1
```

Test 6 — Insert between existing values

```
Input:
10 5 15 3 7 12 20
6

Output:
10 5 15 3 7 12 20 null null 6
```

166. Construct a BST from a Preorder Traversal

Topic: Binary Search Trees

Problem Statement

You are given the preorder traversal of a binary search tree with distinct values. Reconstruct the tree and print it in level-order. In a preorder traversal the root comes first, then the whole left subtree, then the whole right subtree. Because the tree is a BST, every value in the left subtree is smaller than the root and every value in the right subtree is larger. You can build the tree in linear time by keeping a global index into the array and passing an allowed range (lower and upper bound) with each recursive call: if the next value does not fit in the range, return null; otherwise create a node, then build its left subtree with a smaller upper bound and its right subtree with a larger lower bound.

Input / Output format: Line 1: n. Line 2: the preorder traversal. Output: the tree in level-order with 'null' for missing children (trailing nulls removed).

Example

Input

```
6
8 5 1 7 10 12
```

Output

```
8 5 10 1 7 null 12
```

Explanation: 8 is the root; 5 1 7 form the left subtree and 10 12 the right one. Output: 8 5 10 1 7 null 12.

Constraints

- $1 \leq n \leq 10^5$
- All values are distinct
- The sequence is the preorder of a valid BST

Test Cases

Test 1 — Single node

```
Input:
1
4

Output:
4
```

Test 2 — Increasing order (right skewed)

```
Input:
4
1 2 3 4

Output:
1 null 2 null 3 null 4
```

Test 3 — Decreasing order (left skewed)

```
Input:
4
4 3 2 1

Output:
4 3 null 2 null 1
```

Test 4 — Perfect tree

```
Input:
7
8 4 2 6 12 10 14

Output:
```

```
8 4 12 2 6 10 14
```

Test 5 — Two nodes

```
Input:
2
5 3

Output:
5 3
```

Test 6 — Mixed shape

```
Input:
5
10 5 15 12 20

Output:
10 5 15 null null 12 20
```

167. Best Time to Buy and Sell Stock

Topic: Dynamic Programming

Problem Statement

You are given an array `prices` where `prices[i]` is the price of a given stock on day `i`. You want to maximise your profit by choosing a single day to buy one share and a different, later day to sell that share. Return the maximum profit you can achieve from this single transaction. If no profit is possible because the price never rises after a lower price, return 0. You cannot sell before you buy. Traverse the prices once while keeping the minimum price seen so far; for

each day, the best profit if you sell today is the current price minus that minimum. Keep the maximum of these values. The solution runs in linear time using constant extra space.

Input / Output format: Line 1: n. Line 2: the n prices. Output: the maximum profit (0 if none).

Example

Input

```
6
7 1 5 3 6 4
```

Output

```
5
```

Explanation: Buy on day 1 at price 1 and sell on day 4 at price 6. Profit = 5.

Constraints

- $1 \leq n \leq 10^5$
- $0 \leq \text{prices}[i] \leq 10^4$

Test Cases

Test 1 — Prices only fall

```
Input :
```



```
5
7 6 4 3 1

Output:
0
```

Test 2 — Single day

```
Input:
1
5

Output:
0
```

Test 3 — Prices only rise

```
Input:
4
1 2 3 4

Output:
3
```

Test 4 — Constant prices

```
Input:
3
2 2 2

Output:
0
```

Test 5 — Minimum appears after the maximum

```
Input:
6
9 8 1 4 10 2

Output:
9
```

Test 6 — Best deal is in the middle

```
Input:
7
3 8 2 9 1 4 3

Output:
7
```

168. Best Time to Buy and Sell Stock II

Topic: Dynamic Programming

Problem Statement

You are given an array `prices` where `prices[i]` is the price of a stock on day `i`. On each day you may decide to buy and/or sell the stock, but you can hold at most one share at any time. You are allowed to buy and sell on the same day. Find and return the maximum profit you can achieve when the number of transactions is unlimited. The greedy insight is that every rise between two consecutive days can be captured separately: if `prices[i]` is greater than `prices[i - 1]`, add the difference to the profit. The sum of all positive daily differences equals the best possible total. The algorithm makes one pass over the array and needs constant extra space.

Input / Output format: Line 1: `n`. Line 2: the `n` prices. Output: the maximum profit.

Example

Input

```
6
7 1 5 3 6 4
```

Output

```
7
```

Explanation: Buy at 1, sell at 5 (profit 4); buy at 3, sell at 6 (profit 3). Total = 7.

Constraints

- $1 \leq n \leq 3 \cdot 10^4$
- $0 \leq \text{prices}[i] \leq 10^4$

Test Cases

Test 1 — Steadily rising

```
Input:
5
1 2 3 4 5

Output:
4
```

Test 2 — Falling prices

```
Input:
5
5 4 3 2 1

Output:
0
```

Test 3 — Single day

```
Input:
1
3

Output:
0
```

Test 4 — Constant prices

```
Input:
4
6 6 6 6

Output:
0
```

Test 5 — Zigzag

```
Input:
6
1 5 2 6 3 7

Output:
12
```

Test 6 — Two peaks

```
Input:
7
2 9 1 1 8 3 3

Output:
14
```

169. Best Time to Buy and Sell Stock III

Topic: Arrays

Problem Statement

You are given an array prices where prices[i] is the price of a given stock on day i. Find the maximum profit you can achieve if you may complete at most two transactions. A transaction consists of buying one share and selling it later, and you may not hold more than one share at a time, so you must sell the stock before you buy it again. Use four variables that are updated on each day: the best balance after the first buy, after the first sell, after the second buy and after the second sell. Each variable is the maximum of keeping its previous value or making the corresponding move from the previous stage. The final answer is the best balance after the second sell.

Input / Output format: Line 1: n. Line 2: the n prices. Output: the maximum profit with at most two transactions.

Example

Input

```
8
3 3 5 0 0 3 1 4
```

Output

```
6
```

Explanation: Buy at 0 and sell at 3 (profit 3), then buy at 1 and sell at 4 (profit 3). Total = 6.

Constraints

- $1 \leq n \leq 10^5$
- $0 \leq \text{prices}[i] \leq 10^5$

Test Cases

Test 1 — Only one useful transaction

```
Input:
5
1 2 3 4 5

Output:
4
```

Test 2 — Prices only fall

```
Input:
5
7 6 4 3 1

Output:
0
```

Test 3 — Single day

```
Input:
1
1

Output:
0
```

Test 4 — Two separate peaks

```
Input:
6
1 4 2 7 1 5

Output:
10
```

Test 5 — Two transactions beat one

```
Input:
4
1 5 4 6

Output:
6
```

Test 6 — Two equal opportunities

```
Input:
6
2 8 1 7 3 3

Output:
12
```

170. Non-overlapping Intervals

Topic: Arrays

Problem Statement

Given an array of intervals where $\text{intervals}[i] = [\text{start}, \text{end}]$, return the minimum number of intervals you need to remove so that the remaining intervals do not overlap. Two intervals overlap when they share a point in their interiors; intervals that only touch at an endpoint, such as $[1, 2]$ and $[2, 3]$, do not overlap. The classic greedy method is to sort the intervals by their end point. Keep the first interval, then walk through the sorted list: if the next interval starts at or after the end of the last kept interval, keep it and update the end; otherwise remove it. The answer is the total number of intervals minus the number kept.

Input / Output format: Line 1: n . Next n lines: 'start end' for each interval. Output: the minimum number of intervals to remove.

Example

Input

```
4
1 2
2 3
3 4
1 3
```

Output

```
1
```


Explanation: Removing $[1, 3]$ leaves $[1,2]$, $[2,3]$, $[3,4]$, which do not overlap. Output: 1.

Constraints

- $1 \leq n \leq 10^5$
- $-5 \cdot 10^4 \leq \text{start} < \text{end} \leq 5 \cdot 10^4$

Test Cases

Test 1 — Already non-overlapping

```
Input:
3
1 2
3 4
5 6

Output:
0
```

Test 2 — Single interval

```
Input:
1
0 5

Output:
0
```

Test 3 — All intervals identical

```
Input:
3
1 2
1 2
1 2
```

```
Output:  
2
```

Test 4 — Nested intervals

```
Input:  
4  
1 10  
2 3  
4 5  
6 7  
  
Output:  
1
```

Test 5 — Chain of overlaps

```
Input:  
4  
1 4  
2 5  
3 6  
4 7  
  
Output:  
2
```

Test 6 — Negative endpoints

```
Input:  
3  
-5 -1  
-3 0  
-2 2  
  
Output:  
2
```

171. Longest Common Prefix

Topic: Tries

Problem Statement

Given a list of n strings, find the longest string that is a prefix of every string in the list. A prefix of a string is a leading part of it, such as `fl` for `flower`. If the strings have no common prefix, print `-1` instead of an empty string. For the strings `flower`, `flow` and `flight`, the longest common prefix is `fl`. One approach compares the first string with each other string and shortens the candidate prefix until it matches. A Trie gives another method: insert all the strings and walk down from the root while the current node has exactly one child and no string ends there; the characters on that walk form the prefix.

Input / Output format: Line 1: n . Next n lines: one string each. Output: the longest common prefix, or `-1` if there is none.

Example

Input

```
3
flower
flow
flight
```

Output

```
fl
```

Explanation: All three start with fl but the third letters differ (o, o, i). Output: fl.

Constraints

- $1 \leq n \leq 200$
- $1 \leq |\text{string}| \leq 200$
- Strings consist of lowercase English letters

Test Cases

Test 1 — No common prefix

```
Input:
3
dog
racecar
car

Output:
-1
```

Test 2 — Single string

```
Input:
1
capgemini

Output:
capgemini
```

Test 3 — Identical strings

```
Input:
3
test
```

```
test
test

Output:
test
```

Test 4 — One string is the prefix of the others

```
Input:
3
inter
internet
interval

Output:
inter
```

Test 5 — Common prefix of one letter

```
Input:
2
abc
ade

Output:
a
```

Test 6 — Shortest string limits the prefix

```
Input:
3
abcdef
abc
abcxyz

Output:
abc
```

172. One Away

Topic: Strings

Year / Source: Sep 2022 (Code360 on-campus experience, Capgemini Consulting India — coding round 2)

⚠ **Note:** Corresponds to the Code360 problem 'One Away' reported in a Sep 2022 Capgemini experience. Treating identical strings as 'one away' follows the standard definition (zero or one edits).

Problem Statement

You are given two strings, A and B. Determine whether they are at most one edit away from each other. An edit is one of three operations: inserting one character into a string, removing one character from a string, or replacing one character by another. Two identical strings need zero edits, so they also count as one away. For example, pale and ple are one edit apart, because removing the letter a from pale gives ple, while pale and bake need two replacements. Compare the lengths first: if they differ by more than one the answer is false. Otherwise walk both strings with two pointers, allow a single mismatch, and skip the extra character of the longer string.

Input / Output format: Line 1: string A. Line 2: string B. Output: true if the strings are zero or one edit apart, otherwise false.

Example

Input

```
pale
ple
```

Output

```
true
```

Explanation: Removing the letter a from pale gives ple: one edit. Output: true.

Constraints

- $1 \leq |A|, |B| \leq 10^5$
- Strings consist of lowercase English letters

Test Cases

Test 1 — Replace one character

```
Input:
pale
bale

Output:
true
```

Test 2 — Two edits needed

```
Input:
pale
bake

Output:
false
```

Test 3 — Insert one character

```
Input:
abc
abxc

Output:
true
```

Test 4 — Identical strings

```
Input:
hello
hello

Output:
true
```

Test 5 — Lengths differ by two

```
Input:
ab
abcd

Output:
false
```

Test 6 — Single characters

```
Input:
a
b

Output:
true
```

173. Count Even Odd

Topic: Hashing

Year / Source: Feb 2024 (Code360 on-campus experience — online coding round: 'Count Even Odd')

⚠ **Note:** Part 1 (elements with an odd frequency) comes from the reported Code360 statement; the even-frequency second output is inferred from the title 'Count Even Odd'. Verify with the original statement.

Problem Statement

You are given an array of integers `ARR` of size `N`. Your task is to find two things: the number of distinct elements that occur an odd number of times in the array, and the number of distinct elements that occur an even number of times. Each distinct value is counted once, according to its total frequency in the whole array. For example, in the array `1 1 2 3 3 3`, the value `1` occurs twice (even), the value `2` once (odd) and the value `3` three times (odd), so the answer is `2 1`. Build a frequency map with a hash table in one pass, then iterate over its values and increase the matching counter depending on the parity of every frequency.

Input / Output format: Line 1: `N`. Line 2: the `N` integers. Output: two integers 'odd even', the numbers of distinct values with odd and even frequency.

Example

Input

```
6
1 1 2 3 3 3
```

Output

```
2 1
```

Explanation: 1 occurs 2 times (even), 2 occurs once (odd), 3 occurs 3 times (odd). Output: 2 1.

Constraints

- $1 \leq N \leq 10^5$
- $-10^9 \leq \text{ARR}[i] \leq 10^9$

Test Cases

Test 1 — All distinct

```
Input:
5
4 5 6 7 8

Output:
5 0
```

Test 2 — Single element

```
Input:
1
9

Output:
1 0
```

Test 3 — All equal, even count

```
Input:
4
2 2 2 2
```

```
Output:  
0 1
```

Test 4 — All equal, odd count

```
Input:  
5  
3 3 3 3 3  
  
Output:  
1 0
```

Test 5 — Negative values

```
Input:  
6  
-1 -1 0 0 0 5  
  
Output:  
2 1
```

Test 6 — Mixed frequencies

```
Input:  
10  
1 2 2 3 3 3 4 4 4 4  
  
Output:  
2 2
```

174. Group Anagrams

Topic: Hashing

Year / Source: May 2024 (Code360 on-campus experience — coding round: 'Group Anagrams')

Problem Statement

Given an array of strings, group the anagrams together. Two strings are anagrams if one can be formed by rearranging the letters of the other, using every letter exactly once. For example, eat, tea and ate are anagrams of each other. Print each group on its own line, with the words of the group separated by spaces in their original order. The groups must be printed in the order in which their first word appears in the input. Compute a key for every word by sorting its letters, or by counting letter frequencies, store the words in a hash map that maps each key to a list, and finally print the lists. The time complexity is $O(n \cdot k \log k)$ for n words of length k .

Input / Output format: Line 1: n . Next n lines: one word each. Output: one line per group of anagrams (original order inside the group, groups ordered by first appearance).

Example

Input

```
6
eat
tea
tan
ate
nat
bat
```

Output

```
eat tea ate
tan nat
bat
```

Explanation: eat, tea and ate share the key aet; tan and nat share ant; bat is alone. Output: three lines: 'eat tea ate', 'tan nat', 'bat'.

Constraints

- $1 \leq n \leq 10^4$
- $1 \leq |\text{word}| \leq 100$
- Words consist of lowercase English letters

Test Cases

Test 1 — Single word

```
Input:
1
abc

Output:
abc
```

Test 2 — No anagram pair

```
Input:
3
abc
def
ghi

Output:
abc
def
ghi
```

Test 3 — All words are anagrams

```
Input:
3
listen
silent
enlist

Output:
listen silent enlist
```

Test 4 — Repeated identical words

```
Input:
4
ab
ba
ab
ba

Output:
ab ba ab ba
```

Test 5 — Different lengths never group

```
Input:
4
a
ab
ba
aa

Output:
a
ab ba
aa
```

Test 6 — Groups interleaved

```
Input:
6
rat
```

```
cab
tar
bac
art
abc
```

```
Output:
rat tar art
cab bac abc
```

175. Find the Elements Removed from Nested Arrays ($A \supset B \supset C$)

Topic: Arrays

Year / Source: Jun 2026 (Glassdoor candidate report, Airoli — coding round)

⚠ **Note:** Reconstructed from a June 2026 Glassdoor report that described the task as finding the element in A that is not in B and the element in B that is not in C. The exact input format may differ in the real test.

Problem Statement

You are given three integer arrays A, B and C. Array B is obtained from A by deleting exactly one element and then possibly shuffling the rest, and array C is obtained from B by deleting exactly one element and shuffling as well. So the lengths are N, N - 1 and N - 2. Find the element that is present in A but missing from B, and the element that is present in B but missing from C, and print them in this order. The arrays may contain duplicate values, so a set difference is not enough. Use the sum trick, where $\text{sum}(A) - \text{sum}(B)$ is the first element and $\text{sum}(B) - \text{sum}(C)$ is the second, or use frequency maps to find where the counts differ.

Input / Output format: Line 1: N. Line 2: array A (N values). Line 3: array B (N-1 values). Line 4: array C (N-2 values). Output: 'x y' where x is missing from B and y is missing from C.

Example

Input

```
5
4 8 1 3 7
7 4 3 1
4 1 7
```

Output

```
8 3
```

Explanation: A minus B leaves 8; B minus C leaves 3. Output: 8 3.

Constraints

- $3 \leq N \leq 10^5$
- $-10^9 \leq \text{values} \leq 10^9$
- Arrays may contain duplicates

Test Cases

Test 1 — Basic

```
Input:
4
1 2 3 4
4 2 1
1 4

Output:
3 2
```


Test 2 — Smallest size

```
Input:
3
5 6 7
7 5
5

Output:
6 7
```

Test 3 — Duplicates in A

```
Input:
5
2 2 3 3 4
2 3 3 4
2 3 4

Output:
2 3
```

Test 4 — Same value removed twice

```
Input:
5
9 9 9 1 2
9 9 1 2
9 1 2

Output:
9 9
```

Test 5 — Shuffled arrays

```
Input:
5
10 20 30 40 50
50 10 40 20
```

```
20 50 40
```

```
Output:  
30 10
```

Test 6 — Negative numbers

```
Input:  
4  
-3 -1 0 6  
6 -3 0  
0 6
```

```
Output:  
-1 -3
```

176. Longest Consecutive Path in a Grid

Topic: Arrays

Year / Source: Jun 2026 (Glassdoor candidate report, Airoli — coding round)

⚠ **Note:** Reconstructed from the June 2026 Glassdoor report (longest path of consecutive numbers in an $N \times N$ grid, 4-directional moves). Whether the path may also step to value -1 is not stated in the report; this version uses $+1$ only.

Problem Statement

You are given an $N \times N$ grid of integers. You may start from any cell and move in the four directions: up, down, left or right. A move is allowed only if the value of the destination cell is exactly one more than the value of the current cell. Find the length of the longest path that can be made under this rule, counting the cells that are visited, including the starting cell. A path never needs to revisit a cell, because values strictly increase. Use depth-first search with memoisation: the best path that starts at a cell is one plus the best of its valid neighbours. Store the value for each cell so that it is computed only once, and take the maximum over all cells.

Input / Output format: Line 1: N. Next N lines: N integers each. Output: the length (number of cells) of the longest path of consecutive increasing values.

Example

Input

```
3
1 2 9
8 3 4
7 6 5
```

Output

```
8
```

Explanation: The path $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 6 \rightarrow 7 \rightarrow 8$ winds through the grid and visits 8 cells; 9 is not adjacent to 8. Output: 8.

Constraints

- $1 \leq N \leq 500$
- $1 \leq \text{grid}[i][j] \leq 10^6$
- Values may repeat

Test Cases

Test 1 — Single cell

```
Input:
1
5

Output:
1
```

Test 2 — All values equal

```
Input:
2
3 3
3 3

Output:
1
```

Test 3 — Increasing row

```
Input:
3
1 2 3
9 9 9
9 9 9

Output:
3
```

Test 4 — Two equal-length paths

```
Input:
2
1 2
2 1

Output:
2
```

Test 5 — Values differ by two

```
Input:
2
1 3
5 7

Output:
1
```

Test 6 — Vertical path

```
Input:
3
4 9 9
5 9 9
6 7 8

Output:
6
```

177. Move All '#' to the Front of a String

Topic: Strings

Year / Source: 2025–26 Exceller cycle (PrepInsta 'Capgemini Exceller Coding Questions')

Problem Statement

You are given a string that contains the character # in some positions together with other characters. Move all the # characters to the front of the string, so that they come first, while keeping the relative order of all the other characters exactly the same as in the input. The length of the string does not change. For example, the string Ab#c#d becomes ##Abcd. An efficient in-place style solution scans the string from right to left with two pointers: copy each non-# character towards the end of the array using a write pointer, then fill the remaining

positions at the front with #. A simpler approach counts the number of # characters, builds a new string of hashes, and appends the other characters in order.

Input / Output format: Line 1: the string. Output: the string with every '#' moved to the front.

Example

Input

```
Ab#c#d
```

Output

```
##Abcd
```

Explanation: There are two # signs; the remaining characters A b c d keep their order.
Output: ##Abcd.

Constraints

- $1 \leq |s| \leq 10^5$
- s has printable characters without spaces

Test Cases

Test 1 — Hashes at the end

```
Input:
```

```
abc###
```

```
Output:  
###abc
```

Test 2 — Hashes at the start

```
Input:  
###abc
```

```
Output:  
###abc
```

Test 3 — No hash at all

```
Input:  
hello
```

```
Output:  
hello
```

Test 4 — Only hashes

```
Input:  
####
```

```
Output:  
####
```

Test 5 — Alternating

```
Input:  
a#b#c#d
```

```
Output:
```

```
###abcd
```

Test 6 — Digits and symbols

```
Input:  
1#2#3$4  
  
Output:  
##123$4
```

178. Run-Length Encoding of a String

Topic: Strings

Year / Source: 2025–26 Exceller cycle (PrepInsta 'Capgemini Exceller Coding Questions')

⚠ **Note:** The PrepInsta sample shows a2b4e4f2g3; the handling of single characters (count 1 written explicitly) is a convention chosen here — confirm against the platform's sample if it differs.

Problem Statement

In the online test the student is given a string in which several characters are repeated consecutively. The task is to reduce the size of the string using a simple mathematical logic: every maximal group of the same character is replaced by the character followed by the number of times it occurs in the group. For example, aabbbbbeeeffggg is encoded as a2b4e4f2g3. A character that occurs once is written with the count 1, so abc is encoded as a1b1c1. Only consecutive equal characters are grouped, therefore aabaa becomes a2b1a2. Scan the string once, keep the current character and its count, and append the pair to the answer whenever the character changes or the string ends.

Input / Output format: Line 1: the string. Output: its run-length encoding (character followed by the run length, for every run).

Example

Input

```
aabbbbbeeefggg
```

Output

```
a2b4e4f2g3
```

Explanation: Runs: aa, bbbb, eeee, ff, ggg. Output: a2b4e4f2g3.

Constraints

- $1 \leq |s| \leq 10^5$
- s consists of lowercase English letters

Test Cases

Test 1 — Single character

```
Input:
a

Output:
a1
```

Test 2 — No repeated characters

```
Input:  
abc
```

```
Output:  
a1b1c1
```

Test 3 — One long run

```
Input:  
zzzzzzzzzzzzzz
```

```
Output:  
z12
```

Test 4 — Non-consecutive repeats stay separate

```
Input:  
aabaa
```

```
Output:  
a2b1a2
```

Test 5 — Alternating characters

```
Input:  
ababab
```

```
Output:  
a1b1a1b1a1b1
```

Test 6 — Run of ten or more

```
Input:  
aaaaaaaaabbbb
```

```
Output:  
a10b3
```

179. Spiral Matrix Traversal

Topic: Arrays

Year / Source: 2025–26 Exceller cycle (PrepInsta 'Capgemini Exceller Coding Questions')

Problem Statement

You are given a matrix with m rows and n columns. Print all of its elements in spiral order, starting from the top-left cell, moving right along the first row, then down the last column, then left along the last row, then up the first column, and continue spiralling inwards until every element has been printed exactly once. The matrix does not have to be square, so single rows, single columns and rectangles must be handled correctly. Maintain four boundaries: top, bottom, left and right. After traversing a side, shrink the corresponding boundary. Before traversing the bottom row and the left column, check that the boundaries have not crossed, otherwise elements would be printed twice.

Input / Output format: Line 1: m and n . Next m lines: n integers each. Output: the elements in spiral order, space separated.

Example

Input

```
3 3  
1 2 3  
4 5 6  
7 8 9
```

Output

```
1 2 3 6 9 8 7 4 5
```

Explanation: Right along the top: 1 2 3; down: 6 9; left: 8 7; up: 4; then the centre 5. Output: 1 2 3 6 9 8 7 4 5.

Constraints

- $1 \leq m, n \leq 100$
- $-10^9 \leq \text{matrix}[i][j] \leq 10^9$

Test Cases

Test 1 — Rectangle 3x4

```
Input:
3 4
1 2 3 4
5 6 7 8
9 10 11 12

Output:
1 2 3 4 8 12 11 10 9 5 6 7
```

Test 2 — Single row

```
Input:
1 4
1 2 3 4

Output:
1 2 3 4
```

Test 3 — Single column

```
Input:
4 1
1
2
3
4

Output:
1 2 3 4
```

Test 4 — Single cell

```
Input:
1 1
7

Output:
7
```

Test 5 — Tall rectangle 4x2

```
Input:
4 2
1 2
3 4
5 6
7 8

Output:
1 2 4 6 8 7 5 3
```

Test 6 — Negative values

```
Input:
```

```
2 2
-1 -2
-3 -4
```

```
Output:
-1 -2 -4 -3
```

180. Evaluate the Polynomial $a^3 + a^2b + 2a^2b + 2ab^2 + ab^2 + b^3$

Topic: Mathematics

Year / Source: 2025–26 Exceller cycle (PrepInsta 'Capgemini Exceller Coding Questions')

⚠ **Note:** The PrepInsta/Scribd text lists the values in the order a, b (and mentions c); only a and b appear in the expression, so c is not used here. Verify against the platform's original wording.

Problem Statement

Write a function that evaluates the expression $a^3 + a^2b + 2a^2b + 2ab^2 + ab^2 + b^3$ for the given integer values of a and b, and print the result. The reported test asks the student to accept the values, apply the formula exactly as written, and print the number. Combining the like terms shows that the expression equals $a^3 + 3a^2b + 3ab^2 + b^3$, which is the expansion of $(a + b)^3$ cubed, so a shorter solution computes the sum of a and b and multiplies it by itself three times. Either way, use integer arithmetic. Be careful with negative values, since odd powers keep the sign, and choose a type wide enough for the largest input to avoid overflow.

Input / Output format: Line 1: integers a and b. Output: the value of the expression.

Example

Input

```
2 3
```

Output

```
125
```

Explanation: $8 + 12 + 24 + 36 + 18 + 27 = 125$, which equals $(2 + 3)^3$. Output: 125.

Constraints

- $-1000 \leq a, b \leq 1000$

Test Cases

Test 1 — Both zero

```
Input:
0 0

Output:
0
```

Test 2 — One zero

```
Input:
0 4

Output:
64
```

Test 3 — Equal values

```
Input:
1 1

Output:
8
```

Test 4 — Negative values

```
Input:
-2 -3

Output:
-125
```

Test 5 — Opposite values cancel

```
Input:
5 -5

Output:
0
```

Test 6 — Large values

```
Input:
1000 1000

Output:
8000000000
```

181. Dealership Tyres

Topic: Mathematics

Year / Source: 2025–26 Exceller cycle (PrepInsta 'Capgemini Exceller Coding Questions')

⚠ **Note:** Reconstructed from the one-line PrepInsta description ($\text{cars} \times 4 + \text{bikes} \times 2$). The original may phrase it differently (for example as a word problem); verify.

Problem Statement

A vehicle dealership has sold some cars and some bikes. Every car has four tyres and every bike has two tyres. Given the number of cars and the number of bikes, calculate and print the total number of tyres of all the vehicles. Spare tyres are not counted. For example, a dealership with 3 cars and 2 bikes has $3 \times 4 + 2 \times 2 = 16$ tyres. The problem only tests basic arithmetic and input handling, so the solution is a single expression: multiply the number of cars by four, multiply the number of bikes by two, and add the two products. Use a 64-bit integer type when the counts can be large, because four times a big count may exceed the range of a 32-bit integer.

Input / Output format: Line 1: the number of cars and the number of bikes. Output: the total number of tyres.

Example

Input

```
3 2
```

Output

```
16
```

Explanation: 3 cars have 12 tyres and 2 bikes have 4. Output: 16.

Constraints

- $0 \leq \text{cars, bikes} \leq 10^9$

Test Cases

Test 1 — Cars only

```
Input:
5 0

Output:
20
```

Test 2 — Bikes only

```
Input:
0 7

Output:
14
```

Test 3 — No vehicles

```
Input:
0 0

Output:
0
```

Test 4 — Equal numbers

```
Input:  
4 4
```

```
Output:  
24
```

Test 5 — One of each

```
Input:  
1 1
```

```
Output:  
6
```

Test 6 — Large counts

```
Input:  
1000000000 1000000000
```

```
Output:  
6000000000
```

182. Print Palindrome Numbers in a Range

Topic: Mathematics

Year / Source: 2025–26 (technical-interview coding questions reported by candidates)

⚠ **Note:** Reported in a 2025 GeeksforGeeks on-campus technical-interview experience as 'a program to print palindrome numbers'; the exact range form is assumed.

Problem Statement

Given two integers L and R, print every palindromic number in the inclusive range [L, R] in increasing order. A number is a palindrome if it reads the same from left to right and from right to left, for example 121 or 5. All single-digit numbers are palindromes. If the range contains no palindrome, print []. To test a single number, reverse its digits with repeated division and modulo by ten and compare the reversed value with the original, or convert it to a string and compare the string with its reverse. Loop over every number in the range and collect those that pass the check. The complexity is proportional to the size of the range times the number of digits.

Input / Output format: Line 1: L and R. Output: all palindromic numbers in [L, R], space separated, or [].

Example

Input

```
100 150
```

Output

```
101 111 121 131 141
```

Explanation: Among 100 to 150 the palindromes are 101, 111, 121, 131 and 141. Output: 101 111 121 131 141.

Constraints

- $1 \leq L \leq R \leq 10^6$

Test Cases

Test 1 — Single digits

```
Input:
1 9

Output:
1 2 3 4 5 6 7 8 9
```

Test 2 — Range with no palindrome

```
Input:
10 10

Output:
[]
```

Test 3 — Range with a single value

```
Input:
121 121

Output:
121
```

Test 4 — Crossing digit lengths

```
Input:
95 111

Output:
99 101 111
```

Test 5 — Range starting at 1

```
Input:
1 30

Output:
1 2 3 4 5 6 7 8 9 11 22
```

Test 6 — Range with no palindrome, larger

```
Input:
1230 1240

Output:
[]
```

183. Reverse a String

Topic: Strings

Year / Source: 2025–26 (technical-interview coding questions reported by candidates)

⚠ **Note:** Reported as an example of small coding questions that appear in Capgemini technical interviews (2025–26 candidate reports).

Problem Statement

Given a string, print the string in reverse order, so that the last character becomes the first, the second last becomes the second, and so on. For example, the string Capgemini becomes inimegpaC. The reversal must not change the characters themselves; only their positions change, and the case of every letter is kept. In an interview the built-in reverse function is usually not allowed, so the expected method uses two pointers: place one pointer at the start

and one at the end, swap the characters they point to, move the pointers towards each other, and stop when they meet in the middle. This runs in linear time and needs constant extra space if the string can be modified in place.

Input / Output format: Line 1: the string. Output: the reversed string.

Example

Input

```
Capgemini
```

Output

```
inimegpaC
```

Explanation: The characters read from right to left are i n i m e g p a C. Output: inimegpaC.

Constraints

- $1 \leq |s| \leq 10^5$
- s has printable characters and no leading or trailing spaces

Test Cases

Test 1 — Single character

```
Input :  
x
```

```
Output:  
x
```

Test 2 — Palindrome stays the same

```
Input:  
madam  
  
Output:  
madam
```

Test 3 — Digits and letters

```
Input:  
abc123  
  
Output:  
321cba
```

Test 4 — Sentence with a space

```
Input:  
hello world  
  
Output:  
dlrow olleh
```

Test 5 — Two characters

```
Input:  
ab  
  
Output:
```



```
ba
```

Test 6 — Mixed case and symbols

```
Input :  
A#b$C  
  
Output :  
C$b#A
```

184. Fibonacci Series

Topic: Mathematics

Year / Source: 2025–26 (technical-interview coding questions reported by candidates)

⚠ **Note:** Reported as an example of small coding questions asked in Capgemini technical interviews (2025–26 candidate reports).

Problem Statement

Given an integer n , print the first n terms of the Fibonacci series. The series starts with 0 and 1, and every next term is the sum of the previous two terms, so the series is 0, 1, 1, 2, 3, 5, 8 and so on. If n is 1, print only 0. The values are separated by single spaces in the output. An iterative solution keeps two variables for the last two terms, prints the current term, and then updates the variables in a loop that runs n times, which takes linear time and constant extra space. A recursive solution is shorter but recomputes the same terms many times and takes exponential time unless it is memoised.

Input / Output format: Line 1: n . Output: the first n Fibonacci numbers, space separated.

Example

Input

```
7
```

Output

```
0 1 1 2 3 5 8
```

Explanation: The first seven terms are 0 1 1 2 3 5 8. Output: 0 1 1 2 3 5 8.

Constraints

- $1 \leq n \leq 45$

Test Cases

Test 1 — Single term

```
Input:
1

Output:
0
```

Test 2 — Two terms

```
Input:
2
```

```
Output:  
0 1
```

Test 3 — Three terms

```
Input:  
3  
  
Output:  
0 1 1
```

Test 4 — Ten terms

```
Input:  
10  
  
Output:  
0 1 1 2 3 5 8 13 21 34
```

Test 5 — Larger n

```
Input:  
20  
  
Output:  
0 1 1 2 3 5 8 13 21 34 55 89 144 233 377 610 987 1597 2584 4181
```

Test 6 — Maximum n

```
Input:  
45  
  
Output:  
0 1 1 2 3 5 8 13 21 34 55 89 144 233 377 610 987 1597 2584 4181
```

```
6765 10946 17711 28657 46368 75025 121393 196418 317811 514229 832040
1346269 2178309 3524578 5702887 9227465 14930352 24157817 39088169
63245986 102334155 165580141 267914296 433494437 701408733
```

185. Swap Two Numbers Without a Temporary Variable

Topic: Mathematics

Year / Source: 2025–26 (technical-interview coding questions reported by candidates)

⚠ **Note:** Reported as an example of small coding questions asked in Capgemini technical interviews (2025–26 candidate reports).

Problem Statement

Given two integers a and b , swap their values without using a third temporary variable, and print the values after the swap in the order a followed by b . For example, if a is 5 and b is 9, the output is 9 5. There are two classic methods. The arithmetic method sets $a = a + b$, then $b = a - b$, then $a = a - b$. The bitwise method uses exclusive OR: $a = a \oplus b$, $b = a \oplus b$, $a = a \oplus b$. Both methods work with negative numbers and with equal values, but the arithmetic method may overflow a fixed-size integer type when the sum is very large, so the XOR method is safer in languages such as C and Java.

Input / Output format: Line 1: integers a and b . Output: the values of a and b after swapping, separated by a space.

Example

Input

```
5 9
```

Output

```
9 5
```

Explanation: After the swap a holds 9 and b holds 5. Output: 9 5.

Constraints

- $-10^9 \leq a, b \leq 10^9$

Test Cases

Test 1 — Equal values

```
Input:
4 4

Output:
4 4
```

Test 2 — One value is zero

```
Input:
0 7

Output:
7 0
```

Test 3 — Negative values

```
Input:  
-3 8
```

```
Output:  
8 -3
```

Test 4 — Both negative

```
Input:  
-5 -10
```

```
Output:  
-10 -5
```

Test 5 — Both zero

```
Input:  
0 0
```

```
Output:  
0 0
```

Test 6 — Extreme values

```
Input:  
1000000000 -1000000000
```

```
Output:  
-1000000000 1000000000
```
